A Tutorial on CJR: Syntax, Semantics, and Separation Logic

Mechanized development in `theories/`

Abstract

We present CJR (Cangjie Radical), the performance-parity core of a Cangjie-like language: unboxed structs, word-addressed `CPointer` blocks, and final classes, together with `func`, `if`, and `while`. This note is a reading guide to the Coq/Iris mechanization. We give every concept twice—once as the definition that is actually checked, and once as ordinary mathematics with the same content—and we follow each program-logic rule by an account of what it is for. Finally, we walk through the examples that exercise the rules in order, and we conclude with a linked-list development, an array library, and an array list, each with the usual equations of that data structure.

Contents

1 What this development is **6**

2 Tool structure and workflow **6**

3 Syntax **7**
- 3.1 Concrete syntax . . . 7
- 3.2 From Cangjie to the Coq core . . . 8
- 3.3 Locations, identifiers, and types . . . 9
- 3.4 Values and expressions . . . 10
- 3.5 Two kinds of binding . . . 11

4 Machine state **12**
- 4.1 Three regions, one counter . . . 12
- 4.2 Well-formed states . . . 13

5 Small-step semantics **14**
- 5.1 Head reduction . . . 14
- 5.2 Evaluation contexts . . . 15
- 5.3 Determinism . . . 16

6 Separation logic **16**
- 6.1 What is owned . . . 16
- 6.2 Weakest precondition, in this instantiation . . . 17
- 6.3 Structural rules . . . 17
- 6.4 Pure rules . . . 18
- 6.5 Stack rules . . . 19
- 6.6 Raw-heap rules . . . 20
- 6.7 Object rules . . . 21
- 6.8 Exclusivity . . . 21

7 Main theorems **22**
- 7.1 Head reduction is deterministic . . . 22
- 7.2 Well-formedness is an invariant of the verified steps . . . 22
- 7.3 Adequacy . . . 22

8 The examples **23**
- 8.1 Arithmetic: `arith.v` . . . 23
- 8.2 Stack structs: `struct_upd.v` . . . 23
- 8.3 One raw cell: `cell.v` . . . 24
- 8.4 Swap and aliasing: `swap.v` . . . 25
- 8.5 Indexed blocks: `sum_block.v` . . . 26
- 8.6 Final classes: `class_upd.v` . . . 26
- 8.7 Framing across a call: `frame_call.v` . . . 27
- 8.8 A hand-written vtable: `vtable.v` . . . 28

9 A linked list **29**
9.1 Why a node is an object with a boolean tag . . . 29
9.2 The representation predicate . . . 31
9.3 What each operation establishes . . . 32
9.4 Adding an element and removing it . . . 33

10 In-place list reversal **33**
10.1 The representation being reused . . . 34
10.2 The client specification . . . 34
10.3 The iteration definition: `rev_step` . . . 34
10.4 The loop definition: `rev_loop` . . . 35
10.5 The function body: `rev_body` . . . 35
10.6 The call wrapper: `call_reverse` . . . 35
10.7 Loading a local: `sld` . . . 35
10.8 Assigning a local: `sas` . . . 36
10.9 The iteration after substitution: `rev_step_at` . . . 36
10.10 The loop after substitution: `rev_loop_at` . . . 36
10.11 Iteration substitution: statement and proof . . . 36
10.12 Loop substitution: statement and proof . . . 37
10.13 The loop invariant: `rev_inv` . . . 37
10.14 Append right identity: statement and induction proof . . . 37
10.15 Append associativity: statement and proof . . . 38
10.16 Reversing append: statement and proof . . . 38
10.17 Reversing cons: statement and proof . . . 38
10.18 The invariant equation after one step . . . 38
10.19 The invariant equation at an empty suffix . . . 39
10.20 Double reversal of mathematical lists: proof by induction . . . 39
10.21 Recovering the accumulator contents from the exit equation . . . 39
10.22 Opening a cons cell: definition and proof . . . 39
10.23 Closing the new accumulator: definition and proof . . . 40
10.24 One iteration: the full ownership proof . . . 40
10.25 The exit assertion: `rev_done` . . . 42
10.26 The loop theorem: induction and both branches . . . 42
10.27 Packaging the invariant: `wp_rev_loop` . . . 43
10.28 The complete function proof . . . 43
10.29 A client equation: reversing twice . . . 45
10.30 The double-reversal client theorem and its proof . . . 45
10.31 The printed Cangjie program . . . 45
10.32 Executable tests and what they check . . . 47

11 Arrays **48**
11.1 The library . . . 49
11.2 The representation . . . 50
11.3 Allocation of n words . . . 50
11.4 What each operation establishes . . . 51
11.5 The array equations . . . 51

12 Array lists **52**
12.1 The library . . . 52
12.2 The representation . . . 55
12.3 Copy and growth . . . 55
12.4 What each operation establishes . . . 56
12.5 The array-list equations . . . 57

13 Quicksort **58**
13.1 The functional algorithm . . . 58
13.2 Sortedness and permutation . . . 59
13.3 The partition invariant . . . 60
13.4 The length, get, and set interface . . . 63
13.5 The program . . . 64
13.6 The two instances . . . 67
13.7 The printed programs . . . 67

14 Hash maps **72**
14.1 Storage and the representation invariant . . . 72
14.2 Remainders and wrapped indexes . . . 73
14.3 The bounded probe and its proof . . . 73
14.4 Insertion and overwrite . . . 74
14.5 Rehashing: why allocation alone is insufficient . . . 75
14.6 Removal, queries, and client proofs . . . 75
14.7 Executable validation and scope . . . 76
14.8 The checked proof scripts . . . 76

15 Runtime comparison with the Cangjie standard library **101**
15.1 Method and comparable workloads . . . 101
15.2 Measured results . . . 102
15.3 Interpretation, limitations, and reproduction . . . 105

16 Most frequent integer: paired sort and scan **105**
16.1 Specification in elementary mathematics . . . 106
16.2 Gallina specification . . . 106
16.3 Algorithm walkthrough and pseudocode . . . 107
16.4 Formal CJR implementation and emitted Cangjie . . . 107
16.5 Correctness argument and checked components . . . 113
16.6 Testing and comparative benchmarks . . . 116

17 What is deliberately missing **119**

18 Conclusion **119**

19 Reading list **121**

A Limitations and target-code risks **122**
A.1 Fresh memory is zero in the proofs, but not in generated code . . . 122
A.2 HashMap growth does not honor its minimum-capacity argument . . . 122
A.3 HashMap residue computation can take impractical time . . . 123

References **123**

1 What this development is

We take as our starting point a deliberately small core. CJR is sequential, monomorphic, and first-order; it has code pointers, so that a vtable can be written by hand. It has no generics, no inheritance, no `enum`/`match`, and no concurrency. Features of a larger source language that fall outside this cut are treated as sugar that must lower into the core at the same time and space cost. What follows defines that core. We do not define the lowering, a garbage collector, or byte-level layout.

Safety, in this development, is not a type system. The grammar `ty` records the types one intends to use, but the operational semantics is untyped. A program is safe when a weakest-precondition proof exists; adequacy then says that, from the empty machine, execution does not get stuck.

The Coq sources live under `theories/`. The language is an Iris evaluation-context language [6, 5], so the frame rule and the bind rule are Iris's, rather than those of a separate Hoare calculus. The overall shape follows HeapLang, the example language shipped with the Iris Coq development [9].

2 Tool structure and workflow

The intended workflow starts with two related inputs. The first is an informal description of a data structure: its abstract contents, a concrete representation, and the algorithms for its operations. The second is a set of properties that should validate the implementation. These include the main representation-correctness claims (for example, that an implementation of list insertion produces the abstract list with the new element) and equational client properties (for example, that adding an element and then removing it restores the original list). Such laws are separate obligations; they do not follow merely from writing down the operations.

An engineer or an LLM can help translate the informal material into formal definitions and theorem statements. The formal development gives a mathematical model of the abstract structure, a representation predicate connecting that model to the CJR heap, executable CJR expression trees for the operations, and specifications stated as weakest preconditions. The translation is a proposal that must be reviewed: informal text does not become a theorem until its meaning has been made explicit in Rocq/Coq.

The proofs use Iris's separation logic over the small-step semantics of CJR. The Iris weakest-precondition rules describe how each operation transforms owned heap resources and establishes its postcondition. The proved implementation-correctness theorems connect the concrete representation to the abstract model; algebraic laws can then be proved by composing those verified operations. Rocq/Coq's kernel checks the resulting proof terms. In this repository the development is built with Coq 8.20.1 and Iris 4.4.0. The language is called Gallina (two l's); Rocq is the prover, and Iris is the program logic and library used for the heap reasoning.

Once the CJR definitions are available, `theories/examples/emit_cj.v` uses a custom Gallina printer to turn the selected CJR syntax trees into Cangjie source files under `generated/cj/`. This is not Rocq's standard extraction mechanism, and it does not extract proof terms: it serializes the program definitions. A disciplined build runs `make verify` to check proofs and assumptions, then `make cj` to emit the programs. Passing the proof check establishes the stated CJR properties; by itself it does not prove that the printer's output preserves the CJR semantics.

Finally, Cangjie unit tests compile and execute representative cases of the generated code. They are valuable sanity checks for the printer, compiler, and runtime together, but they cover finitely many inputs and do not replace the proofs. The current project contains this formal semantics, Iris proofs, custom printer, and runtime tests. It does not yet implement an automatic LLM front end

that accepts arbitrary informal specifications. The distinction between the proved CJR program and the tested emitted Cangjie is important; Appendix A records concrete risks at that boundary.

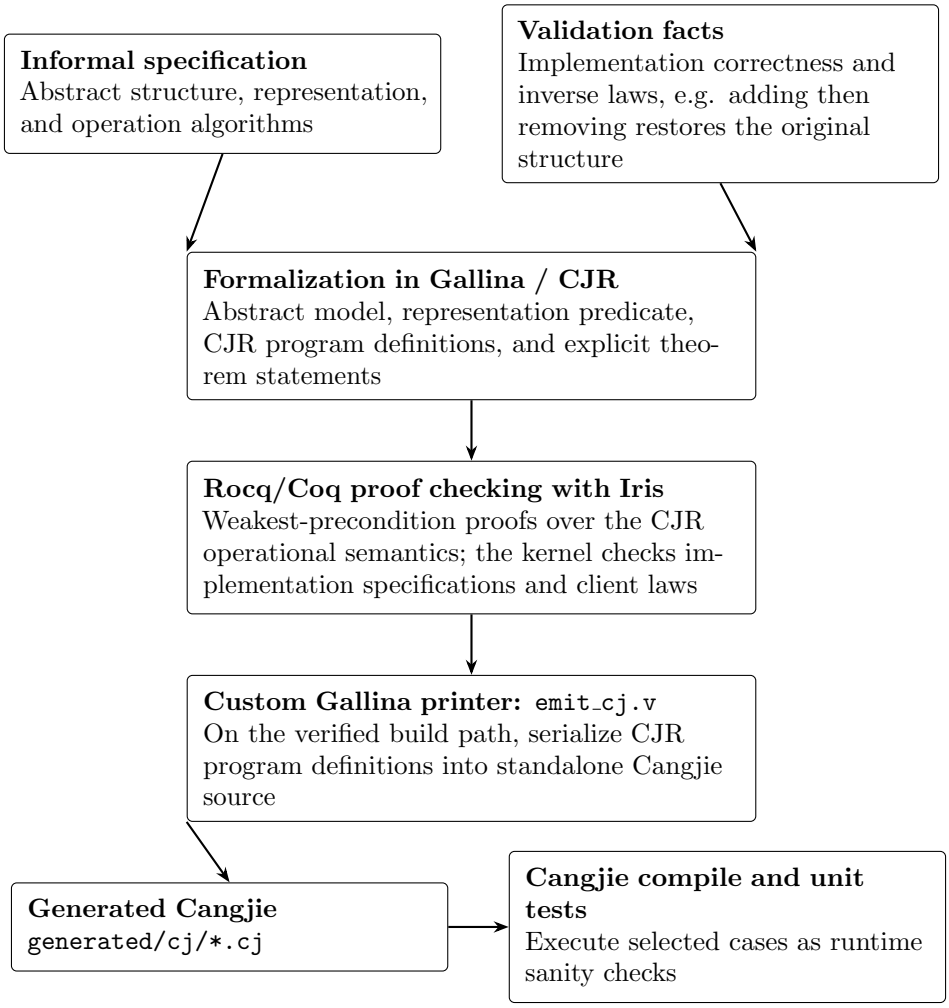


Figure 1: The intended specification-to-test workflow and the components implemented in this repository. The LLM-assisted formalization is a human review point; proof checking validates CJR theorems, while unit tests sample the separately printed Cangjie program.

3 Syntax

3.1 Concrete syntax

The programs in this note are written in Cangjie, as specified by the language documentation [1]. CJR is precisely the fragment of that language that the mechanization covers. The core in `theories/lang.v` is the lowering target, not a second surface syntax. A multi-argument `func` lowers to a single argument—a struct of the arguments—because core application is unary. A class that is not marked `open` cannot be inherited; that is the final class of the core. `CPointer` reads, writes, and offsets are the `unsafe` methods `read`, `write`, and `+` from the standard library, and allocation is the `foreign` function `malloc` from the Cangjie–C interoperability chapter.

```
// types used below: Int64, Float64, Bool, Unit, struct, class, CPointer<T>
```

```
func add(a: Int64, b: Int64): Int64 {
    a + b
}

let frozen: Int64 = 1
var n: Int64 = 1
while (1 <= n) {
    n = n - 1
}

if (n <= 0) {
    0
} else {
    n
}

struct Cell {
    public var x: Int64
    public init(x: Int64) { this.x = x }
}

class Counter {          // not open, so it has no subclasses
    public var value: Int64
    public init(value: Int64) { this.value = value }
}

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit
```

`let` is immutable, and in the core it is substitution. `var` is mutable, and it is a stack slot. The last expression of a block is its value, and `()` is unit.

3.2 From Cangjie to the Coq core

We summarise the correspondence in the table below. Each row pairs one Cangjie form with the constructor in `theories/lang.v` to which it lowers. Field names become positions: the first field of a `struct` or `class` is index 0. A call `f(a, b)` is `App` of `f` to a `StructV` of the arguments, because `App` takes one argument. Forms that mention a string name of a `var` are rewritten by `subst_var` into the administrative stack operations listed in the last rows. Those operations have no spelling in Cangjie; they appear only after a `var` has been given a stack slot.

Cangjie	Coq constructor
`Int64`, `Float64`, `Bool`, `Unit`	`TyInt`, `TyFloat`, `TyBool`, `TyUnit`
`struct S { ... }`	`TyStruct` of the field types
`CPointer<T>`	`TyPtr`
`class C { ... }` with no `open`	`TyObj` of the field count
`(T) -> U`	`TyFun`
`1`, `1.0`, `true`, `()`	`LitInt`, `LitFloat`, `LitBool`, `LitUnit`
a `CPointer` value, including null from `CPointer<T>()`	`LitPtr`, `LitNull`
a class instance	`LitObj`
`func f(x) { e }`	`Rec` (some name, some parameter, body); a finished closure is `RecV`
a struct value such as `Cell(0)`	`StructV`
a name	`Var`
`let x = e1` followed by `e2`	`Let`
`var x = e1` followed by `e2`	`VarBind`
`x = e` for a `var x`	`Assign`
`if (e0) { e1 } else { e2 }`	`If`
`while (e1) { e2 }`	`While`
two expressions in sequence, or a block	`Seq`
`+`, `-`, `<=`, `==` on `Int64`, `&&`	`PlusOp`, `MinusOp`, `LeOp`, `EqOp`, `AndOp`
`f(a)`	`App`
`Cell(e)`	`Struct`, evaluated left to right
`p.x` on a struct	`StructLoad` at the index of `x`
`p.x = e` on a `var` struct	`StructStore` of that variable and index
`Counter(e)`	`New`
`o.value`	`FieldLoad`
`o.value = e`	`FieldStore`
`malloc` of one word, then `CPointer<Int64>(...)`	`Alloc` of `1`, result `LitPtr`
`free(p)`	`Free`
`unsafe { p.read() }`	`Load`
`unsafe { p.write(v) }`	`Store`
`unsafe { p + i }`	`Offset`
no Cangjie spelling; introduced for a `var` slot	`Pop`, `StackLoad`, `StackAssign`, `StackFieldStore`, and the literal `LitStack`

The rest of this section defines those constructors and the machine on which they run.

3.3 Locations, identifiers, and types

A location is an integer. We wrap stack slots, raw words, and objects in distinct constructors so that the four ghost maps of the program logic cannot be confused.

```
Definition loc : Type := Z.
Inductive stack_id := StackId (z : Z).
Inductive raw_id := RawId (z : Z).
Inductive obj_id := ObjId (z : Z).

Inductive ty :=
  | TyInt | TyFloat | TyBool | TyUnit
  | TyStruct (ts : list ty)
  | TyPtr (t : ty)
  | TyObj (nfields : nat)
  | TyFun (t1 t2 : ty).
```

Informally, locations are $\mathbb{Z}$, and the three identifier sorts are the tagged copies $\mathsf{StackId}(\ell)$, $\mathsf{RawId}(\ell)$, and $\mathsf{ObjId}(\ell)$. The type grammar is

$$\tau ::= \mathsf{Int64} \mid \mathsf{Float64} \mid \mathsf{Bool} \mid \mathsf{Unit} \mid \mathsf{struct}\,\overline{\tau} \mid \mathsf{CPointer}\,\tau \mid \mathsf{class}_n \mid \tau \to \tau.$$

Here class_n means a final class with n fields. Nothing in the semantics consults τ. The type is documentation of the fragment; the theorems that follow do not assume it.

3.4 Values and expressions

A binder is an optional name: `None` is an ignored parameter. Literals cover integers, a float payload (for which no float operations are defined), booleans, unit, raw pointers, null, object references, and stack addresses. Values are literals, recursive closures, and unboxed struct tuples. Expressions comprise the core forms, together with administrative forms (`Pop`, `StackLoad`, `StackAssign`, `StackFieldStore`) that are introduced when a mutable variable is allocated.

```
Definition binder := option string.

Inductive base_lit :=
  | LitInt (n : Z) | LitFloat (n : Z) | LitBool (b : bool)
  | LitUnit | LitPtr (l : loc) | LitNull
  | LitObj (o : loc) | LitStack (l : loc).

Inductive val :=
  | LitV (l : base_lit)
  | RecV (f x : binder) (e : expr)
  | StructV (vs : list val)
with expr :=
  | Val (v : val) | Var (x : string)
  | Rec (f x : binder) (e : expr)
  | App (e1 e2 : expr)
  | Let (x : binder) (e1 e2 : expr)
  | VarBind (x : string) (e1 e2 : expr)
  | Assign (x : string) (e : expr)
  | Pop (l : stack_id) (e : expr)
  | StackLoad (e : expr)
  | StackAssign (e1 e2 : expr)
  | StackFieldStore (e1 : expr) (i : nat) (e2 : expr)
  | If (e0 e1 e2 : expr)
  | While (e1 e2 : expr)
  | Seq (e1 e2 : expr)
  | BinOp (op : bin_op) (e1 e2 : expr)
  | Alloc (e : expr) | Free (e : expr)
  | Load (e : expr) | Store (e1 e2 : expr)
  | Offset (e1 e2 : expr)
  | Struct (vs : list val) (es : list expr)
  | StructLoad (e : expr) (i : nat)
  | StructStore (x : string) (i : nat) (e : expr)
  | New (vs : list val) (es : list expr)
  | FieldLoad (e : expr) (i : nat)
  | FieldStore (e1 : expr) (i : nat) (e2 : expr).
```

In mathematics we write values

$$v ::= n \mid f \mid b \mid () \mid \mathsf{ptr}(\ell) \mid \mathsf{null} \mid \mathsf{obj}(\ell) \mid \mathsf{stk}(\ell) \mid \mathsf{rec}\, f\, x.\, e \mid \langle v_0, \ldots, v_{k-1} \rangle$$

and we use the expression forms by the same names as the constructors. BinOp ranges over $+$, $-$, $\leq$, $=$, and boolean conjunction. Integer operations act on LitInt; conjunction acts on booleans. Anything else makes bin_op_eval return None, and the machine gets stuck.

Structs and objects are built by a list zipper. $\mathsf{Struct}\, \overline{v}\, \overline{e}$ has already evaluated the values $\overline{v}$ and still has the expressions $\overline{e}$ to evaluate, left to right. New is the same zipper for a final class. When the expression list is empty, the zipper is done.

`Val` and `to_val` identify which expressions are finished:

```
Definition of_val (v : val) : expr := Val v.
Definition to_val (e : expr) : option val :=
  match e with Val v => Some v | _ => None end.
```

So the values of the language, as Iris sees them, are exactly the expressions $\mathsf{Val}\, v$.

3.5 Two kinds of binding

We begin from a distinction that is already present in the surface language and then explain how the core encodes it. Immutable `let` and function parameters are substitution; mutable `var` is not. A `var` is given a fresh stack slot, and every use or assignment of that name is rewritten into an operation on the slot. A bare struct value is still copied: mutation in place happens only through a `var` slot or through a stored field.

Those surface facts come from the Cangjie 1.0 user manual [1]. The stack-slot encoding below is this note's model of them, not a spelling in the language.

- **Immutable vs mutable bindings.** The chapter *Program Structure* (`https://docs.cangjie-lang.cn/en/docs/1.0.0/user_manual/source_en/basic_programming_concepts/program_structure.html`) introduces the mutability modifiers `let` and `var`: a `let` variable is assigned once (initialization), whereas a `var` variable may be reassigned. The chapter *Function Definition* (`https://docs.cangjie-lang.cn/en/docs/1.0.0/user_manual/source_en/function/define_functions.html`) states that function parameters are immutable and cannot be assigned to in the function body.

- **Struct copying.** The same *Program Structure* chapter distinguishes value types (including `struct`) from reference types (including `class`): assigning a value-type variable copies the instance; later mutation of one copy does not affect the other. *Creating a struct Instance* (`https://docs.cangjie-lang.cn/en/docs/1.0.0/user_manual/source_en/struct/create_instance.html`) repeats that assignment and parameter passing copy a `struct`, and that changing fields through an instance requires both the struct variable and the fields to be `var`. The related page *mut Function* (`https://docs.cangjie-lang.cn/en/docs/1.0.0/user_manual/source_en/struct/mut.html`) adds that `mut` methods need a mutable (`var`) receiver; they cannot be called on a `let` struct.

- **Stored class fields.** The chapter *Classes* (`https://docs.cangjie-lang.cn/en/docs/1.0.0/user_manual/source_en/class_and_interface/class.html`) states that `class` is a reference type: assignment does not copy the object, so field updates through one variable are visible through others that share it. That is the "stored field" case in the sentence above.

What follows—fresh stack slots, `subst_var`, `StackLoad`/`StackAssign`/`StackFieldStore`, `LitStack`, and `Pop`—is the Coq core's administrative rewriting, not Cangjie surface syntax.

```
Definition subst_binder (b : binder) (v : val) (e : expr) : expr :=
  match b with None => e | Some x => subst x v e end.
```

The function $\mathsf{subst}\,x\,v\,e$ replaces free occurrences of x by the value v, and it does not descend under a binder that captures x. On a value, literals are unchanged and the body of a closure is substituted only when x is not the function name or the parameter.

The mutable variant $\mathsf{subst_var}\,x\,\ell\,e$ does a different job. A read $\mathsf{Var}\,x$ becomes $\mathsf{StackLoad}(\mathsf{stk}(\ell))$. An assignment $\mathsf{Assign}\,x\,e$ becomes $\mathsf{StackAssign}(\mathsf{stk}(\ell), e')$. A field write $\mathsf{StructStore}\,x\,i\,e$ becomes $\mathsf{StackFieldStore}(\mathsf{stk}(\ell), i, e')$. Other occurrences of x are rewritten recursively, again respecting capture. The slot address is itself a value, so later stack operations do not look up a string environment.

For a concrete example, the fragment $\mathsf{var}\ x = 1\ \mathsf{in}\ (x = x + 1)$ allocates a fresh slot ℓ, stores 1, and rewrites the body, reaching

$$\mathsf{Pop}\,(\mathsf{StackId}(\ell))\,(\mathsf{StackAssign}(\mathsf{stk}(\ell),\,\mathsf{BinOp}\,+(\mathsf{StackLoad}(\mathsf{stk}(\ell)),\,1))).$$

Informally that is $\mathsf{pop}_\ell\,(\mathsf{stk}(\ell) := \mathsf{stk}(\ell) + 1)$: the read of x is a load from the slot, the assignment writes the slot, and `Pop` deletes it when the body finishes.

What is striking about this split is that it is the whole model of "locals do not need a points-to fact until they are mutable." An immutable binding disappears into the syntax. A mutable binding is a cell, and the cell has an owner.

4 Machine state

4.1 Three regions, one counter

The machine is a single record. Locals and unboxed structs that are stored in `var` slots live in the stack map. `CPointer` words live in the raw map, and each allocated block records its length in a second map on the same addresses. Final class fields live in a map from $(\mathsf{ObjId}(\ell), i)$ to a value. A single counter `next` hands out fresh addresses for all three regions.

```
Record state := MkState {
  cjr_stack  : gmap stack_id val;
  cjr_raw    : gmap raw_id val;
  cjr_blocks : gmap raw_id nat;
  cjr_obj    : gmap (obj_id * nat) val;
  cjr_next   : Z
}.
Definition state_init : state := inhabitant. (* all maps empty, next = 0 *)
```

Write a state as

$$\sigma = (\sigma_s,\ \sigma_r,\ \sigma_b,\ \sigma_o,\ \mathsf{next}(\sigma)).$$

The initial state σ_0 is $(\emptyset, \emptyset, \emptyset, \emptyset, 0)$.

Indexing a pointer is arithmetic on the address, one word per element:

$$\mathsf{ptr}(\ell) + i \;=\; \mathsf{ptr}(\ell + i).$$

There is no byte offset and no padding. A block of length n at base ℓ occupies the words $\ell, \ell+1, \ldots, \ell+n-1$. Blocks are freed explicitly. Class objects are not freed in this development: there is no `delete` for them, and there is no reachability argument.

Null is a pointer value, not an address in σ_r. Loading or storing through null has no rule, so it is stuck. An out-of-bounds or use-after-free access is stuck for the same reason: the lookup premise of load, store, or free fails.

4.2 Well-formed states

Not every 5-tuple is a state that the allocator is allowed to produce. Well-formedness says that every address already in use lies strictly below the counter, that every block's words are present, and that live blocks do not overlap.

```
Definition stack_below m n :=
  forall l, StackId l in dom m -> (l < n)%Z.
(* raw_below, block_below, obj_below are the same idea. *)

Definition block_cover blocks raw next :=
  forall l n, blocks !! RawId l = Some n ->
    (l + Z.of_nat n <= next)%Z /\
    forall i, i < n -> is_Some (raw !! RawId (l + Z.of_nat i)).

Definition blocks_disjoint blocks :=
  forall l1 n1 l2 n2,
    blocks !! RawId l1 = Some n1 ->
    blocks !! RawId l2 = Some n2 ->
    l1 <> l2 ->
    (l1 + Z.of_nat n1 <= l2 \/ l2 + Z.of_nat n2 <= l1)%Z.

Record state_wf (s : state) : Prop := {
  wf_stack : stack_below (cjr_stack s) (cjr_next s);
  wf_raw   : raw_below (cjr_raw s) (cjr_next s);
  wf_block : block_below (cjr_blocks s) (cjr_next s);
  wf_obj   : obj_below (cjr_obj s) (cjr_next s);
  wf_cover : block_cover (cjr_blocks s) (cjr_raw s) (cjr_next s);
  wf_disj  : blocks_disjoint (cjr_blocks s)
}.
```

In mathematics, $\mathsf{wf}(\sigma)$ is the conjunction of:

$\forall \ell.\ \mathsf{StackId}(\ell) \in \mathrm{dom}(\sigma_s) \Rightarrow \ell < \mathsf{next}(\sigma)$,
and the same for raw addresses, block bases, and object addresses,
$\forall \ell, n.\ \sigma_b(\mathsf{RawId}(\ell)) = \mathsf{Some}(n) \Rightarrow \ell + n \leq \mathsf{next}(\sigma)\ \wedge\ \forall i < n.\ \sigma_r(\mathsf{RawId}(\ell + i)) \neq \mathsf{None}$,
distinct live blocks have disjoint integer intervals.

The freshness lemmas are immediate from the first four clauses. If $\mathsf{wf}(\sigma)$ and $\ell = \mathsf{next}(\sigma)$, then ℓ is absent from the stack, from the raw heap, from the block map, and, at every field index, from the object heap. Allocation is therefore the insertion of a key that is not already present.

The initial state is well formed: `state_init_wf`. Each successful head step that the program logic uses is proved to preserve wf: stack allocation, deletion, and update; raw store; one-word allocation and freeing; class allocation; and field update.

One-word allocation is worth spelling out, because it is the case the logic rule talks about. From base $\ell = \mathsf{next}(\sigma)$,

$$\begin{aligned} \sigma_r' &= \{\mathsf{RawId}(\ell) \mapsto 0\} \cup \sigma_r, \\ \sigma_b' &= \sigma_b[\mathsf{RawId}(\ell) \mapsto 1], \\ \mathsf{next}' &= \ell + 1. \end{aligned}$$

The new word is in range, the new block is covered, and disjointness holds because the new interval $[\ell, \ell+1)$ starts at the old counter, past every previous address. Freeing that block deletes the length and the word. Other blocks keep their cells precisely because their intervals were disjoint from $[\ell, \ell+1)$.

Class allocation at $o = \mathsf{next}(\sigma)$ inserts the fields $(\mathsf{ObjId}(o), i) \mapsto v_i$ for a list $v_0, \ldots, v_{k-1}$ and advances the counter by one. Those keys are fresh because every old object address is below o.

5 Small-step semantics

5.1 Head reduction

A head step is a relation

$$(e, \sigma) \longrightarrow (e', \sigma')$$

with no observations and no forked threads. Both are hard-wired: `observation` is the empty type, and every rule produces an empty list of threads. The language is deterministic on the nose.

Pure computation. These rules neither read nor write the heaps.

$$\begin{aligned}
\mathsf{rec}\, f\, x.\, e &\longrightarrow \mathsf{rec}\, f\, x.\, e && \text{(as a value)} \\
(\mathsf{rec}\, f\, x.\, e)\, v &\longrightarrow e[x \mapsto v][f \mapsto \mathsf{rec}\, f\, x.\, e] \\
\mathsf{let}\, x = v \mathsf{in} e &\longrightarrow e[x \mapsto v] \\
\mathsf{if\ true\ then}\ e_1\ \mathsf{else}\ e_2 &\longrightarrow e_1 \\
\mathsf{if\ false\ then}\ e_1\ \mathsf{else}\ e_2 &\longrightarrow e_2 \\
\mathsf{while}\ e_1\ \mathsf{do}\ e_2 &\longrightarrow \mathsf{if}\ e_1\ \mathsf{then}\ (e_2; \mathsf{while}\ e_1\ \mathsf{do}\ e_2)\ \mathsf{else}\ () \\
v;\, e &\longrightarrow e \\
v_1 \oplus v_2 &\longrightarrow v && \text{when } [\![\oplus]\!](v_1, v_2) = \mathsf{Some}(v) \\
\mathsf{ptr}(\ell) + i &\longrightarrow \mathsf{ptr}(\ell + i) \\
\langle \overline{v},\, v :: \overline{e} \rangle &\longrightarrow \langle \overline{v} {+\!+} [v],\, \overline{e} \rangle \\
\langle \overline{v} \rangle &\longrightarrow \langle v_0, \ldots \rangle && \text{(the struct value)} \\
\langle \overline{v} \rangle.i &\longrightarrow v && \text{when } v \text{ is the } i\text{th component.}
\end{aligned}$$

The `while` rule duplicates the loop. It is a head step, not an evaluation context, so the copies are syntax rather than a shared mutable closure. The condition and the body are evaluated later, by ordinary contexts.

Stack. Let $\ell = \mathsf{next}(\sigma)$.

$\mathsf{var}\ x = v\ \mathsf{in}\ e \longrightarrow \mathsf{pop}_\ell\,(e\{x \mapsto \ell\})$	and $\sigma_s(\mathsf{StackId}(\ell)) := v,\ \mathsf{next} := \ell + 1$
$\mathsf{pop}_\ell\, v \longrightarrow v$	and delete the slot
$\mathsf{stk}(\ell) \longrightarrow v$	if the slot holds v
$\mathsf{stk}(\ell) := v \longrightarrow ()$	if the slot exists; it then holds v
$\mathsf{stk}(\ell).i := v \longrightarrow ()$	if the slot holds a struct and index i is in range.

The last rule replaces component i with set_nth and writes the struct back into the same slot. No raw or object address is allocated. The most important consequence is that a stack update can be framed against a raw points-to fact: the two maps are different components of σ.

Raw heap. For $n > 0$, allocation of n words at $\ell = \mathsf{next}(\sigma)$ returns $\mathsf{ptr}(\ell)$, records length $Z.\mathsf{to_nat}(n)$, zeroes the words, and sets the counter to $\ell + n$. The proved program-logic rule specialises this to $n = 1$.

$\mathsf{free}(\mathsf{ptr}(\ell)) \longrightarrow ()$	if a block length is recorded at ℓ; that block's words are deleted
$!\,\mathsf{ptr}(\ell) \longrightarrow v$	if the word holds v
$\mathsf{ptr}(\ell) := v \longrightarrow ()$	if the word exists; it then holds v.

Free deletes whatever length is stored. The one-word reasoning rule asks separately for the length fact 1, so the client of that rule cannot free a longer block by mistake.

Objects. The object zipper steps like the struct zipper. When it finishes, at $o = \mathsf{next}(\sigma)$,

$$\mathsf{new}(v_0, \ldots, v_{k-1}) \longrightarrow \mathsf{obj}(o)$$

and the fields $(o, i) \mapsto v_i$ are added. Field load and field store are lookup and update at $(\mathsf{ObjId}(o), i)$. Store requires the field to exist already. A missing field, like a missing word, is stuck.

5.2 Evaluation contexts

Head steps fire only at the root. The context items say where a subexpression may be reduced. Each item has one hole, and the hole is the expression currently being evaluated. Arguments are evaluated left to right. `if` evaluates only the condition. `while` does not evaluate under itself: the unfolding rule is the head step above.

Representative items, written as contexts:

$$E[\mathsf{let}\, x = [\,]\ \mathsf{in}\ e],\ E[\mathsf{var}\, x = [\,]\ \mathsf{in}\ e],\ E[\mathsf{pop}_\ell\,[\,]],$$
$$E[[\,]\, e],\ E[v\,[\,]],\ E[\mathsf{if}\,[\,]\ \mathsf{then}\ e_1\ \mathsf{else}\ e_2],\ E[[\,]; e],$$
$$E[[\,] \oplus e],\ E[v \oplus [\,]],\ E[!\,[\,]],\ E[\mathsf{free}\,[\,]],\ E[[\,] := e],\ E[v := [\,]].$$

Struct and `new` items evaluate the next expression in the zipper. The Coq constructor list is `ectx_item`, and `fill_item` implements the grammar.

Iris then builds the language in the standard way.

```
Lemma cjr_ectxi_mixin : EctxiLanguageMixin of_val to_val fill_item base_step.
Canonical Structure cjr_ectxi_lang := EctxiLanguage cjr_ectxi_mixin.
Canonical Structure cjr_ectx_lang := EctxLanguageOfEctxi cjr_ectxi_lang.
Canonical Structure cjr_lang := LanguageOfEctx cjr_ectx_lang.
```

A configuration steps by finding a context K and a head redex:

$$K[e] \longrightarrow K[e'] \quad \text{when} \quad (e, \sigma) \longrightarrow (e', \sigma').$$

Filling a non-value never produces a value, a value never head-reduces, and a head reduction of a filled term implies that the hole was already a value. Those are the mixin obligations; they are what let Iris treat K as a program context in the bind rule.

5.3 Determinism

```
Lemma base_step_det e s k e' s' t k2 e2 s2 t2 :
  base_step e s k e' s' t ->
  base_step e s k2 e2 s2 t2 ->
  k = k2 /\ e' = e2 /\ s' = s2 /\ t = t2.
```

Theorem 1 (Determinism of head reduction)**.** *If* $(e, \sigma) \longrightarrow (e_1, \sigma_1)$ *and* $(e, \sigma) \longrightarrow (e_2, \sigma_2)$*, then* $e_1 = e_2$ *and* $\sigma_1 = \sigma_2$*. Observations and forked threads, which are always empty, agree as well.*

The proof is inversion of both derivations. Where a rule reads a map or calls $[\![\oplus]\!]$, the two lookups are the same, so the two results are the same. There is no scheduler and no choice of fresh address other than $\mathsf{next}(\sigma)$.

6 Separation logic

6.1 What is owned

The program logic does not talk about the whole machine. It talks about the cells a piece of code is allowed to use. Ownership is a ghost map—one authoritative map per region—with a points-to fact for each cell. Full ownership is the fraction 1.

```
Definition stack_mapsto (l : stack_id) (v : val) : iProp S :=
  ghost_map_elem stack_name l (DfracOwn 1) v.
Definition raw_mapsto (l : raw_id) (v : val) : iProp S :=
  ghost_map_elem raw_name l (DfracOwn 1) v.
Definition block_mapsto (l : raw_id) (n : nat) : iProp S :=
  ghost_map_elem block_name l (DfracOwn 1) n.
Definition obj_mapsto (o : obj_id) (i : nat) (v : val) : iProp S :=
  ghost_map_elem obj_name (o, i) (DfracOwn 1) v.
```

The notations in the sources are $\ell \mapsto_s v$, $\ell \mapsto_r v$, $\ell \mapsto_b n$, and $o \mapsto_o [i]v$. We use four maps, rather than a single heap, because a stack slot, a raw word, a block length, and a class field are different kinds of cell. A stack proof cannot accidentally be applied to a pointer.

The state interpretation ties the ghost maps to the concrete machine and remembers well-formedness:

```
Definition cjr_state_interp (s : state) : iProp S :=
  ghost_map_auth stack_name 1 (cjr_stack s) *
  ghost_map_auth raw_name 1 (cjr_raw s) *
  ghost_map_auth block_name 1 (cjr_blocks s) *
  ghost_map_auth obj_name 1 (cjr_obj s) *
  pure (state_wf s).
```

At every step, Iris owns $\mathsf{StateInterp}(\sigma)$. A points-to fact is a fragment of one of those authoritative maps. Updating a cell is updating both the fragment and the matching entry of σ, and the wf conjunct is re-established by the lemma for that step.

6.2 Weakest precondition, in this instantiation

The Iris instance is the standard one for a deterministic language with no thread pool of interest:

$$\begin{aligned} \mathsf{stateInterp}(\sigma) &= \mathsf{cjr_state_interp}(\sigma), \\ \mathsf{forkPost}(_) &= \top, \\ \mathsf{numLatersPerStep}(_) &= 0. \end{aligned}$$

The weakest precondition $\mathsf{wp}\, e\, \{\Phi\}$ is Iris's. For this tutorial the reading is:

Definition 2 (Specification)**.** $\mathsf{wp}\, e\, \{\Phi\}$ *means: for every well-formed extension of the current ghost ownership, executing e neither gets stuck nor forks, and if it terminates with a value v then $\Phi(v)$ holds. Partial correctness is enough: a diverging loop satisfies a weakest precondition whose postcondition is never reached.*

The later $\triangleright$ and the credit $£1$ appearing in the rules are Iris's accounting for steps, in the mode `HasLc`. When a rule says

$$\ell \mapsto_r v \mathrel{-\!\!*} \triangleright(£1 \mathrel{-\!\!*} \ell \mapsto_r v \mathrel{-\!\!*} \Phi(v)) \vdash \mathsf{wp}\, (!\,\ell)\, \{\Phi\},$$

the intended content is appealingly simple: ownership of the cell is enough to load it, and one gets the same cell back, together with the value that was there. The later and the credit are discharged by the step. In the example proofs they are thrown away with an introduction `iIntros "> _"`. They do not change the memory reasoning.

6.3 Structural rules

These three rules are why the logic is a separation logic rather than a Hoare logic with an explicit frame side-condition. Bind and the pure-step lemma are proved in `primitive_laws.v` by appealing to Iris. The frame rule is Iris's own `wp_frame_l`; this development does not re-prove it.

```
Lemma wp_bind K e Phi :
  wp e (fun v => wp (fill K (Val v)) Phi) |- wp (fill K e) Phi.
```

Lemma 3 (Bind)**.** *If K is an evaluation context, then*

$$\mathsf{wp}\, e\, \{\, v.\, \mathsf{wp}\, K[v]\, \{\Phi\}\,\} \;\vdash\; \mathsf{wp}\, K[e]\, \{\Phi\}.$$

Intuition. To prove a specification of a term with a hole, we prove a specification of the hole, and we allow the postcondition of the hole to talk about the rest of the program. Left-to-right evaluation is just a choice of K. No memory fact has to be mentioned. Facts that the hole does not use stay outside this rule and are framed.

Lemma 4 (Frame)**.**

$$P * \mathsf{wp}\, e\, \{\Phi\} \;\vdash\; \mathsf{wp}\, e\, \{\, v.\, P * \Phi(v)\,\}.$$

Intuition. If a proof of e never mentions P, then e cannot depend on the cells inside P, and those cells survive e unchanged. This is the rule that makes "the other pointer is untouched" a consequence of not putting that pointer into the callee's precondition. It is not proved by induction on CJR expressions.

Lemma 5 (Consequence)**.**

$$\mathsf{wp}\, e\, \{\Phi\} \;\wedge\; (\forall v.\, \Phi(v) \mathrel{-\!\!*} \Psi(v)) \;\vdash\; \mathsf{wp}\, e\, \{\Psi\}.$$

Intuition. A stronger postcondition can be weakened. The examples use this when a lemma such as `bump_wp` returns both a pure result and a framed cell, and the continuation only needs one of them. In Coq it is `wp_wand` or `wp_mono`.

6.4 Pure rules

A pure head step is one that is defined in every state, does not change the state, and has exactly one successor. The generic lemma is `wp_pure_step`. Each concrete rule below is an instance. Where the step produces a value immediately, the postcondition is under a later, because that is how the generic lemma returns the credit. Where the step produces another expression, the rule simply asks for a weakest precondition of that expression.

Closures and application.

```
Lemma wp_rec f x e Phi :
  later (pound 1 -*- Phi (RecV f x e)) |- wp (Rec f x e) Phi.
Lemma wp_app f x e v Phi :
  wp (subst_binder x v (subst_binder f (RecV f x e) e)) Phi
    |- wp (App (Val (RecV f x e)) (Val v)) Phi.
```

$$\triangleright(\Phi(\mathsf{rec}\, f\, x.\, e)) \vdash \mathsf{wp}\, (\mathsf{rec}\, f\, x.\, e)\, \{\Phi\},$$
$$\mathsf{wp}\, e[x \mapsto v][f \mapsto \mathsf{rec}\, f\, x.\, e]\, \{\Phi\} \vdash \mathsf{wp}\, ((\mathsf{rec}\, f\, x.\, e)\, v)\, \{\Phi\}.$$

Intuition. Evaluating a function literal yields the closure as a value. Calling it substitutes the argument and, if the function is recursive, the closure itself. No heap fact is required: the code is data, and the data is the value in hand. That is what lets a vtable be an ordinary struct of closures.

Let, if, while, sequencing, arithmetic, offset.

$$\mathsf{wp}\, e[x \mapsto v]\, \{\Phi\} \vdash \mathsf{wp}\, (\mathsf{let}\, x = v\, \mathsf{in}\, e)\, \{\Phi\}$$
$$\mathsf{wp}\, e_1\, \{\Phi\} \vdash \mathsf{wp}\, (\mathsf{if}\, \mathsf{true}\, \mathsf{then}\, e_1\, \mathsf{else}\, e_2)\, \{\Phi\}$$
$$\mathsf{wp}\, e_2\, \{\Phi\} \vdash \mathsf{wp}\, (\mathsf{if}\, \mathsf{false}\, \mathsf{then}\, e_1\, \mathsf{else}\, e_2)\, \{\Phi\}$$
$$\mathsf{wp}\, (\mathsf{if}\, e_1\, \mathsf{then}\, (e_2;\, \mathsf{while}\, e_1\, \mathsf{do}\, e_2)\, \mathsf{else}\, ())\, \{\Phi\} \vdash \mathsf{wp}\, (\mathsf{while}\, e_1\, \mathsf{do}\, e_2)\, \{\Phi\}$$
$$\mathsf{wp}\, e\, \{\Phi\} \vdash \mathsf{wp}\, (v; e)\, \{\Phi\}$$
$$[\![\oplus]\!](v_1, v_2) = \mathsf{Some}(v) \Rightarrow \triangleright(\Phi(v)) \vdash \mathsf{wp}\, (v_1 \oplus v_2)\, \{\Phi\}$$
$$\triangleright(\Phi(\mathsf{ptr}(\ell + i))) \vdash \mathsf{wp}\, (\mathsf{ptr}(\ell) + i)\, \{\Phi\}.$$

Intuition for `while`. The loop rule does not mention an invariant; it unfolds the loop once into a conditional. The invariant is whatever assertion one has in hand when returning to the same syntactic loop. If the condition is false, the unfolding takes the else branch and returns unit. If it is

true, the body runs and the same rule applies again. Termination is not proved; the unfolding is used only as many times as the concrete execution needs.

Intuition for offset. Adding an index to a pointer is arithmetic. It does not check that the resulting address lies in a block. The check happens when someone loads or stores: without a points-to fact at the computed address, the load rule does not apply, and the machine would be stuck. Bounds are therefore a matter of which points-to facts the proof owns, not of a side condition on $+$.

Struct values.

$$\triangleright(\Phi(\langle \overline{v} \rangle)) \vdash \mathsf{wp}\, \langle \overline{v} \rangle\, \{\Phi\},$$
$$\overline{v}(i) = v \Rrightarrow \triangleright(\Phi(v)) \vdash \mathsf{wp}\, (\langle \overline{v} \rangle.i)\, \{\Phi\}.$$

Intuition. A finished zipper is the unboxed tuple. Reading a component copies it out. The tuple is not a heap cell, so there is nothing to frame and nothing to free. If the index is out of range, there is no rule.

Building a struct or an object from a non-empty expression list is also a pure step (`StructStepS`, `NewStepS`). The development uses `wp_pure_step` for that step in the vtable example rather than a named lemma.

6.5 Stack rules

```
Lemma wp_var_bind x v e Phi :
  (forall l, StackId l |->s v -*-
     wp (subst_var x l e)
        (fun w => exists v', StackId l |->s v' * Phi w))
  |- wp (VarBind x (Val v) e) Phi.

Lemma wp_stack_load l v Phi :
  l |->s v -*- later (pound 1 -*- l |->s v -*- Phi v)
    |- wp (StackLoad (stk l)) Phi.

Lemma wp_stack_assign l v w Phi :
  l |->s v -*- later (pound 1 -*- l |->s w -*- Phi unit)
    |- wp (StackAssign (stk l) (Val w)) Phi.

Lemma wp_stack_field l vs i w v Phi :
  vs !! i = Some w ->
  l |->s StructV vs -*-
    later (pound 1 -*- l |->s StructV (set_nth i v vs) -*- Phi unit)
    |- wp (StackFieldStore (stk l) i (Val v)) Phi.
```

In mathematics, with ℓ a stack address:

$$\big(\forall \ell.\ \ell \mapsto_s v \mathrel{-\!\!*} \mathsf{wp}\, e\{x \mapsto \ell\}\, \{\, w.\ \exists v'.\ \ell \mapsto_s v' * \Phi(w)\,\}\big) \vdash \mathsf{wp}\, (\mathsf{var}\ x = v\ \mathsf{in}\ e)\, \{\Phi\},$$
$$\ell \mapsto_s v \mathrel{-\!\!*} \triangleright(\ell \mapsto_s v \mathrel{-\!\!*} \Phi(v)) \vdash \mathsf{wp}\, (\mathsf{stk}(\ell))\, \{\Phi\},$$
$$\ell \mapsto_s v \mathrel{-\!\!*} \triangleright(\ell \mapsto_s w \mathrel{-\!\!*} \Phi(())) \vdash \mathsf{wp}\, (\mathsf{stk}(\ell) := w)\, \{\Phi\},$$
$$\overline{v}(i) = w \Rrightarrow \ell \mapsto_s \langle \overline{v} \rangle \mathrel{-\!\!*} \triangleright\big(\ell \mapsto_s \langle \overline{v}[i \mapsto u] \rangle \mathrel{-\!\!*} \Phi(())\big) \vdash \mathsf{wp}\, (\mathsf{stk}(\ell).i := u)\, \{\Phi\}.$$

Intuition for ***var****.* The rule allocates a fresh slot, hands exclusive ownership of it to the proof, and asks for a proof of the body after the name has been compiled into stack operations. The

contents may be changed. At the end the slot must be given back, at whatever value it then has. The pop administrative step deletes it, so the caller does not see a stack fact. This is lexical scope: the owner of the cell is the body, and the cell does not escape.

The existential is essential. If the postcondition demanded the original value, an assignment inside the body could not be verified, because assignment replaces the points-to fact.

Intuition for load, assign, and field update. Load reads and returns the same fact. Assignment exchanges the fact for one about the new value. Field update is assignment of a struct, with one component changed, and it is only available when that component already exists. A missing component is stuck, and there is no points-to fact that would let the rule fire. None of these rules allocates on the raw heap or the object heap.

6.6 Raw-heap rules

```
Lemma wp_alloc_one Phi :
  (forall l, RawId l |->r LitV (LitInt 0) -*-
             RawId l |->b 1 -*- Phi (LitV (LitPtr l)))
    |- wp (Alloc (Val (LitV (LitInt 1)))) Phi.

Lemma wp_free_one l v Phi :
  RawId l |->r v -*- RawId l |->b 1 -*- later (Phi unit)
    |- wp (Free (Val (LitV (LitPtr l)))) Phi.

Lemma wp_load l v Phi :
  RawId l |->r v -*- later (RawId l |->r v -*- Phi v)
    |- wp (Load (Val (LitV (LitPtr l)))) Phi.

Lemma wp_store l v w Phi :
  RawId l |->r v -*- later (RawId l |->r w -*- Phi unit)
    |- wp (Store (Val (LitV (LitPtr l))) (Val w)) Phi.
```

$$\big(\forall \ell.\ \ell \mapsto_r 0 * \ell \mapsto_b 1 \mathrel{-\!\!*} \Phi(\mathsf{ptr}(\ell))\big) \vdash \mathsf{wp}\,(\mathsf{alloc}(1))\,\{\Phi\},$$
$$\ell \mapsto_r v * \ell \mapsto_b 1 \mathrel{-\!\!*} \triangleright(\Phi(())) \vdash \mathsf{wp}\,(\mathsf{free}(\ell))\,\{\Phi\},$$
$$\ell \mapsto_r v \mathrel{-\!\!*} \triangleright(\ell \mapsto_r v \mathrel{-\!\!*} \Phi(v)) \vdash \mathsf{wp}\,(!\,\ell)\,\{\Phi\},$$
$$\ell \mapsto_r v \mathrel{-\!\!*} \triangleright(\ell \mapsto_r w \mathrel{-\!\!*} \Phi(())) \vdash \mathsf{wp}\,(\ell := w)\,\{\Phi\}.$$

Intuition. Allocation creates a cell that was not owned before, initialised to zero, together with a separate fact that a block of length one starts there. The two facts have different jobs. The points-to fact lets one load and store the word. The block fact lets one free the block. Free consumes both, so the word cannot be used again: there is no fact left to feed to the load rule, and the concrete word has been deleted, so the machine would get stuck if one tried.

Load does not disturb the cell. Store keeps ownership and changes the value. Neither rule needs the block fact. A client may therefore load and store while the right to free sits unused in the context, and the frame rule keeps it there.

The operational rule for allocation accepts any positive length. The verified rule above is the one-word instance, which the small examples use. The array library uses the general rule: a points-to fact per word, plus one block fact for the base and the length.

6.7 Object rules

```
Lemma wp_new vs Phi :
  (forall o, ([* list] i |-> v in vs, ObjId o |->o[i] v) -*-
             Phi (LitV (LitObj o)))
    |- wp (New vs []) Phi.

Lemma wp_field_load o i v Phi :
  ObjId o |->o[i] v -*- later (ObjId o |->o[i] v -*- Phi v)
    |- wp (FieldLoad (Val (LitV (LitObj o))) i) Phi.

Lemma wp_field_store o i v w Phi :
  ObjId o |->o[i] v -*- later (ObjId o |->o[i] w -*- Phi unit)
    |- wp (FieldStore (Val (LitV (LitObj o))) i (Val w)) Phi.
```

$$\big(\forall o.\ \mathop{*}_{i<k} o \mapsto_o [i]v_i \mathrel{-\!\!*} \Phi(\mathsf{obj}(o))\big) \vdash \mathsf{wp}\,(\mathsf{new}(v_0,\ldots,v_{k-1}))\,\{\Phi\},$$
$$o \mapsto_o [i]v \mathrel{-\!\!*} \triangleright(o \mapsto_o [i]v \mathrel{-\!\!*} \Phi(v)) \vdash \mathsf{wp}\,(o.i)\,\{\Phi\},$$
$$o \mapsto_o [i]v \mathrel{-\!\!*} \triangleright(o \mapsto_o [i]w \mathrel{-\!\!*} \Phi(())) \vdash \mathsf{wp}\,(o.i := w)\,\{\Phi\}.$$

Intuition. `new` of a finished argument list allocates a fresh object and returns one points-to fact per field, with the initialising values. There is no block fact and no free rule: the object stays allocated. Field load and field store are the same idea as raw load and store, on a different map. Owning $o \mapsto_o [i]v$ says nothing about $o \mapsto_o [j]w$ for $j \neq i$, and nothing about any raw word. That is why a raw cell can be framed across a field update, and why two fields of one object can be reasoned about separately.

6.8 Exclusivity

```
Lemma stack_mapsto_exclusive l v1 v2 :
  l |->s v1 -*- l |->s v2 |- False.
Lemma raw_mapsto_exclusive l v1 v2 :
  l |->r v1 -*- l |->r v2 |- False.
```

Lemma 6 (Exclusive ownership)**.** *For a stack address or a raw address ℓ, and any values v_1, v_2,*

$$\ell \mapsto_s v_1 * \ell \mapsto_s v_2 \vdash \bot, \qquad \ell \mapsto_r v_1 * \ell \mapsto_r v_2 \vdash \bot.$$

Intuition. Full ownership is unique. The ghost-map algebra rejects two fragments of fraction 1 at the same key, even if they agree on the value. This is the specification of non-aliasing. If a swap is specified with two points-to assumptions, applying it to one address twice is impossible: the precondition is false. The machine might still have a single cell there; the logic refuses to pretend there are two.

The same algebra applies to block facts and to object fields. The development records the stack and raw cases because those are the ones the examples need to point at.

7 Main theorems

7.1 Head reduction is deterministic

Theorem 1 above is `base_step_det`. Restated: the head reduction relation is a partial function of (e, σ). Combined with the context machinery, a configuration has at most one successor. Stuckness is therefore unambiguous. Either the term is a value, or exactly one head redex is ready and the rule applies, or no rule applies and the program is stuck.

7.2 Well-formedness is an invariant of the verified steps

The state interpretation contains $\ulcorner \mathsf{wf}(\sigma) \urcorner$. Each memory rule proves that the state after the step is well formed if the state before was. The statements are `wf_stack_alloc`, `wf_stack_delete`, `wf_stack_upd`, `wf_raw_store`, `wf_alloc_one`, `wf_free_one`, `wf_new`, and `wf_field_store`.

Lemma 7 (Allocator invariant)**.** $\mathsf{wf}(\sigma_0)$. *If* $\mathsf{wf}(\sigma)$ *and a verified memory rule steps to* σ'*, then* $\mathsf{wf}(\sigma')$.

The one-word cases are the ones used by the logic. For example, freeing a block of length 1 at ℓ preserves coverage of every other block because disjointness puts their words outside $\{\ell\}$.

7.3 Adequacy

```
Lemma cjr_adequacy S e (phi : val -> Prop) :
  (forall `{!cjrGS S} `{!invGS_gen HasLc S},
     |- wp e (fun v => pure (phi v))) ->
  adequate NotStuck e state_init (fun v _ => phi v).
```

Theorem 8 (Adequacy)**.** *Let* φ *be a pure predicate on values. Suppose that in every Iris instantiation of the CJR ghost state one can prove*

$$\mathsf{wp}\, e \,\{\, v.\, \ulcorner \varphi(v) \urcorner \,\}.$$

Then, executing e *from* σ_0*, every reachable configuration can take a step or is a value, and every final value* v *satisfies* $\varphi(v)$.

What the proof does. It allocates four empty ghost maps, packages them as the CJR ghost state, and observes $\mathsf{wf}(\sigma_0)$. Iris's adequacy theorem for weakest preconditions then applies. The hypothesis is a proof that does not depend on a particular ghost name, so it can be used at the names chosen here.

What it does not say. It does not say that every closed expression is safe. An expression with a free variable, a null dereference, or a load of an address nobody allocated has no such proof, and the theorem is silent. It also does not extract a functional-correctness result beyond the pure φ that was proved. If the postcondition is only "the result is unit", adequacy says the program returns unit without getting stuck; the interesting memory facts were used and consumed inside the proof.

The closed examples `sum_one`, `bump_keep`, `cell`, `class_bump`, and `vtable_prog` are instantiated at this theorem. The other examples are open: their hypotheses are points-to facts, so they are stated as weakest preconditions rather than as runs from σ_0.

8 The examples

Each example uses a feature that the previous one does not. They are the files in `theories/examples/`, in the order below. The programs are written in the Cangjie fragment above. The Coq definitions are the core `expr` terms those files actually prove.

8.1 Arithmetic: `arith.v`

Program.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

func sumOne(): Int64 {
    var n = 1
    var s = 0
    while (1 <= n) {
        s = s + 1
        n = n - 1
    }
    s
}

main(): Unit {
    sumOne()
}
```

Theorem.

$$\mathsf{wp}\,\mathsf{sum_one}\,\{\, v.\ulcorner v = 1\urcorner\}.$$

From σ_0, the program returns 1 and does not get stuck (`sum_one_adequate`).

What is new. Nothing is allocated on either heap. The proof uses `var`, stack load and assignment, `while`, `if`, sequencing, and integer $+$ and $-$. The loop condition is true once: n is 1, then 0. On the true branch, s becomes 1 and n becomes 0. On the false branch the loop returns unit, and the trailing read returns s.

The stack facts at the end are $s \mapsto_s 1$ and $n \mapsto_s 0$. They are given back to `wp_var_bind` through the existential, and `pop` deletes them. The pure postcondition does not mention the stack. That is the scope rule from the previous section, exercised on a loop that really mutates.

8.2 Stack structs: `struct_upd.v`

Program.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

struct Cell {
    public var x: Int64
```

```
    public init(x: Int64) {
        this.x = x
    }
}

func bump(): Int64 {
    var p = Cell(0)
    p.x = 1
    p.x
}

func bumpKeep(): Int64 {
    let r = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 1))) }
    bump()
    unsafe { r.read() }
}

main(): Unit {
    bumpKeep()
}
```

Theorems.

$$\ell \mapsto_r v \vdash \mathsf{wp}\,\mathsf{bump}\,\{\, w.\ulcorner w = 1 \urcorner * \ell \mapsto_r v \,\},$$
$$\mathsf{wp}\,\mathsf{bump_keep}\,\{\, v.\ulcorner v = 0 \urcorner \,\}.$$

The closed run from σ_0 returns 0 (`bump_adequate`).

What is new. The struct lives in a stack slot, not on the raw heap. The field write is `wp_stack_field`: the slot changes from $\langle 0 \rangle$ to $\langle 1 \rangle$, and the read copies the component out with `wp_struct_load`. No raw address is allocated by bump itself.

The first theorem is the frame. Its hypothesis is an arbitrary raw cell, and the postcondition returns that same cell. The proof of the stack update never mentions it, so it cannot have been changed. The closed program makes the same point operationally: the fresh raw word is 0 before bump and 0 after, which is what the final load returns. The field update's result 1 is established on the way and then discarded by sequencing, because the observation we keep is the unchanged raw word.

8.3 One raw cell: `cell.v`

Program.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

func cell(): Unit {
    let p = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 1))) }
    unsafe { p.write(7) }
    unsafe { p.read() }
    unsafe { free(CPointer<Unit>(p)) }
}
```

```
main(): Unit {
    cell()
}
```

Theorem.

$$\mathsf{wp}\,\mathsf{cell}\,\{\,v.\,\ulcorner v = ()\urcorner\,\}.$$

From σ_0, the program returns unit and does not get stuck.

What is new. This is the first use of $\mapsto_r$ and $\mapsto_b$. Allocation produces $\ell \mapsto_r 0$ and $\ell \mapsto_b 1$. The store replaces the points-to fact by $\ell \mapsto_r 7$. The load reads 7 and returns the same fact. The program then discards that 7 by sequencing and frees the block. Free consumes both facts and returns unit.

The pure specification does not say "the loaded value was 7"; that fact is internal to the proof. What adequacy reports is that the whole expression, whose value is the value of `free`, returns unit without a stuck load, store, or double free. After `free`, no points-to fact remains, so a further load could not be verified.

8.4 Swap and aliasing: `swap.v`

Program.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

func swap(l1: CPointer<Int64>, l2: CPointer<Int64>): Unit {
    let t = unsafe { l1.read() }
    unsafe { l1.write(unsafe { l2.read() }) }
    unsafe { l2.write(t) }
}

main(): Unit {
    let l1 = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 1))) }
    let l2 = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 1))) }
    unsafe { l1.write(1) }
    unsafe { l2.write(2) }
    swap(l1, l2)
}
```

Theorems.

$$\ell_1 \mapsto_r v_1 * \ell_2 \mapsto_r v_2 \vdash \mathsf{wp}\,\mathsf{swap}(\ell_1, \ell_2)\,\{\,_.\ \ell_1 \mapsto_r v_2 * \ell_2 \mapsto_r v_1\,\},$$
$$\ell \mapsto_r v_1 * \ell \mapsto_r v_2 \vdash \bot.$$

What is new. The postcondition needs both cells. The proof loads ℓ_2 while still owning ℓ_1, stores into ℓ_1, then stores the saved value into ℓ_2. Each rule mentions only the cell it touches; the other fact sits in the context and is framed.

The second lemma is `raw_mapsto_exclusive`, re-exported as `swap_aliased`. It is not a second proof of the same swap. It says that the swap specification cannot be instantiated at one address twice: the hypothesis of two points-to facts at that address is absurd. An overlapping specification would be a different theorem—for example a single cell updated twice—and it is not claimed here.

8.5 Indexed blocks: `sum_block.v`

Program.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

func sumAt(p: CPointer<Int64>): Int64 {
    var i = 0
    var s = 0
    while (i <= 0) {
        s = s + unsafe { (p + i).read() }
        i = i + 1
    }
    s
}

main(): Unit {
    let p = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 1))) }
    unsafe { p.write(4) }
    sumAt(p)
}
```

Theorem.

$$p \mapsto_r 4 * p \mapsto_b 1 \vdash \mathsf{wp}\,\mathsf{sum_at}(p)\left\{ v.\ \ulcorner v = 4 \urcorner * p \mapsto_r 4 * p \mapsto_b 1 \right\}.$$

What is new. The address that is loaded is not the base written in the source; it is the value of Offset, that is $p + i$. On the only iteration, i is 0, and $p + 0 = p$, so the load is justified by the points-to fact at p. The addition itself is the pure offset rule; it does not consume the points-to fact. The loop then increments i to 1, the condition $i \leq 0$ fails, and the sum 4 is returned.

The block-length fact is never used by the load, and it is still there at the end. That is the split between "I may read this word" and "I may free this block". The example does not free. A load of $p + 1$ would not be provable from these hypotheses: there is no points-to fact at $p + 1$, and a one-word block does not contain that word.

The loop invariant, in the ordinary sense, is the assertion carried across the unfolding: i and s are stack cells, s holds the sum of the prefix already visited, and the raw word is unchanged. For a one-iteration loop the prefix is either empty or the single element 4.

8.6 Final classes: `class_upd.v`

Program.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
```

```
foreign func free(ptr: CPointer<Unit>): Unit

class Box {
    public var value: Int64
    public init(value: Int64) {
        this.value = value
    }
}

func classBump(): Int64 {
    let r = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 1))) }
    let o = Box(0)
    o.value = 5
    o.value
    unsafe { r.read() }
}

main(): Unit {
    classBump()
}
```

Theorem.

$$\mathsf{wp}\,\mathsf{class_bump}\,\{\,v.\,\ulcorner v = 0 \urcorner\,\}.$$

From σ_0, the program returns 0 and does not get stuck.

What is new. `new` allocates on the object map, not the raw map. The client receives $o \mapsto_o [0]0$. The field store changes it to $o \mapsto_o [0]5$, and the field load reads 5. The raw word allocated first is 0, is never stored, and is what the program returns.

So the closed observation is again the framed cell: object update does not change the raw heap. The proof makes the separation concrete. After `wp_alloc_one`, the raw points-to fact stays in the context through `wp_new`, `wp_field_store`, and `wp_field_load`. None of those lemmas takes a raw fact as a hypothesis. The final `wp_load` is what consumes it.

8.7 Framing across a call: `frame_call.v`

Program.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

func write(x: CPointer<Int64>): Unit {
    unsafe { x.write(9) }
}

func frameCall(p: CPointer<Int64>, q: CPointer<Int64>): Int64 {
    write(p)
    unsafe { q.read() }
}

main(): Unit {
```

```
    let p = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 1))) }
    let q = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 1))) }
    unsafe { q.write(3) }
    frameCall(p, q)
}
```

Theorem.

$$p \mapsto_r v * q \mapsto_r 3 \vdash \mathsf{wp}\ \mathsf{write_call}(p, q)\ \{\, w.\ \ulcorner w = 3 \urcorner * p \mapsto_r 9 * q \mapsto_r 3 \,\}.$$

What is new. The callee is a closure. `wp_rec` makes the closure, `wp_app` substitutes the pointer p for x, and `wp_store` needs only $p \mapsto_r v$. The fact $q \mapsto_r 3$ is not in the callee's precondition. It is framed across the call, then used by the load, which returns 3.

This is the frame rule applied at a function boundary. Nothing about `App` is special: substitution puts a value in the body, and the body is an ordinary store. The guarantee that q is unchanged is simply the guarantee that the store's proof did not receive q.

8.8 A hand-written vtable: `vtable.v`

Program.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

struct VTable {
    public var inc: (Counter) -> Unit
    public init(inc: (Counter) -> Unit) {
        this.inc = inc
    }
}

class Counter {
    public var value: Int64
    public init(value: Int64) {
        this.value = value
    }
}

func inc(self: Counter): Unit {
    self.value = self.value + 1
}

func run(): Int64 {
    let box = VTable(inc)
    let o = Counter(0)
    box.inc(o)
    o.value
}

main(): Unit {
    run()
```

```
}
```

Theorem.

$$\mathsf{wp}\ \mathsf{vtable_prog}\ \{\, v.\ \ulcorner v = 1 \urcorner \,\}.$$

From σ_0, the program returns 1 and does not get stuck.

What is new. The function is a value, the struct is a tuple holding that value, and the call is an ordinary application of a component that was loaded out of the tuple. There is no method table in the machine and no dynamic dispatch rule. A "method" is a function whose first argument is the object, and a "vtable" is a struct the programmer assembled.

The proof follows that reading. First, the closure is formed by `wp_rec`. Second, the struct zipper evaluates the closure and becomes $\langle inc \rangle$ by the pure struct rules. `new` yields $o \mapsto_o [0]0$. `wp_struct_load` copies the closure out; this does not touch the object. `wp_app` substitutes the object reference into the body. The body loads field 0, adds one, and stores 1. Finally, the trailing field load reads that 1, which is the value of the whole program.

All three regions of the story meet: a stack-free unboxed struct of code, a code pointer, and an object field. The raw heap is not used. The earlier examples already showed that leaving it unused is safe.

9 A linked list

The file `theories/examples/list.v` is a self-contained implementation of a singly linked list, together with the equations that say the implementation really is a list. It is larger than the earlier examples because the specification is not a single pure result: it is a predicate on the object heap, and the programs are functions that allocate and read nodes.

9.1 Why a node is an object with a boolean tag

A textbook list cell is a pair of a head and a pointer to the rest, and the empty list is the null pointer. CJR has null, and loading through null is stuck, which is the right machine behaviour. What it does not have is an equality test on pointers or on object references. `EqOp` compares integers only. A program therefore cannot branch on "is this pointer null?".

The sticking point is resolved by using a non-`open` class with three fields, and a boolean the language can already branch on.

```
Definition nil_body : expr :=
  Let (Some "n")
    (New [LitV (LitBool false); LitV (LitInt 0); LitV LitUnit] [])
    (Seq (FieldStore (Var "n") 2 (Var "n")) (Var "n")).

Definition cons_body : expr :=
  Let (Some "x") (StructLoad (Var "a") 0)
    (Let (Some "xs") (StructLoad (Var "a") 1)
      (New [] [Val (LitV (LitBool true)); Var "x"; Var "xs"])).
```

Field 0 is the tag: false means empty, true means cons. Field 1 is the head integer, which is 0 and unused when the tag is false. Field 2 is the tail. On an empty node it is that same node; on a cons node it is the value that represents the rest of the list, itself either an empty node or another

cons node. The allocation writes unit into the field and then stores the new object there before `nil` returns. The points-to fact owns the field. The value in the field is a pointer to the object, so the fact does not own the object a second time, and the predicate does not unfold. Objects are not freed, so a cons cell stays in the heap after its tail has been read. That matches the language: there is a `new` rule and no rule that deletes an object.

In Cangjie the operations are ordinary functions. Core application takes one argument, so `cons(x, xs)` lowers to a call on the struct of those two arguments. The other functions take the node.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

class Node {
    public var isCons: Bool
    public var value: Int64
    var next_raw: ?Node
    public mut prop next: Node {
        get() {
            match (this.next_raw) {
                case Some(n) => n
                case None => this
            }
        }
        set(n) { this.next_raw = n }
    }
    public init(isCons: Bool, value: Int64, next: ?Node) {
        this.isCons = isCons
        this.value = value
        this.next_raw = next
    }
}

func nil(): Node {
    let n = Node(false, 0, None)
    n.next = n
    n
}

func cons(x: Int64, xs: Node): Node {
    Node(true, x, xs)
}

func head(n: Node): Int64 {
    n.value
}

func tail(n: Node): Node {
    n.next
}

func empty(n: Node): Bool {
```

```
    if (n.isCons) { false } else { true }
}

main(): Unit {
    let e = nil()
    let l = cons(1, e)
    let h = head(l)
    let t = tail(l)
    empty(t)
}
```

A Cangjie constructor cannot receive the object it is building, so the printed class takes an optional tail and stores it in a raw field. Unit, the placeholder written by the allocation, is printed as `None`. The property `next` reads that raw field back as a `Node`: the stored node when it is present, and the node itself when it is empty. After `nil` returns, the raw tail has been set to the node. `empty` does not read `next`. The functions above are the Cangjie surface; `cons` of two arguments lowers to the unary core call on the struct $\langle x, xs \rangle$.

`empty` returns the negation of the tag. An empty node is empty; a cons node is not. The conditional is the only test. It does not inspect the tail pointer.

9.2 The representation predicate

On paper, a mathematical list xs is related to a value p by $\mathsf{isList}(xs, p)$.

Definition 9 (List representation)**.** $\mathsf{isList}([], p)$ *holds when* $p = \mathsf{obj}(o)$ *and*

$$o \mapsto_o [0]\mathsf{false} * o \mapsto_o [1]0 * o \mapsto_o [2]\mathsf{obj}(o).$$

$\mathsf{isList}(x :: xs, p)$ *holds when* $p = \mathsf{obj}(o)$ *and there is a tail value* q *with*

$$o \mapsto_o [0]\mathsf{true} * o \mapsto_o [1]x * o \mapsto_o [2]q * \mathsf{isList}(xs, q).$$

The Coq predicate is the same recursion, as a separation-logic assertion.

```
Fixpoint is_list (xs : list Z) (p : val) : iProp S :=
  match xs with
  | [] => exists o, pure (p = LitV (LitObj o)) *
      ObjId o |->o[0] LitV (LitBool false) *
      ObjId o |->o[1] LitV (LitInt 0) *
      ObjId o |->o[2] LitV (LitObj o)
  | x :: xs' => exists o nxt, pure (p = LitV (LitObj o)) *
      ObjId o |->o[0] LitV (LitBool true) *
      ObjId o |->o[1] LitV (LitInt x) *
      ObjId o |->o[2] nxt *
      is_list xs' nxt
  end.
```

What the separating conjunction is doing. The three fields of one node are three separate facts, so a proof may load the tag and still own the head and the tail. The recursive fact $\mathsf{isList}(xs, q)$ owns every node of the tail and nothing of the cons cell. Those two regions are disjoint. That is why "the tail is the original list" can be stated by returning the original pointer together with $\mathsf{isList}(xs, p)$, while the cons cell's own three fields sit beside it rather than inside it.

A value that is not an object reference does not satisfy isList. The lemma `is_list_obj` records the useful direction: if the predicate holds, the value is $\mathsf{obj}(o)$ for some o. The proofs rewrite with that equality before they substitute, so an abstract pointer is a literal object reference by the time it is placed in a struct. Substitution then computes.

9.3 What each operation establishes

Empty list.

$$\mathsf{wp}\,\mathsf{nil}()\,\{\,v.\ \mathsf{isList}([], v)\,\}.$$

`wp_new` of the three initialising values returns three points-to facts, with unit in the tail. `wp_field_store` then writes the new object into that field, which is the empty clause. Nothing else is allocated. This is `nil_spec`.

Cons.

$$\mathsf{isList}(xs, p) \vdash \mathsf{wp}\,\mathsf{cons}\,\langle x, p\rangle \left\{\, v.\ \exists o.\ v = \mathsf{obj}(o) * o \mapsto_o [0]\mathsf{true} * o \mapsto_o [1]x * o \mapsto_o [2]p * \mathsf{isList}(xs, p) \,\right\}.$$

The argument struct is unboxed. `wp_struct_load` copies x and then p out of it; those loads do not touch the heap. The body then allocates one object. The third field is the original pointer p, not a copy of the tail's nodes. The fact $\mathsf{isList}(xs, p)$ is framed across allocation: `wp_new` does not ask for it, so it cannot be changed. The postcondition is the unfolding of $\mathsf{isList}(x :: xs, v)$ with the tail pointer pinned to the pointer that was passed in. Pinning it is what makes the later inverse law an equality of values, not only an equality of mathematical lists up to some existential. This is `cons_spec`.

Head.

$$\mathsf{isList}(x :: xs, p) \vdash \mathsf{wp}\,\mathsf{head}\,p\,\{\,v.\ \ulcorner v = x \urcorner * \mathsf{isList}(x :: xs, p)\,\}.$$

The proof opens the cons clause, loads field 1, and puts the same facts back together. The list predicate after the call is the list predicate before the call. Head is a pure observation. This is `head_spec`.

Tail.

$$\mathsf{isList}(x :: xs, p) \vdash \mathsf{wp}\,\mathsf{tail}\,p \left\{\, v.\ \mathsf{isList}(xs, v) * \exists o.\ p = \mathsf{obj}(o) * o \mapsto_o [0]\mathsf{true} * o \mapsto_o [1]x * o \mapsto_o [2]v \,\right\}.$$

Loading field 2 returns the tail pointer and gives the points-to fact back. The recursive clause of isList already owned $\mathsf{isList}(xs, v)$ separately from the cons cell, so the postcondition splits the list into the cell that was examined and the list that remains. It does not duplicate nodes. This is `tail_spec`.

Emptiness.

$$\mathsf{isList}([], p) \vdash \mathsf{wp}\,\mathsf{empty}\,p\,\{\,v.\ \ulcorner v = \mathsf{true} \urcorner * \mathsf{isList}([], p)\,\},$$
$$\mathsf{isList}(x :: xs, p) \vdash \mathsf{wp}\,\mathsf{empty}\,p\,\{\,v.\ \ulcorner v = \mathsf{false} \urcorner * \mathsf{isList}(x :: xs, p)\,\}.$$

Both proofs load the tag and take one branch of `if`. The other branch is dead in that specification: a proof of the empty law never has to show anything about the cons branch, and conversely. The predicate is restored, so emptiness is an observation rather than a destructive read. These are `empty_nil` and `empty_cons`.

9.4 Adding an element and removing it

The program that the inverse law talks about is the composition of the two functions, not a fused implementation.

```
Definition cons_then_tail (x : Z) (p : val) : expr :=
  Let (Some "node") (call_cons x p)
    (App (Rec None (Some "n") tail_body) (Var "node")).
```

Theorem 10 (Cons then tail)**.**

$$\mathsf{isList}(xs, p) \vdash \mathsf{wp}\,(\mathsf{tail}\,(\mathsf{cons}\,\langle x, p\rangle))\,\{\,v.\ \ulcorner v = p \urcorner * \mathsf{isList}(xs, v)\,\}.$$

The Coq statement is `cons_then_tail_spec`. Read it as an equation of lists. The value returned by removing the element just added is the original pointer, and that pointer still represents the original mathematical list.

The proof follows the program in order.

1. `cons_spec` allocates the node and frames $\mathsf{isList}(xs, p)$. Its postcondition says field 2 of the new object is exactly p, and the original list predicate is still owned.

2. The `let` substitutes that object for `node`.

3. `tail` is the function above. It loads field 2. The load rule returns the value that the points-to fact describes, which is p, and it returns the points-to fact as well.

4. The cons cell's three fields are no longer needed for the equation. They are discarded. Iris assertions are affine, so ownership of leftover cells may be dropped. The object remains allocated in the machine; the specification simply stops mentioning it. What remains is $\ulcorner v = p \urcorner$ and $\mathsf{isList}(xs, p)$.

Two companion readings are already in the lemmas above, and they are the other usual equations. Head after cons returns x and still owns $\mathsf{isList}(x :: xs, \cdot)$. Emptiness distinguishes [] from $x :: xs$. Together they say that the tag, the head field, and the tail field agree with the mathematical constructors, and that reading them does not damage the list those constructors describe.

The next section proves in-place reversal of an arbitrary finite list using this representation. It opens one cons cell at a time, rewires its tail, and transfers that cell into a separate accumulator. Append remains outside this development.

10 In-place list reversal

The file `theories/examples/list_rev.v` proves the classic iterative reversal of a singly linked list. This example changes the links of existing cons cells. It does not allocate replacement cons cells, copy the integers, or change the tag and head fields. It does allocate one fresh empty sentinel at entry, because the two list predicates in the loop invariant must own separate empty nodes. The old sentinel remains allocated when the loop finishes. The functional theorem describes the reversed contents; the proof also explains which field changes at each iteration and why that mutation is safe.

10.1 The representation being reused

There is one list representation in the development: `list.is_list` from the preceding section. Reversal imports it rather than defining a second predicate. Its two clauses are

$$\mathsf{isList}([], p) \equiv \exists o.\ \ulcorner p = \mathsf{obj}(o) \urcorner * o \mapsto_o [0]\mathsf{false} * o \mapsto_o [1]0 * o \mapsto_o [2]\mathsf{obj}(o),$$
$$\mathsf{isList}(x :: v, p) \equiv \exists o, q.\ \ulcorner p = \mathsf{obj}(o) \urcorner * o \mapsto_o [0]\mathsf{true} * o \mapsto_o [1]x * o \mapsto_o [2]q * \mathsf{isList}(v, q).$$

The predicate owns all three fields of each node, including the terminal empty node. Its separating conjunction makes the recursive tail separate from the current cons cell. A finite list described this way cannot contain a cycle of cons cells or repeated ownership of a cons cell. The sentinel's self-link is allowed because the empty clause owns its three fields directly and does not recurse through that link.

This ownership is the permission to perform the reversal. Another logical client cannot simultaneously hold full ownership of the same fields. An ordinary program variable can still contain an alias to a node: the algorithm mutates that node, so observations through such an alias change too. The specification transfers the input list's ownership into ownership of the returned list; it does not promise that the original head still denotes the original sequence.

10.2 The client specification

The main statement is `reverse_spec`:

$$\mathsf{isList}(xs, p) \vdash \mathsf{wp}\ \mathsf{reverse}(p)\ \{\, r.\ \mathsf{isList}(\mathsf{rev}(xs), r)\ \}.$$

Here xs is a mathematical list of integers and p is a runtime value. The list predicate connects them. The output predicate establishes both the order of all values and a well-formed owned list. There is no positive-length precondition; the empty and singleton cases are included.

The theorem uses Iris's ordinary weakest precondition, hence partial correctness and safety. It states the postcondition whenever execution returns and rules out getting stuck under its ownership precondition. The loop proof uses induction on the unprocessed finite list, which explains why each iteration consumes one remaining element. This chapter does not claim a separate total-correctness theorem, exact instruction count, physical pointer identity theorem, or verified equivalence between the CJR semantics and the Cangjie compiler. Executable tests complement the checked CJR theorem.

By framing, a separate assertion R can accompany the input and output: $\mathsf{isList}(xs, p) * R$ is transformed into $\mathsf{isList}(\mathsf{rev}(xs), r) * R$. The proof needs none of the resources in R; a disjoint second list is a concrete example.

10.3 The iteration definition: `rev_step`

```
Definition rev_step : expr :=
  Let (Some "nxt") (FieldLoad (Var "cur") 2)
    (Seq (FieldStore (Var "cur") 2 (Var "acc"))
      (Seq (Assign "acc" (Var "cur"))
        (Assign "cur" (Var "nxt")))).
```

The immutable `let nxt` reads field 2 before the program overwrites it. Without this saved value, the algorithm would lose access to the remaining input. The field store then changes `cur.next` to the accumulator. The assignments make the current node the new accumulator head and advance

`cur` to the saved tail. Their order matters: assigning `cur` first would lose the reference needed for `acc := cur`.

Only field 2 is changed. The current node retains its true tag and its integer. After the store, that node and the old accumulator describe one new list with contents $x :: u$. The saved tail describes the remaining suffix v'. The logical split follows the physical change in links.

10.4 The loop definition: `rev_loop`

```
Definition rev_loop : expr :=
  While (FieldLoad (Var "cur") 0) rev_step.
```

The condition loads the current node's tag. True means a cons node and permits one iteration; false means the empty sentinel and exits. The program does not use null as the stopping condition. The guard never reads field 2 of the empty node, so its self-link does not cause an extra iteration.

10.5 The function body: `rev_body`

```
Definition rev_body : expr :=
  Let (Some "acc0") call_nil
    (VarBind "acc" (Var "acc0")
      (VarBind "cur" (Var "n")
        (Seq rev_loop (Var "acc")))).
```

`call_nil` allocates and initializes the fresh empty accumulator. `VarBind` allocates mutable stack bindings for `acc` and `cur`; `n` is the immutable function argument. The final expression reads the accumulator after the loop.

10.6 The call wrapper: `call_reverse`

```
Definition call_reverse (p : val_cjr) : expr :=
  App (Rec None (Some "n") rev_body) (Val p).
```

`call_reverse` packages the body as a closure with one immutable argument `n`, then applies it to the supplied runtime value. The wrapper makes the client theorem talk about one complete function call rather than an open expression with a free argument.

For an input with n cons cells, the source algorithm traverses those cells once and uses a constant number of local references. The only new list object is the empty sentinel. This is the intended linear-time, constant-extra-space algorithm; no resource-cost theorem is asserted by `reverse_spec`.

10.7 Loading a local: `sld`

```
Definition sld (z : Z) : expr := StackLoad (Val (LitV (LitStack z))).
```

A `VarBind` replaces reads of a mutable variable with loads from a fresh stack slot. `sld z` abbreviates this concrete stack-load expression. Its argument is the slot's location, not the object reference stored there. The iteration proof uses the stack points-to fact to obtain that reference and then separately uses object-field ownership to access the node.

10.8 Assigning a local: `sas`

```
Definition sas (z : Z) (e : expr) : expr :=
  StackAssign (Val (LitV (LitStack z))) e.
```

`sas z e` evaluates the expression e and writes the resulting value into stack slot z. It changes the local reference rather than any object field. This distinction accounts for the three different writes in one iteration: one object-link write and two local-reference writes.

10.9 The iteration after substitution: `rev_step_at`

```
Definition rev_step_at (la lc : Z) : expr :=
  Let (Some "nxt") (FieldLoad (sld lc) 2)
    (Seq (FieldStore (sld lc) 2 (sld la))
      (Seq (sas la (sld lc)) (sas lc (Var "nxt")))).
```

After allocating the accumulator and current bindings at locations l_a, l_c, this is the iteration that actually executes in the CJR semantics. It reads the saved tail through `sld lc`, stores `sld la` in the current object's link, and performs the two local assignments using `sas`. It exposes the evaluation contexts used in the ownership proof. It is a proof-facing form of `rev_step`, not a second algorithm.

10.10 The loop after substitution: `rev_loop_at`

```
Definition rev_loop_at (la lc : Z) : expr :=
  While (FieldLoad (sld lc) 0) (rev_step_at la lc).
```

The guard loads field 0 of the object referenced by the current stack slot, and the body is the substituted iteration. The accumulator location appears only in the body. Cangjie is printed from `rev_body`, whose local variables retain their source names; these administrative stack forms are used to connect that source expression to primitive proof rules.

10.11 Iteration substitution: statement and proof

```
Lemma rev_step_at_subst (la lc : Z) :
  subst_var "acc" la (subst_var "cur" lc rev_step) = rev_step_at la lc.
Proof.
  cbn [rev_step subst_var sas sld]. f_equal; repeat f_equal; reflexivity.
Qed.
```

This equality connects the named iteration with code operating on two stack slots. The proof reduces `subst_var` structurally through the expression constructors. `f_equal` reduces equality of matching constructors to equality of their arguments; the remaining equalities are reflexive. This is an ordinary Coq equality between syntax trees. It assumes no ownership and executes no command.

10.12 Loop substitution: statement and proof

```
Lemma rev_loop_at_subst (la lc : Z) :
  subst_var "acc" la (subst_var "cur" lc rev_loop) = rev_loop_at la lc.
Proof.
  unfold rev_loop_at. cbn [rev_loop subst_var sld].
  rewrite rev_step_at_subst. reflexivity.
Qed.
```

This proof unfolds the loop, reduces substitution in the condition, and uses `rev_step_at_subst` for its body. Thus it connects the entire named loop to the loop verified over concrete stack slots. The main function proof uses the reverse order of the substitutions, which is definitionally the same for these two distinct variable names.

10.13 The loop invariant: `rev_inv`

```
Definition rev_inv (xs : list Z) (la lc : Z) : iProp Σ :=
  ∃ u v acc cur,
    ⌜ rev u ++ v = xs ⌝ ∗
    StackId la ↦s acc ∗
    StackId lc ↦s cur ∗
    is_list v cur ∗
    is_list u acc.
```

The witnesses u, v are respectively the contents of the accumulator and the unprocessed suffix. The witnesses a, c are their runtime references. The assertion says

$$\exists u, v, a, c.\ \ulcorner \mathsf{rev}(u) +\!\!+ v = xs \urcorner * l_a \mapsto_s a * l_c \mapsto_s c * \mathsf{isList}(v, c) * \mathsf{isList}(u, a).$$

The accumulator already has reversed order. Reversing it mathematically recovers the processed prefix of the original input. Appending the remaining suffix must recover the entire original list. The stack facts describe the current values of the two mutable bindings, and the two list predicates give separate ownership of both chains.

For $xs = [1, 2, 3]$, the successive pairs (u, v) are $([], [1, 2, 3])$, $([1], [2, 3])$, $([2, 1], [3])$, and $([3, 2, 1], [])$. For instance, $\mathsf{rev}([2, 1]) +\!\!+ [3] = [1, 2, 3]$. At entry, $u = []$ and $v = xs$. At exit, $v = []$, and the equation gives $u = \mathsf{rev}(xs)$.

Fresh allocation of the accumulator sentinel is essential to this particular invariant. Reusing the original terminal node would demand its full ownership inside both list predicates. The fresh sentinel lets the two complete chains remain separate throughout the traversal. At exit the original sentinel is owned by $\mathsf{isList}([], c)$, while the new sentinel terminates the result.

10.14 Append right identity: statement and induction proof

```
Lemma app_nil_r' (xs : list Z) : xs ++ [] = xs.
Proof. induction xs as [|z xs IH]; simpl; [reflexivity|rewrite IH; reflexivity]. Qed.
```

`app_nil_r'` proves $xs +\!\!+ [] = xs$. Induction on xs has a reflexive empty case. In the cons case, simplification exposes the unchanged head and an append on the tail; rewriting the induction hypothesis completes the equality. This identity is needed when the loop exits.

10.15 Append associativity: statement and proof

```
Lemma app_assoc' (xs ys zs : list Z) : (xs ++ ys) ++ zs = xs ++ (ys ++ zs).
Proof. induction xs as [|x xs IH]; simpl; [reflexivity|rewrite IH; reflexivity]. Qed.
```

`app_assoc'` proves that append associates. Again, induction on the first list makes the empty case reflexive and reduces the cons case to the same statement for its tail. The identity changes grouping, not element order. It is the algebraic step that lets one processed element move between the prefix and suffix descriptions.

10.16 Reversing append: statement and proof

```
Lemma rev_app (xs ys : list Z) : rev (xs ++ ys) = rev ys ++ rev xs.
Proof.
  induction xs as [|x xs IH]; simpl.
  - rewrite app_nil_r'. reflexivity.
  - rewrite IH. rewrite app_assoc'. reflexivity.
Qed.
```

`rev_app` proves $\mathsf{rev}(xs +\!\!+ ys) = \mathsf{rev}(ys) +\!\!+ \mathsf{rev}(xs)$. The empty case reduces to right identity of append. In the cons case, $\mathsf{rev}(x :: xs) = \mathsf{rev}(xs) +\!\!+ [x]$. The induction hypothesis moves $\mathsf{rev}(ys)$ to the front; associativity then reconciles the two groupings of the three appended lists.

10.17 Reversing cons: statement and proof

```
Lemma rev_cons (x : Z) (u : list Z) : rev (x :: u) = rev u ++ [x].
Proof. simpl. done. Qed.
```

`rev_cons` is the defining cons equation of Coq's `rev`. Simplification already proves it. Even this small lemma is useful as a named explanation of why prepending x to the accumulator corresponds to extending the processed prefix at its right end.

10.18 The invariant equation after one step

```
Lemma rev_prefix_step (u v : list Z) (x : Z) (v' : list Z) :
  v = x :: v' → rev (x :: u) ++ v' = rev u ++ v.
Proof.
  intros ->. rewrite rev_cons. rewrite app_assoc'. simpl. reflexivity.
Qed.
```

Suppose $v = x :: v'$. Replacing the accumulator u by $x :: u$ and remaining list v by v' preserves the recovered original list:

$$\begin{aligned}\mathsf{rev}(x :: u) +\!\!+ v' &= (\mathsf{rev}(u) +\!\!+ [x]) +\!\!+ v' \\ &= \mathsf{rev}(u) +\!\!+ ([x] +\!\!+ v') \\ &= \mathsf{rev}(u) +\!\!+ (x :: v').\end{aligned}$$

The proof substitutes the premise $v = x :: v'$, rewrites `rev_cons`, uses associativity, and simplifies the singleton append. Thus the mutation proof can delegate its pure equality to this lemma.

10.19 The invariant equation at an empty suffix

```
Lemma rev_prefix_nil (u : list Z) :
  rev u ++ [] = rev u.
Proof. apply app_nil_r'. Qed.
```

`rev_prefix_nil` records the exit identity as a direct application of right identity. It is an explanatory corollary; the loop proof can use `app_nil_r'` directly.

10.20 Double reversal of mathematical lists: proof by induction

```
Lemma rev_rev (xs : list Z) : rev (rev xs) = xs.
Proof.
  induction xs as [|x xs IH]; simpl; [reflexivity|].
  rewrite rev_app. simpl. rewrite IH. reflexivity.
Qed.
```

`rev_rev` proves involution: $\mathsf{rev}(\mathsf{rev}(xs)) = xs$. The empty case is reflexive. For a cons list, the outer reversal sees an append; `rev_app` turns it into the reversed singleton followed by the double-reversed tail. Simplification and the induction hypothesis produce $x :: xs$.

10.21 Recovering the accumulator contents from the exit equation

```
Lemma rev_eq_rev (u xs : list Z) : rev u = xs → u = rev xs.
Proof.
  intros Heq. transitivity (rev (rev u)).
  - symmetry. apply rev_rev.
  - f_equal. exact Heq.
Qed.
```

`rev_eq_rev` derives $\mathsf{rev}(u) = xs \Rightarrow u = \mathsf{rev}(xs)$. Its proof passes through $\mathsf{rev}(\mathsf{rev}(u))$: involution gives the first equality, and congruence under `rev` gives the second. This is what converts the invariant equation at loop exit into the desired representation predicate for the result.

10.22 Opening a cons cell: definition and proof

```
  Lemma is_list_cons_destruct (x : Z) (v : list Z) (p : val_cjr) :
    is_list (x :: v) p -∗
    ∃ o nxt, ⌜ p = LitV (LitObj o) ⌝ ∗
      ObjId o ↦ₒ[0] LitV (LitBool true) ∗
      ObjId o ↦ₒ[1] LitV (LitInt x) ∗
      ObjId o ↦ₒ[2] nxt ∗
      is_list v nxt.
  Proof.
    iIntros "H". iDestruct "H" as (o nxt) "(%Hp & H0 & H1 & H2 & Hv)".
    iExists o, nxt. iSplit; [done|]. iFrame.
  Qed.
```

This lemma exposes exactly the resources in the cons clause of `is_list`. The pointer equality identifies the runtime value as an object reference. The three field facts authorize reading the tag, reading the head if needed, and reading or overwriting the link. The recursive predicate owns the suffix independently. `iDestruct` opens the existential witnesses and separating conjunction;

`iExists`, `iSplit`, and `iFrame` return the same resources in the stated form. No memory changes and no ownership is duplicated by this lemma.

10.23 Closing the new accumulator: definition and proof

```
Lemma is_list_snoc_acc (x : Z) (u : list Z) (acc : val_cjr) (o : loc) :
  is_list u acc -∗
  ObjId o ↦_o[0] LitV (LitBool true) -∗
  ObjId o ↦_o[1] LitV (LitInt x) -∗
  ObjId o ↦_o[2] acc -∗
  is_list (x :: u) (LitV (LitObj o)).
Proof.
  iIntros "Hu H0 H1 H2". iExists o, acc. iSplit; [done|]. iFrame.
Qed.
```

Despite its historical name, this lemma constructs a cons at the front of the accumulator. After the current node's link has become a, ownership of $\mathsf{isList}(u, a)$ and the three fields of that node yields $\mathsf{isList}(x :: u, \mathsf{obj}(o))$. The proof chooses o, a as the existential witnesses, proves the pointer equality by reflexivity, and frames the four supplied resources. This is folding the predicate after mutation; it is not allocation and it does not copy a node.

10.24 One iteration: the full ownership proof

```
Lemma wp_rev_step_at (xs u : list Z) (x : Z) (v : list Z)
    (la lc : Z) (acc cur : val_cjr) :
  rev u ++ x :: v = xs →
  StackId la ↦_s acc -∗ StackId lc ↦_s cur -∗
  is_list (x :: v) cur -∗ is_list u acc -∗
  WP (rev_step_at la lc)
    {{ _, ∃ nxt,
       ⌜ rev (x :: u) ++ v = xs ⌝ ∗
       StackId la ↦_s cur ∗ StackId lc ↦_s nxt ∗
       is_list v nxt ∗ is_list (x :: u) cur }}.
Proof.
  intros Hpure. iIntros "Ha Hc Hv Hu".
  iDestruct (is_list_cons_destruct with "Hv") as (o nxt)
    "(%Hp & Htag & Hhead & Hnext & Hv)".
  subst cur. unfold rev_step_at, sld, sas.
  iApply (wp_bind [LetCtx (Some "nxt")
    (Seq (FieldStore (StackLoad (Val (LitV (LitStack lc)))) 2
            (StackLoad (Val (LitV (LitStack la)))))
      (Seq (StackAssign (Val (LitV (LitStack la)))
              (StackLoad (Val (LitV (LitStack lc)))))
        (StackAssign (Val (LitV (LitStack lc))) (Var "nxt"))))]).
  iApply (wp_bind [FieldLoadCtx 2]).
  iApply (wp_stack_load with "Hc"). iIntros "!> _ Hc".
  iApply (wp_field_load with "Hnext"). iIntros "!> _ Hnext".
  iApply wp_let. simpl.
  iApply (wp_bind [SeqCtx
    (Seq (StackAssign (Val (LitV (LitStack la)))
            (StackLoad (Val (LitV (LitStack lc)))))
      (StackAssign (Val (LitV (LitStack lc))) (Val nxt)))]).
```

```
  iApply (wp_bind [FieldStoreLCtx 2 (StackLoad (Val (LitV (LitStack la))))]).
  iApply (wp_stack_load with "Hc"). iIntros "!> _ Hc".
  iApply (wp_bind [FieldStoreRCtx (LitV (LitObj o)) 2]).
  iApply (wp_stack_load with "Ha"). iIntros "!> _ Ha".
  iApply (wp_field_store with "Hnext"). iIntros "!> _ Hnext".
  iApply wp_seq.
  iApply (wp_bind [SeqCtx (StackAssign (Val (LitV (LitStack lc))) (Val nxt))]).
  iApply (wp_bind [StackAssignRCtx (LitV (LitStack la))]).
  iApply (wp_stack_load with "Hc"). iIntros "!> _ Hc".
  iApply (wp_stack_assign with "Ha"). iIntros "!> _ Ha".
  iApply wp_seq.
  iApply (wp_stack_assign with "Hc"). iIntros "!> _ Hc".
  iExists nxt. iSplit.
  { iPureIntro. rewrite (rev_prefix_step u (x :: v) x v); done. }
  iFrame "Ha Hc Hv".
  iApply (is_list_snoc_acc with "Hu Htag Hhead Hnext").
Qed.
```

The pure premise supplies the invariant equation with a nonempty remaining list. The four spatial premises supply the two stack slots and both lists. The proof opens the remaining cons cell and substitutes its pointer equality. It then follows the source iteration in evaluation order:

1. `wp_bind` with a `LetCtx` focuses on the initializer of `nxt`. A nested `FieldLoadCtx` first loads the current pointer from its stack slot. `wp_field_load` reads field 2, producing the saved tail reference and returning its ownership. The stack load also returns the stack fact. These returned facts must remain available for later commands.

2. `wp_let` substitutes the saved reference into the continuation. A `SeqCtx` focuses on the store while retaining both assignments as pending code. The store's left and right evaluation contexts load `cur` and `acc`, respectively. `wp_field_store` replaces only the link fact $o \mapsto_o [2]nxt$ by $o \mapsto_o [2]a$. The tag, head, old accumulator, and suffix stay framed.

3. After the store returns unit, `wp_seq` takes the sequencing step. The next `SeqCtx` focuses on `acc := cur`. A `StackAssignRCtx` loads the current reference before the stack write. `wp_stack_assign` replaces $l_a \mapsto_s a$ by $l_a \mapsto_s \mathsf{obj}(o)$, retaining ownership of l_c.

4. Another sequencing step reaches `cur := nxt`. The saved tail reference is already a value. The stack write replaces $l_c \mapsto_s \mathsf{obj}(o)$ by $l_c \mapsto_s nxt$. The program now has the references claimed by the new invariant.

5. The proof chooses the saved tail as the existential witness, applies `rev_prefix_step` to the pure equation, and frames the stack slots and suffix. Finally `is_list_snoc_acc` folds the modified node and old accumulator into the new accumulator predicate.

`iIntros "!> _ ..."` after each primitive memory rule introduces the later step and discards the step credit that the rule supplies. The remaining named hypothesis is the returned ownership. Discarding a credit is harmless; discarding the link or stack ownership would prevent a later access. The code uses explicit evaluation contexts because a store or assignment returns a value inside a pending sequence: the residual sequencing step must be handled before proving the next command. The bind rule arranges the continuation; it does not itself execute that sequencing step.

10.25 The exit assertion: `rev_done`

```
Definition rev_done (xs : list Z) (la lc : Z) : iProp Σ :=
  ∃ acc cur, StackId la ↦_s acc ∗ StackId lc ↦_s cur ∗
    is_list (rev xs) acc ∗ is_list [] cur.
```

An invariant alone does not imply that the loop has finished. For example, the entry state satisfies `rev_inv` with the entire input still unprocessed. The loop's postcondition therefore separately states that `cur` owns an empty list and `acc` owns the reversed input. Both stack facts are retained for the enclosing variable scopes. This is a stronger assertion than merely re-establishing the invariant, and avoids an invalid inference that $\mathsf{rev}(u) +\!\!+ v = xs$ forces $v = []$.

10.26 The loop theorem: induction and both branches

```
Lemma wp_rev_loop_run (xs v : list Z) (la lc : Z) :
  ∀ u acc cur, rev u ++ v = xs →
  StackId la ↦_s acc -∗ StackId lc ↦_s cur -∗
  is_list v cur -∗ is_list u acc -∗
  WP (rev_loop_at la lc) {{ _, rev_done xs la lc }}.
Proof.
  induction v as [|x v IH]; intros u acc cur Hpure;
    iIntros "Ha Hc Hv Hu".
  - iDestruct "Hv" as (o) "(%Hp & Htag & Hhead & Hnext)".
    subst cur. unfold rev_loop_at at 1.
    iApply wp_while.
    iApply (wp_bind [IfCtx
      (Seq (rev_step_at la lc) (While (FieldLoad (sld lc) 0) (rev_step_at la lc)))
      (Val (LitV LitUnit))]).
    iApply (wp_bind [FieldLoadCtx 0]).
    iApply (wp_stack_load with "Hc"). iIntros "!> _ Hc".
    iApply (wp_field_load with "Htag"). iIntros "!> _ Htag".
    iApply wp_if_false. iApply wp_value'.
    rewrite app_nil_r' in Hpure.
    apply rev_eq_rev in Hpure. subst u.
    iExists acc, (LitV (LitObj o)). iFrame "Ha Hc Hu".
    iExists o. iSplit; [done|]. iFrame.
  - iDestruct (is_list_cons_destruct with "Hv") as (o nxt)
      "(%Hp & Htag & Hhead & Hnext & Hv)".
    subst cur. unfold rev_loop_at at 1.
    iApply wp_while.
    iApply (wp_bind [IfCtx
      (Seq (rev_step_at la lc) (While (FieldLoad (sld lc) 0) (rev_step_at la lc)))
      (Val (LitV LitUnit))]).
    iApply (wp_bind [FieldLoadCtx 0]).
    iApply (wp_stack_load with "Hc"). iIntros "!> _ Hc".
    iApply (wp_field_load with "Htag"). iIntros "!> _ Htag".
    iApply wp_if_true.
    iApply (wp_bind [SeqCtx (rev_loop_at la lc)]).
    iApply (wp_wand with "[Ha Hc Htag Hhead Hnext Hv Hu]").
    { iApply (wp_rev_step_at xs u x v with "Ha Hc [Htag Hhead Hnext Hv] Hu").
      - exact Hpure.
      - iExists o, nxt. iSplit; [done|]. iFrame. }
    iIntros (w) "Hstep".
```

```
    iDestruct "Hstep" as (nxt') "(%Hpure' & Ha & Hc & Hv & Hu)".
    iApply wp_seq.
    iApply (IH (x :: u) (LitV (LitObj o)) nxt' Hpure' with "Ha Hc Hv Hu").
Qed.
```

The induction is on the mathematical remaining list v. The statement quantifies over u, a, c after fixing v, so the induction hypothesis is available for any new accumulator and current reference. Fixing those values before induction would make the hypothesis too weak for the updates.

Empty remaining list. Opening $\mathsf{isList}([], c)$ identifies the sentinel and gives its false tag. `wp_while` unfolds one loop test into an `if`. The `IfCtx` retains the two branches while the nested load context reads the current reference and its tag. The field load returns false and `wp_if_false` selects unit. There is no body iteration in this case. Right identity reduces the invariant equation to $\mathsf{rev}(u) = xs$; `rev_eq_rev` changes it to $u = \mathsf{rev}(xs)$. The proof supplies both references to `rev_done`, frames the accumulator and stack facts, and reconstructs the empty sentinel predicate from all three fields. The guard consumed none of the sentinel's ownership.

Nonempty remaining list. Opening $\mathsf{isList}(x :: v, c)$ gives the true tag and saved-tail witness. The guard proof reads the tag and selects the true branch. To invoke the body lemma, it must reassemble the cons predicate from the returned tag, head, link, and suffix ownership. A `SeqCtx` isolates the iteration from the recursive loop. `wp_wand` first proves the iteration with its own postcondition, then supplies that postcondition to the remaining proof.

The resulting existential tail reference and pure equation describe $x :: u$ and v. After `wp_seq`, the induction hypothesis applies to this strictly smaller remaining list with the updated references. It produces `rev_done` directly. The proof never assumes that the loop has exited while a cons remains, and never needs to guess a final accumulator from the original invariant equation alone.

10.27 Packaging the invariant: `wp_rev_loop`

```
Lemma wp_rev_loop (xs : list Z) (la lc : Z) :
  rev_inv xs la lc -∗
  WP (rev_loop_at la lc) {{ _, rev_done xs la lc }}.
Proof.
  iIntros "Hinv".
  iDestruct "Hinv" as (u v acc cur) "(%Hpure & Ha & Hc & Hv & Hu)".
  iApply (wp_rev_loop_run xs v la lc u acc cur Hpure with "Ha Hc Hv Hu").
Qed.
```

This wrapper opens the four existential witnesses of `rev_inv` and its pure equality, then applies `wp_rev_loop_run` with those witnesses and all four spatial resources. There is no additional operational argument. It is the interface used by the function proof, which can initialize an invariant without exposing its witnesses at the call site.

10.28 The complete function proof

```
Lemma reverse_spec (xs : list Z) (l : val_cjr) :
  is_list xs l -∗
  WP call_reverse l {{ v, is_list (rev xs) v }}.
Proof.
```

```
  iIntros "Hl".
  iDestruct (is_list_obj with "Hl") as (ol) "(%Hl & Hl)".
  subst l. unfold call_reverse.
  iApply (wp_bind [AppLCtx (Val (LitV (LitObj ol)))]).
  iApply wp_rec. iIntros "!> _".
  iApply wp_app. cbn [rev_body subst].
  iApply (wp_bind [LetCtx (Some "acc0")
    (VarBind "acc" (Var "acc0")
      (VarBind "cur" (Val (LitV (LitObj ol))) (Seq rev_loop (Var "acc"))))]).
  iApply (wp_wand with "[]").
  { iApply nil_spec. }
  iIntros (acc0) "Hnil". iApply wp_let. simpl.
  iApply wp_var_bind. iIntros (la) "Ha". simpl.
  iApply wp_var_bind. iIntros (lc) "Hc". simpl.
  change (subst_var "cur" lc (subst_var "acc" la rev_loop))
    with (rev_loop_at la lc).
  iApply (wp_bind [SeqCtx (sld la)]).
  iApply (wp_wand with "[Ha Hc Hl Hnil]").
  { iApply wp_rev_loop. iExists [], xs, acc0, (LitV (LitObj ol)).
    iSplit; [done|]. iFrame. }
  iIntros (w) "Hdone".
  iDestruct "Hdone" as (acc cur) "(Ha & Hc & Hrev & Hnil)".
  iApply wp_seq.
  iApply (wp_stack_load with "Ha"). iIntros "!> _ Ha".
  iExists cur. iFrame "Hc".
  iExists acc. iFrame "Ha Hrev".
Qed.
```

First `is_list_obj` identifies the input value as an object reference while retaining its list ownership. This matters because the language's value type also includes closures: substituting a variable through an arbitrary value need not leave that value syntactically unchanged. An object literal has no free variables, so the substitutions reduce as required.

The `AppLCtx` evaluates the closure, `wp_rec` creates it, and `wp_app` substitutes the object argument into the body. A `LetCtx` focuses on `call_nil`. The proof uses `wp_wand` with an empty resource selection to invoke `nil_spec`: the allocator requires no input ownership, and the original list remains framed outside this call. The returned sentinel gives $\mathsf{isList}([], a_0)$.

The two `wp_var_bind` applications allocate the accumulator and current stack slots. Their postconditions require returning ownership of each slot at scope exit. After substitution the loop is `rev_loop_at`, and the last expression is `sld` of the accumulator slot. A `SeqCtx` focuses on the loop. Its invariant uses witnesses $[], xs, a_0, p$; the pure equality simplifies to $xs = xs$, and the two stack facts and two list predicates are exactly the owned resources already available.

`wp_rev_loop` yields the stronger exit assertion. After the final sequencing step, `wp_stack_load` returns the accumulator reference and its slot ownership. The two `iExists` and `iFrame` steps satisfy the nested variable scopes' requirements in the reverse order of their introduction: current slot first, accumulator slot second. The result predicate $\mathsf{isList}(\mathsf{rev}(xs), a)$ remains for the client. Ownership of the old sentinel may be discarded because Iris is affine; this is not a deallocation step in the machine.

10.29 A client equation: reversing twice

```
Definition reverse_twice (p : val_cjr) : expr :=
  Let (Some "r") (call_reverse p)
    (App (Rec None (Some "n") rev_body) (Var "r")).
```

This client composes two calls using an immutable intermediate reference.

10.30 The double-reversal client theorem and its proof

```
Lemma reverse_twice_spec (xs : list Z) (p : val_cjr) :
  is_list xs p -∗ WP reverse_twice p {{ v, is_list xs v }}.
Proof.
  iIntros "Hl". unfold reverse_twice.
  iApply (wp_bind [LetCtx (Some "r")
    (App (Rec None (Some "n") rev_body) (Var "r"))]).
  iApply (wp_wand with "[Hl]").
  { iApply (reverse_spec with "Hl"). }
  iIntros (r) "Hr". iApply wp_let. simpl.
  iApply (wp_wand with "[Hr]").
  { iApply (reverse_spec with "Hr"). }
  iIntros (v) "Hv". rewrite rev_rev. iFrame.
Qed.
```

The proof focuses on the first call with `LetCtx`, applies `reverse_spec`, and receives ownership of a list with contents $\mathsf{rev}(xs)$. After substituting the returned reference, it applies the same theorem a second time. `rev_rev` rewrites the second result's contents to xs. This is an equation of represented sequences. Each call creates a fresh empty sentinel, so it is not a theorem that every heap field or the terminal sentinel's identity is restored.

10.31 The printed Cangjie program

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

class Node {
    public var isCons: Bool
    public var value: Int64
    var next_raw: ?Node
    public mut prop next: Node {
        get() {
            match (this.next_raw) {
                case Some(n) => n
                case None => this
            }
        }
        set(n) { this.next_raw = n }
    }
    public init(isCons: Bool, value: Int64, next: ?Node) {
        this.isCons = isCons
```

```
        this.value = value
        this.next_raw = next
    }
}

func nil(): Node {
    let n = Node(false, 0, None)
    n.next = n
    n
}

func cons(x: Int64, xs: Node): Node {
    Node(true, x, xs)
}

func head(n: Node): Int64 {
    n.value
}

func tail(n: Node): Node {
    n.next
}

func empty(n: Node): Bool {
    if (n.isCons) { false } else { true }
}

func reverse(n: Node): Node {
    let acc0 = nil()
    var acc = acc0
    var cur = n
    while (cur.isCons) {
        let nxt = cur.next
        cur.next = acc
        acc = cur
        cur = nxt
    }
    acc
}

main(): Unit {
    let e = nil()
    let l = cons(1, e)
    reverse(l)
}
```

`p_list_rev` in `emit_cj.v` declares the same `Node` layout, reuses the five list constructors and observers, and adds `reverse` with `rev_body`. Its closed example builds a singleton and reverses it. The printer renders the source `VarBind`s as mutable Cangjie locals and field 2 as `next`. The generated file is standalone so its node declarations do not collide with `list.cj` in the tests. It is included in the normal `make cj` pipeline and the expected-output check. The executable under test is this generated file, without a separately handwritten reversal implementation.

10.32 Executable tests and what they check

```
@Test
class ListRevTests {
    @TestCase
    func emptyList(): Unit {
        let e = nil()
        let r = reverse(e)
        @Expect(empty(r), true)
        @Expect(head(r), 0)
        @Expect(empty(tail(r)), true)
        @Expect(empty(e), true)
    }

    @TestCase
    func singleton(): Unit {
        let n = cons(7, nil())
        let r = reverse(n)
        @Expect(empty(r), false)
        @Expect(head(r), 7)
        @Expect(empty(tail(r)), true)
        // The original cons node now ends the result.
        @Expect(empty(tail(n)), true)
    }

    @TestCase
    func severalNodes(): Unit {
        let r = reverse(cons(1, cons(2, cons(3, cons(4, nil())))))
        @Expect(head(r), 4)
        @Expect(head(tail(r)), 3)
        @Expect(head(tail(tail(r))), 2)
        @Expect(head(tail(tail(tail(r)))), 1)
        @Expect(empty(tail(tail(tail(tail(r))))), true)
    }

    @TestCase
    func duplicatesAndNegativeValues(): Unit {
        let r = reverse(cons(-2, cons(0, cons(-2, cons(9, nil())))))
        @Expect(head(r), 9)
        @Expect(head(tail(r)), -2)
        @Expect(head(tail(tail(r))), 0)
        @Expect(head(tail(tail(tail(r)))), -2)
        @Expect(empty(tail(tail(tail(tail(r))))), true)
    }

    @TestCase
    func doubleReversal(): Unit {
        let r = reverse(reverse(cons(5, cons(-1, cons(5, nil())))))
        @Expect(head(r), 5)
        @Expect(head(tail(r)), -1)
        @Expect(head(tail(tail(r))), 5)
        @Expect(empty(tail(tail(tail(r)))), true)
    }
```

```
    @TestCase
    func rewiresOriginalNodesAndFramesOtherList(): Unit {
        let e = nil()
        let c = cons(3, e)
        let b = cons(2, c)
        let a = cons(1, b)
        let other = cons(42, nil())
        let r = reverse(a)
        @Expect(head(r), 3)
        // Observe the old references: copying would leave these links unchanged.
        @Expect(head(tail(c)), 2)
        @Expect(head(tail(b)), 1)
        @Expect(empty(tail(a)), true)
        @Expect(empty(e), true)
        @Expect(head(other), 42)
        @Expect(empty(tail(other)), true)
    }
}
```

The six test cases cover empty input, a singleton, four distinct elements, duplicates and negative values, double reversal, and mutation through saved references while a separate list remains unchanged. The nonempty sequence tests inspect every expected element and check the terminal empty tag, so a missing or extra node is observable. The mutation test saves references to all three original cons nodes and observes their changed links. A copying implementation that left those original links unchanged would fail that test. The second list tests the concrete behavior expected from framing.

These are finite runtime checks of the printed Cangjie on the installed compiler. They do not replace the general theorem over arbitrary mathematical lists. Conversely the Coq proof checks the CJR term and does not prove the compiler correct. The test command is

```
cjc generated/cj/list_rev.cj test/list_rev_test.cj \
    --test -o test/bin/list_rev_test
test/bin/list_rev_test
```

The dated test report records the actual run. The assumption audit includes `reverse_spec` and `reverse_twice_spec`; neither theorem is closed using an admitted obligation or an added axiom.

11 Arrays

The file `theories/examples/array.v` is a library, in the same sense as the list: the language has no array type, and the library builds one from a final class and a raw block. An array of n integers is an object with two fields and a contiguous block of n words. Field 0 holds n. Field 1 holds the base pointer of the block. Word i of the block holds element i. The index is a computed integer, so the elements cannot be object fields: a field index in the core is a static natural number, whereas `Offset` and `Load` take a value.

`Alloc` of zero words is stuck, so `make` asks for a positive length. An index outside $0, \ldots, n-1$ is likewise stuck: the specification owns only the words of the block, and a load with no points-to fact does not reduce.

11.1 The library

In Cangjie the value is a non-`open` class. Allocation of n words is `malloc` of n times the size of `Int64`; the core step is `Alloc` of the word count n, which returns a pointer to n zero words. `get` and `set` take the array and the index, and `set` also takes the new element. Core application is unary, so those calls lower to one struct argument, as `cons` did.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

class Array {
    public var len: Int64
    public var data: CPointer<Int64>
    public init(len: Int64, data: CPointer<Int64>) {
        this.len = len
        this.data = data
    }
}

func make(n: Int64): Array {
    let p = unsafe { CPointer<Int64>(malloc(UIntNative(8 * n))) }
    Array(n, p)
}

func length(a: Array): Int64 {
    a.len
}

func get(arr: Array, i: Int64): Int64 {
    let p = arr.data
    unsafe { (p + i).read() }
}

func set(arr: Array, i: Int64, x: Int64): Unit {
    let p = arr.data
    unsafe { (p + i).write(x) }
}

main(): Unit {
    let a = make(4)
    set(a, 0, 7)
    let x = get(a, 0)
    length(a)
}
```

The core terms are the same operations with the struct lowering made explicit.

```
Definition make_body : expr :=
  Let (Some "p") (Alloc (Var "n"))
    (New [] [Var "n"; Var "p"]).

Definition get_body : expr :=
```

```
  Let (Some "arr") (StructLoad (Var "args") 0)
    (Let (Some "i") (StructLoad (Var "args") 1)
      (Let (Some "p") (FieldLoad (Var "arr") 1)
        (Load (Offset (Var "p") (Var "i"))))).

Definition set_body : expr :=
  Let (Some "arr") (StructLoad (Var "args") 0)
    (Let (Some "i") (StructLoad (Var "args") 1)
      (Let (Some "x") (StructLoad (Var "args") 2)
        (Let (Some "p") (FieldLoad (Var "arr") 1)
          (Store (Offset (Var "p") (Var "i")) (Var "x"))))).
```

`length` is a field load of index 0.

11.2 The representation

Definition 11 (Array representation)**.** $\mathsf{isArray}(xs, a)$ *holds when* $a = \mathsf{obj}(o)$*, the base pointer is* p*, and*

$$o \mapsto_o [0]|xs| * o \mapsto_o [1]\mathsf{ptr}(p) * p \mapsto_b |xs| * \mathop{*}_{i<|xs|} (p+i) \mapsto_r xs_i.$$

The block fact is the right to treat $[p, p + |xs|)$ as one allocation. The separating conjunction of the words is what lets a proof load or store one index and still own the others. Two different indices are two different addresses, so the facts do not overlap.

The Coq predicate is that assertion. `array_cells` is the big separating conjunction, indexed by position in xs.

```
Definition array_cells (base : Z) (xs : list Z) : iProp S :=
  (* separating conjunction over i |-> xs_i at address base+i *)
  [ * list] i |-> x in xs,
    RawId (base + Z.of_nat i) |->r LitV (LitInt x).

Definition is_array (xs : list Z) (a : val) : iProp S :=
  exists o base, pure (a = LitV (LitObj o)) *
    ObjId o |->o[0] LitV (LitInt (Z.of_nat (length xs))) *
    ObjId o |->o[1] LitV (LitPtr base) *
    RawId base |->b (length xs) *
    array_cells base xs.
```

11.3 Allocation of n words

The one-word rule is not enough for `make`. The machine already steps `Alloc` of any positive word count: it zeroes that many words, records the length at the base, and advances the counter by the count. `wp_alloc` is the matching logic rule. From $0 < n$,

$$\mathsf{wp}\,\mathsf{alloc}(n)\left\{ v.\ \exists \ell.\ v = \mathsf{ptr}(\ell) * \ell \mapsto_b n * \mathop{*}_{i<n} (\ell+i) \mapsto_r 0 \right\}.$$

The proof is the one-word proof with a finite map of cells instead of a singleton. `init_cells` builds that map, `wf_alloc` shows the resulting machine is well formed, and the ghost-map update hands back one points-to fact per word.

11.4 What each operation establishes

Make. For $0 < n$,

$$\mathsf{wp}\,\mathsf{make}(n)\,\{\,a.\ \mathsf{isArray}(\mathsf{replicate}(n,0),a)\,\}.$$

Allocation produces the zero words and the block fact. The following `new` stores n and the pointer in the object. This is `make_spec`.

Length.

$$\mathsf{isArray}(xs,a) \vdash \mathsf{wp}\,\mathsf{length}(a)\,\{\,v.\ \ulcorner v = |xs|\urcorner * \mathsf{isArray}(xs,a)\,\}.$$

The proof loads field 0 and returns the same predicate. Length is an observation. This is `length_spec`.

Get. If $xs_i = x$, writing i for the mathematical index,

$$\mathsf{isArray}(xs,a) \vdash \mathsf{wp}\,\mathsf{get}(a,i)\,\{\,v.\ \ulcorner v = x\urcorner * \mathsf{isArray}(xs,a)\,\}.$$

The body reads the pointer from field 1, adds i, and loads that word. The lookup lemma for a big separating conjunction lends out the one cell and takes it back, so the other cells are not mentioned by the load rule and stay in the predicate. This is `get_spec`.

Set. If index i is in range,

$$\mathsf{isArray}(xs,a) \vdash \mathsf{wp}\,\mathsf{set}(a,i,x)\,\{\,v.\ \ulcorner v = ()\urcorner * \mathsf{isArray}(xs[i \mapsto x],a)\,\}.$$

The store updates that one word. The length stored in the object and the block length are unchanged, because replacing an element does not change $|xs|$. This is `set_spec`.

11.5 The array equations

The programs are the compositions of the functions above. Each one is a `let` of one call followed by the next, not a fused body.

Theorem 12 (Read after write)**.** *If index i is in range, then*

$$\mathsf{isArray}(xs,a) \vdash \mathsf{wp}\,(\mathsf{get}(\mathsf{set}(a,i,x),i))\,\{\,v.\ \ulcorner v = x\urcorner * \mathsf{isArray}(xs[i \mapsto x],a)\,\}.$$

If j is also in range and $i \neq j$, then

$$\mathsf{isArray}(xs,a) \vdash \mathsf{wp}\,(\mathsf{get}(\mathsf{set}(a,i,x),j))\,\{\,v.\ \ulcorner v = xs_j\urcorner * \mathsf{isArray}(xs[i \mapsto x],a)\,\}.$$

These are `set_then_get_same` and `set_then_get_other`. The first is `set_spec` followed by `get_spec` at the updated list, whose lookup at i is x. The second uses the same `set_spec`, then `get_spec` at j. The lookup of an updated list at a different index is the old element. The separating conjunction is why the store at i does not have a points-to fact for j among its hypotheses: that fact is framed, so it cannot change.

Theorem 13 (Length is invariant)**.** *If index i is in range, then*

$$\mathsf{isArray}(xs,a) \vdash \mathsf{wp}\,(\mathsf{length}(\mathsf{set}(a,i,x)))\,\{\,v.\ \ulcorner v = |xs|\urcorner * \mathsf{isArray}(xs[i \mapsto x],a)\,\}.$$

For $0 < n$ and $i < n$,

$$\mathsf{wp}\,(\mathsf{length}(\mathsf{make}(n)))\,\{\,v.\ \exists a.\ \ulcorner v = n\urcorner * \mathsf{isArray}(\mathsf{replicate}(n,0),a)\,\},$$
$$\mathsf{wp}\,(\mathsf{get}(\mathsf{make}(n),i))\,\{\,v.\ \exists a.\ \ulcorner v = 0\urcorner * \mathsf{isArray}(\mathsf{replicate}(n,0),a)\,\}.$$

The length equation is `set_then_length_spec`: after `set`, `length` reads the field that still holds $|xs|$. The two `make` equations are `make_then_length_spec` and `make_then_get_spec`. A fresh array is a vector of zeroes of the requested length, and reading it does not change that fact.

What this development does not prove is an arbitrary loop over the array, such as a sum of every cell. The cells are already a separating conjunction indexed by the mathematical list, which is the induction hypothesis such a loop would use. The equations above are the base: read and write agree with list update, and they do not disturb the length or the other cells.

12 Array lists

The file `theories/examples/array_list.v` lowers the resizable-buffer core of Cangjie's `ArrayList`: a default constructor, the live length, the capacity, emptiness, reading and writing an index, append, removal at an index, growth, and the version counter. The source class continues past that core, with an iterator, the `Collection` interface, ranges, higher-order methods, and an embedded sort. Those stay out. The core is monomorphic in `Int64`, and it uses the array library already proved: a final object, a raw block, and a word-by-word copy.

`Alloc` of zero words is stuck, so the only constructor is the default one, at capacity 16. There is no shift, so growth doubles by adding the capacity to itself. Array-list `get`, `set`, and `remove` access the backing buffer without range checks. Their specifications require an index in $[0, \mathit{size})$. CJR does not model exception propagation, catching, or recovery. `set` leaves the version alone. `add` and `remove` increment it. Growth replaces the buffer and drops the old block and the old array object. The source relies on collection for that drop. This language has a one-word free and no collection, so the old facts are not returned to the allocator. Iris is affine, and the proof simply does not use them again.

12.1 The library

In Cangjie the value is a non-`open` class. Field 0 is the buffer, itself the array of the previous section. Field 1 is the live length. Field 2 is the version. The buffer's own field 0 is the capacity, and its field 1 is the base pointer of a block of that many words. The live prefix of the block is the mathematical list. The tail is zero, which is what `Alloc` writes and what removal writes back.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

class Array {
    public var len: Int64
    public var data: CPointer<Int64>
    public init(len: Int64, data: CPointer<Int64>) {
        this.len = len
        this.data = data
    }
}

class ArrayList {
    public var myData: Array
    public var mySize: Int64
    public var myVersion: Int64
```

```
    public init(myData: Array, mySize: Int64, myVersion: Int64) {
        this.myData = myData
        this.mySize = mySize
        this.myVersion = myVersion
    }

    public func size(): Int64 {
        this.mySize
    }

    public func capacity(): Int64 {
        let buf = this.myData
        buf.len
    }

    public func version(): Int64 {
        this.myVersion
    }

    public func isEmpty(): Bool {
        let n = this.mySize
        n == 0
    }

    public func get(i: Int64): Int64 {
        let buf = this.myData
        let p = buf.data
        unsafe { (p + i).read() }
    }

    public func set(i: Int64, x: Int64): Unit {
        let buf = this.myData
        let p = buf.data
        unsafe { (p + i).write(x) }
    }

    public func grow(min: Int64): Unit {
        let buf = this.myData
        let pcap = buf.len
        let doubled = pcap + pcap
        let newCap = if (min <= doubled) { doubled } else { min }
        let src = buf.data
        let len = this.mySize
        let soff = 0
        let doff = 0
        let dst = unsafe { CPointer<Int64>(malloc(UIntNative(8 * newCap))) }
        let nb = Array(newCap, dst)
        var i = 0
        while ((i + 1) <= len) {
            unsafe { (dst + (i + doff)).write(unsafe { (src + (i + soff)).read() }) }
            i = i + 1
        }
        this.myData = nb
    }
```

```
    public func add(x: Int64): Unit {
        let n = this.mySize
        let buf = this.myData
        let pcap = buf.len
        let min = n + 1
        if (pcap <= n) {
            this.grow(min)
        }
        let buf2 = this.myData
        let p = buf2.data
        unsafe { (p + n).write(x) }
        let n1 = n + 1
        this.mySize = n1
        let ver = this.myVersion
        this.myVersion = ver + 1
    }

    public func remove(at: Int64): Int64 {
        let n = this.mySize
        let buf = this.myData
        let p = buf.data
        let removed = unsafe { (p + at).read() }
        let src = p
        let dst = p
        let soff = at + 1
        let doff = at
        let len = (n - at) - 1
        var i = 0
        while ((i + 1) <= len) {
            unsafe { (dst + (i + doff)).write(unsafe { (src + (i + soff)).read() }) }
            i = i + 1
        }
        let last = n - 1
        unsafe { (p + last).write(0) }
        this.mySize = last
        let ver = this.myVersion
        this.myVersion = ver + 1
        removed
    }
}

func initArrayList(): ArrayList {
    let p = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 16))) }
    let buf = Array(16, p)
    ArrayList(buf, 0, 0)
}

main(): Unit {
    let a = initArrayList()
    a.add(1)
    a.add(2)
    let r = a.remove(0)
    a.size()
```

```
}
```

Core application is unary, so each method lowers to a closure applied to one struct of arguments, as `get` and `set` did for arrays. The pure model of the three mutating operations is ordinary list algebra.

```
Definition al_set (xs : list Z) (i : nat) (x : Z) : list Z := <[i := x]> xs.
Definition al_add (xs : list Z) (x : Z) : list Z := xs ++ [x].
Definition al_remove (xs : list Z) (i : nat) : list Z :=
  take i xs ++ drop (S i) xs.
```

Appending puts x at index $|xs|$ and leaves every earlier index unchanged. Updating index i reads back x there and leaves every other index unchanged. Removing index i shortens the list by one and shifts the suffix down.

12.2 The representation

Two objects. The inner one is the buffer. The outer one is the list.

Definition 14 (Array-list representation)**.** *Write* $\mathsf{slots}(xs, c)$ *for* xs *followed by* $c - |xs|$ *zeroes, which is defined only for* $|xs| \le c$*.* $\mathsf{isBuf}(slots, b)$ *is the array predicate of the previous section with the whole capacity in place of the live prefix: field* 0 *holds* $|slots|$*, field* 1 *holds the base pointer, and the block owns every word of slots.* $\mathsf{isArrayList}(xs, c, v, a)$ *holds when* $0 < c$*,* $|xs| \le c$*,* $a = \mathsf{obj}(o)$*, and*

$$o \mapsto_o [0]b * o \mapsto_o [1]|xs| * o \mapsto_o [2]v * \mathsf{isBuf}(\mathsf{slots}(xs, c), b).$$

The Coq predicates are `slots_of`, `is_buf`, and `is_arraylist`. The cells are `array_cells`, reused from the array library, so a proof of a single load or store is the same lookup lemma as `get` and `set` there. The zero tail is part of the owned block. Appending into spare capacity overwrites the first zero. Removing the last live word writes a zero back, so the tail stays zero.

12.3 Copy and growth

Copy is one loop. It binds an index at 0 and, while that index plus one is at most the length, loads one word and stores it one step further along. The loop is `copy_vars`. Two invariants share that body.

The first is a copy between two blocks, used by growth. Source and destination do not overlap, so each iteration may read the source and write the destination in either order. `copy_disjoint` says that after *len* steps the destination holds the source chunk at the destination offset, and the source is unchanged.

The second is a forward shift on one block, used by removal. The source offset is the destination offset plus one, so each word is read before the next store overwrites it. `copy_shift` says that the live prefix is the list with the suffix shifted down by one. A left shift of length zero, which is removal of the last element, is the loop condition failing at once.

Growth asks for a minimum capacity. The new capacity is $2c$ when the minimum is at most $2c$, and the minimum otherwise.

$$\mathsf{growCap}(c, m) = \begin{cases} 2c & \text{if } m \le 2c, \\ m & \text{otherwise.} \end{cases}$$

Both arms allocate a positive block, copy the live prefix into it, and store the new array object in field 0. The mathematical list and the version are unchanged. The old block remains allocated

and is not mentioned by the postcondition. This is `grow_spec`. The Coq name of the capacity is `grow_cap`; doubling is `doubled`, which is addition because the core has no shift.

Append uses that rule only when the live length already equals the capacity. Then the minimum is the length plus one, which is at most $2c$ for $c > 0$, so the capacity doubles. When the length is strictly below the capacity, append writes into the zero tail and the capacity stays.

12.4 What each operation establishes

Init.

$$\mathsf{wp}\,\mathsf{init}()\,\{\,a.\ \mathsf{isArrayList}([], 16, 0, a)\,\}.$$

Allocation of 16 words produces the zero block. The inner object stores 16 and the pointer. The outer object stores that buffer, length 0, and version 0. This is `init_spec`. `default_capacity` is 16.

Size, capacity, emptiness, version. Each is a field load, or a load followed by a comparison with zero.

$$\mathsf{isArrayList}(xs, c, v, a) \vdash \mathsf{wp}\,\mathsf{size}(a)\,\{\,r.\ \ulcorner r = |xs| \urcorner * \mathsf{isArrayList}(xs, c, v, a)\,\},$$
$$\mathsf{isArrayList}(xs, c, v, a) \vdash \mathsf{wp}\,\mathsf{capacity}(a)\,\{\,r.\ \ulcorner r = c \urcorner * \mathsf{isArrayList}(xs, c, v, a)\,\},$$
$$\mathsf{isArrayList}(xs, c, v, a) \vdash \mathsf{wp}\,\mathsf{isEmpty}(a)\,\{\,r.\ \ulcorner r = (|xs| = 0) \urcorner * \mathsf{isArrayList}(xs, c, v, a)\,\},$$
$$\mathsf{isArrayList}(xs, c, v, a) \vdash \mathsf{wp}\,\mathsf{version}(a)\,\{\,r.\ \ulcorner r = v \urcorner * \mathsf{isArrayList}(xs, c, v, a)\,\}.$$

These are `size_spec`, `capacity_spec`, `is_empty_spec`, and `version_spec`. Capacity reads the buffer's length field, which `slots_of` keeps equal to c.

Get and set. If $xs_i = y$,

$$\mathsf{isArrayList}(xs, c, v, a) \vdash \mathsf{wp}\,\mathsf{get}(a, i)\,\{\,r.\ \ulcorner r = y \urcorner * \mathsf{isArrayList}(xs, c, v, a)\,\},$$
$$\mathsf{isArrayList}(xs, c, v, a) \vdash \mathsf{wp}\,\mathsf{set}(a, i, x)\,\{\,r.\ \ulcorner r = () \urcorner * \mathsf{isArrayList}(xs[i \mapsto x], c, v, a)\,\}.$$

Both go through the buffer pointer and the raw word at index i. `get` and `set` perform the load or store directly, using the valid-index precondition. `set` stores into the live prefix and does not touch field 2. These are `al_get_spec` and `al_set_spec`. The names are prefixed because the array library already exports `get_spec` and `set_spec`.

Add.

$$\mathsf{isArrayList}(xs, c, v, a) \vdash \mathsf{wp}\,\mathsf{add}(a, x)\,\{\,r.\ \ulcorner r = () \urcorner * \mathsf{isArrayList}(xs +\!\!+ [x],\ \mathsf{addCap}(c, |xs|),\ v + 1, a)\,\}.$$

$\mathsf{addCap}(c, n)$ is c when $n < c$, and $\mathsf{growCap}(c, n + 1)$ otherwise. The body writes x at the old length, stores the new length, and increments the version. When the buffer is already full it first calls `grow`. This is `add_spec`.

Remove. If $xs_j = y$,

$$\mathsf{isArrayList}(xs, c, v, a) \vdash \mathsf{wp}\,\mathsf{remove}(a, j)\,\{\,r.\ \ulcorner r = y \urcorner * \mathsf{isArrayList}(\mathsf{delete}(xs, j),\ c,\ v + 1, a)\,\}.$$

$\mathsf{delete}(xs, j)$ is the prefix of length j followed by the suffix after j. The body loads y before the shift, copies the suffix down by one, writes zero at the old last index, stores the new length, and increments the version. The capacity is the same block. This is `remove_spec`.

12.5 The array-list equations

The programs are compositions of the calls above. Each one is a `let` of one call followed by the next, proved from the triples by framing, as `set_then_get` was for arrays.

Theorem 15 (A fresh list)**.**

$$\begin{aligned}
&\mathsf{wp}\,(\mathsf{size}(\mathsf{init}()))\,\{\,r.\ \exists a.\ \ulcorner r = 0\urcorner * \mathsf{isArrayList}([], 16, 0, a)\,\},\\
&\mathsf{wp}\,(\mathsf{isEmpty}(\mathsf{init}()))\,\{\,r.\ \exists a.\ \ulcorner r = \mathsf{true}\urcorner * \mathsf{isArrayList}([], 16, 0, a)\,\},\\
&\mathsf{wp}\,(\mathsf{capacity}(\mathsf{init}()))\,\{\,r.\ \exists a.\ \ulcorner r = 16\urcorner * \mathsf{isArrayList}([], 16, 0, a)\,\},\\
&\mathsf{wp}\,(\mathsf{version}(\mathsf{init}()))\,\{\,r.\ \exists a.\ \ulcorner r = 0\urcorner * \mathsf{isArrayList}([], 16, 0, a)\,\}.
\end{aligned}$$

Size and emptiness are `init_then_size_spec` and `init_then_empty_spec`. Capacity and version are `init_then_capacity_spec` and `init_then_version_spec`.

Theorem 16 (Append)**.**

$$\begin{aligned}
&\mathsf{isArrayList}(xs, c, v, a) \vdash \mathsf{wp}\,(\mathsf{size}(\mathsf{add}(a, x)))\\
&\qquad\{\,r.\ \ulcorner r = |xs| + 1\urcorner * \mathsf{isArrayList}(xs +\!\!+ [x],\ \mathsf{addCap}(c, |xs|),\ v + 1, a)\,\},\\
&\mathsf{isArrayList}(xs, c, v, a) \vdash \mathsf{wp}\,(\mathsf{get}(\mathsf{add}(a, x), |xs|))\\
&\qquad\{\,r.\ \ulcorner r = x\urcorner * \mathsf{isArrayList}(xs +\!\!+ [x],\ \mathsf{addCap}(c, |xs|),\ v + 1, a)\,\}.
\end{aligned}$$

If $i < |xs|$ and $xs_i = y$, the same append followed by `get` *at i returns y. The capacity is c when $|xs| < c$, and $2c$ when $|xs| = c$. The version after append is $v + 1$.*

Size and the new-index lookup are `add_then_size_spec` and `add_then_get_new_spec`. An older index is `add_then_get_old_spec`. Spare capacity is `add_then_capacity_room_spec`, a full buffer is `add_then_capacity_full_spec`, and the version is `add_then_version_spec`. The lookup at the new index is the lemma that an appended element sits at the old length. The lookup at an older index is the lemma that append does not change a shorter prefix, including across the growth that replaces the block.

Theorem 17 (Update)**.** *If $xs_i = y$, then reading index i after writing x there returns x, and the version is still v. If j is also in range and $i \neq j$, reading j returns xs_j. The length after the update is still $|xs|$.*

The two lookups are `set_then_get_same_spec` and `set_then_get_other_spec`. Length and version are `set_then_size_spec` and `set_then_version_spec`. The version equation is the observation that `set` does not store field 2.

Theorem 18 (Append then remove)**.**

$$\begin{aligned}
&\mathsf{isArrayList}(xs, c, v, a) \vdash \mathsf{wp}\,(\mathsf{remove}(\mathsf{add}(a, x), |xs|))\\
&\qquad\{\,r.\ \ulcorner r = x\urcorner * \mathsf{isArrayList}(xs,\ \mathsf{addCap}(c, |xs|),\ v + 2, a)\,\}.
\end{aligned}$$

Removing an in-range index and then reading the size returns $|xs| - 1$.

The round trip is `add_then_remove_last_spec`. Removal of the element just appended restores xs. The capacity is whatever append left behind, which may be $2c$, and the version has been incremented twice. The size equation is `remove_then_size_spec`.

What this development does not prove is the iterator, equality of two lists, or a bulk operation such as `filter`. The predicate already owns the live prefix and the zero tail as one separating conjunction, which is the induction hypothesis a further loop would use. The equations above are the base: length, lookup, update, append, and removal agree with the list algebra, growth only changes the capacity, and the version moves only on append and removal.

13 Quicksort

`theories/examples/qsort.v` is an in-place Lomuto quicksort of `Int64`. One Gallina function is the functional specification. One core-language program, parameterised by length, read, and write, is proved to compute that function. The fixed array and the array list each supply those three operations. The algorithm applies their specifications and leaves isArray and isArrayList folded.

The slice is half-open, $[lo, hi)$. The pivot is the element at index $hi - 1$. A scan walks j from lo up to that index. Each value $\leq$ the pivot is swapped into the stored prefix and the prefix boundary i advances. The pivot is then swapped to i. The recursive calls sort $[lo, i)$ and $[i + 1, hi)$. The exported operation reads the length n and sorts $[0, n)$: every word of an array, or the live prefix of an array list. The arithmetic in the program is subtraction and $\leq$.

13.1 The functional algorithm

Lists are mathematical lists of integers. xs_k means the lookup $xs!!k$, and $xs[k \mapsto z]$ means the update $\langle k := z \rangle\, xs$. A missing lookup leaves the list alone.

```
Definition swap_list (xs : list Z) (i j : nat) : list Z :=
  match xs !! i, xs !! j with
  | Some xi, Some xj => <[i := xj]> (<[j := xi]> xs)
  | _, _ => xs
  end.
```

swap(xs, i, j) exchanges the elements at two in-range indexes. The code performs the two writes in the other order, first index i and then index j. When both lookups exist, the two orders agree: if $i = j$ both are the identity, and if $i \neq j$ the updates commute. That equation is `swap_list_code`.

```
Lemma swap_list_code xs i j xi xj :
  xs !! i = Some xi -> xs !! j = Some xj ->
  <[j := xi]> (<[i := xj]> xs) = swap_list xs i j.
```

The scan is a fuelled recursion. The fuel n is the number of indexes still to visit. i is the boundary of the stored prefix, j is the next index to read, and *bound* is the index of the pivot, which the scan does not move.

```
Fixpoint lomuto (n : nat) (xs : list Z) (pivot : Z) (i j bound : nat)
  : list Z * nat :=
  match n with
  | O => (xs, i)
  | S n =>
      if Nat.ltb j bound then
        match xs !! j with
        | Some v =>
            if Z.leb v pivot then
              lomuto n (swap_list xs i j) pivot (S i) (S j) bound
            else lomuto n xs pivot i (S j) bound
        | None => (xs, i)
        end
      else (xs, i)
  end.
```

While $j <$ *bound* and index j holds v, a value $v \leq$ *pivot* is exchanged with index i and both

cursors advance. A value $v > pivot$ leaves the list unchanged and advances only j. The result is the list and the final boundary.

Partition reads the pivot, runs the scan from $i = j = lo$ with fuel $(hi - 1) - lo$, and exchanges the pivot with the returned boundary.

```
Definition partition_list (xs : list Z) (lo hi : nat) : list Z * nat :=
  match xs !! (hi - 1)%nat with
  | Some pivot =>
      match lomuto ((hi - 1) - lo)%nat xs pivot lo lo (hi - 1)%nat with
      | (zs, q) => (swap_list zs q (hi - 1)%nat, q)
      end
  | None => (xs, lo)
  end.
```

The recursive sort is also fuelled. The fuel is at least the length of the slice. A slice of length at most one is already finished. A longer slice is partitioned, the left part is sorted, and the right part is sorted on that result. Both recursive calls reuse the predecessor fuel: each of those slices has length at most $hi - lo - 1$.

```
Fixpoint qsort_slice (n : nat) (xs : list Z) (lo hi : nat) : list Z :=
  match n with
  | O => xs
  | S n =>
      if Nat.leb hi (lo + 1)%nat then xs
      else
        let '(ys, p) := partition_list xs lo hi in
        qsort_slice n (qsort_slice n ys lo p) (S p) hi
  end.

Definition qsort (xs : list Z) : list Z :=
  qsort_slice (length xs) xs 0%nat (length xs).
```

$\mathsf{qsort}(xs)$ sorts the whole list. The test $hi \leq lo + 1$ is the Gallina form of the program test $(lo + 1) \leq (hi - 1)$ failing. On a slice of length 0 or 1 the function returns its input.

13.2 Sortedness and permutation

A slice is sorted when every pair of elements in it is in non-decreasing order. Indexes outside the slice are not constrained.

```
Definition slice_sorted (xs : list Z) (lo hi : nat) : Prop :=
  forall i j x y, (lo <= i)%nat -> (i < j)%nat -> (j < hi)%nat ->
    xs !! i = Some x -> xs !! j = Some y -> (x <= y)%Z.
```

In mathematics, $\mathsf{sorted}(xs, lo, hi)$ means

$$lo \leq i < j < hi \wedge xs_i = x \wedge xs_j = y \implies x \leq y.$$

Equal values satisfy the comparison in either order. The specification records the multiset of values, which a permutation preserves, and this order.

The slice specification is the induction hypothesis of the fuel recursion.

$$ys = \mathsf{qsortSlice}(n, xs, lo, hi).$$

The source writes a permutation as $x \equiv_p y$, a pure assertion in corner brackets, and a stack cell as $l \mapsto_s v$. The listings spell those with `Permutation`, `bi_pure`, and `stack_mapsto`. The wand is written `-*`.

Lemma 19 (Slice)**.** *If $hi \le |xs|$ and $hi - lo \le n$, then $|ys| = |xs|$, ys is a permutation of xs, ys and xs agree at every index outside $[lo, hi)$, and* sorted(ys, lo, hi)*.*

```
Lemma qsort_slice_spec n xs lo hi :
  (hi <= length xs)%nat ->
  (hi - lo <= n)%nat ->
  let ys := qsort_slice n xs lo hi in
  length ys = length xs /\ Permutation ys xs /\
  (forall k, (k < lo \/ hi <= k)%nat -> ys !! k = xs !! k) /\
  slice_sorted ys lo hi.
```

The whole-list facts are that lemma at $lo = 0$, $hi = |xs|$, and $n = |xs|$.

Lemma 20 (Permutation)**.** qsort(xs) *is a permutation of xs.*

```
Lemma qsort_perm xs : Permutation (qsort xs) xs.
```

Lemma 21 (Sorted)**.** sorted(qsort(xs), 0, $|xs|$)*.*

```
Lemma qsort_sorted xs : slice_sorted (qsort xs) 0%nat (length xs).
```

One concrete run is computed in Coq.

```
Lemma qsort_example : qsort [3%Z; 1%Z; 2%Z] = [1%Z; 2%Z; 3%Z].
```

13.3 The partition invariant

The scan is the one loop. Its invariant relates the list at the start of the slice, xs_0, to the list xs at an intermediate cursor (i, j). The pivot value sits at *bound* throughout the scan. The final exchange of the pivot happens after the loop, so the invariant still has the pivot at the end of the slice.

```
Record lomuto_inv (xs0 xs : list Z) (lo i j bound : nat) (pivot : Z) : Prop := {
  linv_len : length xs = length xs0;
  linv_perm : Permutation xs xs0;
  linv_lo : (lo <= i)%nat;
  linv_ij : (i <= j)%nat;
  linv_jb : (j <= bound)%nat;
  linv_lenb : (bound < length xs)%nat;
  linv_piv : xs !! bound = Some pivot;
  linv_out : forall k, (k < lo \/ bound < k)%nat -> xs !! k = xs0 !! k;
  linv_left : forall k x, (lo <= k < i)%nat -> xs !! k = Some x -> (x <= pivot)%Z;
  linv_mid : forall k x, (i <= k < j)%nat -> xs !! k = Some x -> (pivot < x)%Z
}.
```

Read as a picture of $[lo, bound]$:

- The length is unchanged, and the list is a permutation of the original.

- The cursors satisfy $lo \le i \le j \le bound$, and $bound$ is in range.
- Index $bound$ still holds the pivot.
- Every index outside $[lo, bound]$ is unchanged.
- Every element of $[lo, i)$ is $\le$ the pivot.
- Every element of $[i, j)$ is $>$ the pivot.
- The segment $[j, bound)$ has not been classified yet.

At the start, $i = j = lo$, so both classified segments are empty.

```
Lemma lomuto_inv_start xs lo bound pivot :
  (lo <= bound)%nat ->
  (bound < length xs)%nat ->
  xs !! bound = Some pivot ->
  lomuto_inv xs xs lo lo lo bound pivot.
```

One iteration preserves the invariant. If $j < bound$ and $xs_j = v$, the next state is the swapped list with both cursors advanced when $v \le pivot$, and the same list with only j advanced when $v > pivot$. In the first branch the old element at i, which was in the unclassified or the greater segment, moves to j, and v becomes the new end of the lesser prefix. In the second branch v becomes the new end of the greater segment.

```
Lemma lomuto_step xs0 xs lo i j bound pivot v :
  lomuto_inv xs0 xs lo i j bound pivot ->
  (j < bound)%nat ->
  xs !! j = Some v ->
  lomuto_inv xs0
    (if Z.leb v pivot then swap_list xs i j else xs)
    lo (if Z.leb v pivot then S i else i) (S j) bound pivot.
```

Running the fuelled scan from a state that satisfies the invariant, with fuel $bound - j$, finishes in a state whose cursors are $(p, bound)$.

```
Lemma lomuto_go n xs0 xs lo i j bound pivot ys p :
  lomuto_inv xs0 xs lo i j bound pivot ->
  (n = bound - j)%nat ->
  lomuto n xs pivot i j bound = (ys, p) ->
  lomuto_inv xs0 ys lo p bound bound pivot.
```

The final exchange moves the pivot from $bound$ to p. What partition establishes, for a non-empty in-range slice, is the predicate `part_ok`.

```
Definition part_ok (xs ys : list Z) (lo hi p : nat) (pivot : Z) : Prop :=
  length ys = length xs /\ Permutation ys xs /\ (lo <= p < hi)%nat /\
  ys !! p = Some pivot /\
  (forall k, (k < lo \/ hi <= k)%nat -> ys !! k = xs !! k) /\
  (forall k x, (lo <= k < p)%nat -> ys !! k = Some x -> (x <= pivot)%Z) /\
  (forall k x, (p < k < hi)%nat -> ys !! k = Some x -> (pivot < x)%Z).

Lemma partition_list_ok xs lo hi ys p :
```

```
  (lo < hi)%nat -> (hi <= length xs)%nat ->
  partition_list xs lo hi = (ys, p) ->
  exists pivot, part_ok xs ys lo hi p pivot.
```

So ys has the same length as xs and is a permutation of it, the returned index p lies in $[lo, hi)$, index p holds the pivot, indexes outside the slice are untouched, $[lo, p)$ is $\leq$ the pivot, and (p, hi) is $>$ the pivot.

The recursive calls sort those two sides. A permutation of a side can rearrange it, so the comparison with the pivot has to be re-established after the recursive call. The argument counts elements that fail the comparison. $\mathsf{count}(P, xs)$ is the number of elements of xs for which P returns true.

```
Fixpoint count_if (P : Z -> bool) (xs : list Z) : nat :=
  match xs with
  | [] => 0
  | x :: xs => (if P x then 1%nat else 0%nat) + count_if P xs
  end.
```

If a slice of xs lies entirely in P, a list ys of the same length that permutes xs and agrees with xs outside that slice still lies entirely in P on that slice. The count of failures on the slice is zero on both sides, because the count is preserved by permutation and the outside regions contribute the same count.

```
Lemma region_preserved (P : Z -> bool) (xs ys : list Z) (lo hi : nat) :
  length ys = length xs ->
  (lo <= hi <= length xs)%nat ->
  (forall k, (k < lo \/ hi <= k)%nat -> ys !! k = xs !! k) ->
  Permutation ys xs ->
  (forall k x, (lo <= k < hi)%nat -> xs !! k = Some x -> P x = true) ->
  forall k x, (lo <= k < hi)%nat -> ys !! k = Some x -> P x = true.
```

Applied to $P(z) = (z \leq pivot)$ on the left of p, and to $P(z) = (pivot < z)$ on the right of p, this is what lets the two sorted sides and the pivot in the middle combine into one sorted slice. A slice of length at most one is sorted outright.

```
Lemma slice_sorted_short xs lo hi :
  (hi <= lo + 1)%nat -> slice_sorted xs lo hi.
```

Two adjacent sorted slices, with every element of $[lo, p)$ at most the pivot and every element of (p, hi) greater than the pivot, form one sorted slice.

```
Lemma slice_sorted_join xs lo p hi pivot :
  (lo <= p)%nat -> (p < hi)%nat ->
  xs !! p = Some pivot ->
  (forall k x, (lo <= k < p)%nat -> xs !! k = Some x -> (x <= pivot)%Z) ->
  (forall k x, (p < k < hi)%nat -> xs !! k = Some x -> (pivot < x)%Z) ->
  slice_sorted xs lo p ->
  slice_sorted xs (S p) hi ->
  slice_sorted xs lo hi.
```

13.4 The length, get, and set interface

The core language has no interfaces and no generics, so the shared program is a Coq section. Its parameters are a ghost type G, a representation R, and three expression bodies.

```
Variable G : Type.
Variable R : G -> list Z -> val -> iProp.
Variable len_body get_body set_body : expr.
```

$R(g, xs, a)$ says that the value a currently represents the list xs, with ghost information g. For the array, G is unit and R ignores it. For the array list, g is the pair of the capacity and the version.

Each body must be closed apart from its own parameter. Length binds `"a"`. Reading and writing bind `"args"`. Substitution of any other name, whether a value or a stack location, leaves the body unchanged. The six hypotheses have this shape.

```
Hypothesis len_sub : forall x v, x <> "a" -> subst x v len_body = len_body.
Hypothesis len_subvar :
  forall x (l : loc), x <> "a" -> subst_var x l len_body = len_body.
Hypothesis get_sub : forall x v, x <> "args" -> subst x v get_body = get_body.
Hypothesis get_subvar :
  forall x (l : loc), x <> "args" -> subst_var x l get_body = get_body.
Hypothesis set_sub : forall x v, x <> "args" -> subst x v set_body = set_body.
Hypothesis set_subvar :
  forall x (l : loc), x <> "args" -> subst_var x l set_body = set_body.
```

The three specifications are the only facts the algorithm applies about the library. Each one is stated at the call form the program builds: a closure applied to one struct of arguments. The ghost g is the same in the precondition and the postcondition. A write changes the list and leaves g alone.

```
Hypothesis len_spec : forall g xs (a : val),
  R g xs a -*
  WP App (Rec None (Some "a") len_body) (Val a)
    {{ v, bi_pure (v = LitV (LitInt (Z.of_nat (length xs)))) * R g xs a }}.

Hypothesis get_spec : forall g xs (a : val) (i : nat) x,
  xs !! i = Some x ->
  R g xs a -*
  WP App (Rec None (Some "args") get_body)
       (Val (StructV [a; LitV (LitInt (Z.of_nat i))]))
    {{ v, bi_pure (v = LitV (LitInt x)) * R g xs a }}.

Hypothesis set_spec : forall g xs (a : val) (i : nat) y x,
  xs !! i = Some y ->
  R g xs a -*
  WP App (Rec None (Some "args") set_body)
       (Val (StructV [a; LitV (LitInt (Z.of_nat i)); LitV (LitInt x)]))
    {{ v, bi_pure (v = LitV LitUnit) * R g (<[i := x]> xs) a }}.
```

In mathematics, with the call written as the operation it prints to:

Definition 22 (Library interface)**.**

$$R(g, xs, a) \vdash \mathsf{wp}\,(\mathsf{length}(a))\{\, v.\ulcorner v = |xs|\urcorner * R(g, xs, a)\,\},$$
$$xs_i = x \land R(g, xs, a) \vdash \mathsf{wp}\,(\mathsf{get}(a, i))\{\, v.\ulcorner v = x\urcorner * R(g, xs, a)\,\},$$
$$xs_i = y \land R(g, xs, a) \vdash \mathsf{wp}\,(\mathsf{set}(a, i, x))\{\, v.\ulcorner v = ()\urcorner * R(g,\ xs[i \mapsto x],\ a)\,\}.$$

The value that represents the container is a literal object. Substituting into it does nothing, which is the predicate `rigid`. An instance opens the representation far enough to see $a = \mathsf{lit}(o)$ and then rebuilds it. A literal satisfies `rigid` by computation, which is `lit_rigid`.

```
Definition rigid (v : val) : Prop :=
  (forall x w, subst_val x w v = v) /\
  (forall x (l : loc), subst_var_val x l v = v).
```

13.5 The program

Application in the core language is unary, so a call of two or three arguments builds a struct, binds it, and applies the library closure to that struct. The printer erases the struct and prints `get(a, i)` or `a.get(i)`, according to which body was supplied. `call2` binds the result. `call3` is the call itself. `call3_seq` sequences the call before a continuation.

```
Definition call2 (body : expr) (a b : expr) (dst : string) (k : expr) : expr :=
  Let (Some "pack") (Struct [] [a; b])
    (Let (Some dst) (App (Rec None (Some "args") body) (Var "pack")) k).

Definition call3 (body : expr) (a b c : expr) : expr :=
  Let (Some "pack") (Struct [] [a; b; c])
    (App (Rec None (Some "args") body) (Var "pack")).

Definition call3_seq (body : expr) (a b c k : expr) : expr :=
  Let (Some "pack") (Struct [] [a; b; c])
    (Let None (App (Rec None (Some "args") body) (Var "pack")) k).
```

Swap reads index i into `t`, reads index j into `u`, writes `u` at i, and writes `t` at j.

```
Definition swap_body : expr :=
  Let (Some "a") (StructLoad (Var "args") 0)
  (Let (Some "i") (StructLoad (Var "args") 1)
  (Let (Some "j") (StructLoad (Var "args") 2)
    (call2 get_body (Var "a") (Var "i") "t"
      (call2 get_body (Var "a") (Var "j") "u"
        (call3_seq set_body (Var "a") (Var "i") (Var "u")
          (call3 set_body (Var "a") (Var "j") (Var "t")))))))).
```

Lemma 23 (Swap)**.** *If a is rigid, $xs_i = x_i$, and $xs_j = x_j$, then*

$$R(g, xs, a) \vdash \mathsf{wp}\,(\mathsf{swap}(a, i, j))\{\, v.\ulcorner v = ()\urcorner * R(g,\ \mathsf{swap}(xs, i, j),\ a)\,\}.$$

```
Lemma swap_spec g xs (a : val) (i j : nat) xi xj :
  rigid a ->
  xs !! i = Some xi -> xs !! j = Some xj ->
```

```
  R g xs a -*
  WP call_swap a i j
    {{ v, bi_pure (v = LitV LitUnit) * R g (swap_list xs i j) a }}.
```

The scan is the `while` of partition. Its condition is $(j+1) \leq (hi-1)$, that is $j < hi-1$. The body reads index j. When the value is $\leq$ the pivot it swaps indexes i and j and advances i. It then advances j. The two cursors are mutable `var` bindings, allocated by `VarBind` from the initial value lo.

```
Definition scan (a hi pivot : expr) : expr :=
  While (BinOp LeOp (plus1 (Var "j")) (minus1 hi))
    (call2 get_body a (Var "j") "v"
      (Let None
        (If (BinOp LeOp (Var "v") pivot)
          (Let None (call3 swap_body a (Var "i") (Var "j"))
            (Assign "i" (plus1 (Var "i"))))
          unitv)
        (Assign "j" (plus1 (Var "j"))))).
```

After the two `var` bindings are opened, i and j reside in stack cells l_i and l_j. The loop specification is an induction on the number of steps remaining, $(hi-1)-j$. The postcondition is the Gallina scan: the list is $\mathsf{lomuto}(n, xs, pivot, i, j, hi-1)$, the cell l_i holds the returned boundary, and the cell l_j holds $hi-1$. The invariant maintained at each iteration is $\mathsf{lomutoInv}$ of the section above, instantiated at the current cursors. The base case is the failing loop test, $j = hi-1$.

```
Lemma wp_scan n g xs (a : val) (hi : nat) (pivot : Z) (i j : nat) (li lj : loc) :
  rigid a ->
  (0 < hi)%nat ->
  ((hi - 1) - j = n)%nat ->
  (j <= hi - 1)%nat ->
  (i <= j)%nat ->
  (hi - 1 < length xs)%nat ->
  stack_mapsto (StackId li) (nval i) -*
  stack_mapsto (StackId lj) (nval j) -*
  R g xs a -*
  WP scan_open a (Z.of_nat hi) pivot li lj
  {{ _, exists ys p,
      bi_pure (lomuto n xs pivot i j (hi - 1) = (ys, p)) *
      stack_mapsto (StackId li) (nval p) *
      stack_mapsto (StackId lj) (nval (hi - 1)) * R g ys a }}.
```

Partition reads the pivot at $hi-1$, allocates i and j at lo, runs the scan, swaps the pivot into the returned boundary, and returns that boundary.

```
Definition partition_body : expr :=
  Let (Some "a") (StructLoad (Var "args") 0)
  (Let (Some "lo") (StructLoad (Var "args") 1)
  (Let (Some "hi") (StructLoad (Var "args") 2)
    (call2 get_body (Var "a") (minus1 (Var "hi")) "pivot"
      (VarBind "i" (Var "lo")
        (VarBind "j" (Var "lo")
          (Let None (scan (Var "a") (Var "hi") (Var "pivot"))
            (Let None (call3 swap_body (Var "a") (Var "i") (minus1 (Var "hi")))
```

```
              (Var "i")))))))).
```

Lemma 24 (Partition)**.** *If a is rigid, $lo < hi \le |xs|$, and $xs_{hi-1} = pivot$, then*

$$R(g, xs, a) \vdash \mathsf{wp}\,(\mathsf{partition}(a, lo, hi))$$
$$\{\, v.\ \exists\, ys, p.\ \ulcorner \mathsf{partition}(xs, lo, hi) = (ys, p)\urcorner * \ulcorner v = p\urcorner * R(g, ys, a)\,\}.$$

```
Lemma partition_spec g xs (a : val) (lo hi : nat) (pivot : Z) :
  rigid a ->
  (lo < hi)%nat ->
  (hi <= length xs)%nat ->
  xs !! (hi - 1)%nat = Some pivot ->
  R g xs a -*
  WP call_partition a lo hi
  {{ v, exists ys p,
      bi_pure (partition_list xs lo hi = (ys, p)) *
      bi_pure (v = nval p) * R g ys a }}.
```

The recursive procedure is named `qsort`. Its test is $(lo+1) \le (hi-1)$, that is $hi \ge lo+2$. The failing branch is unit, so the printed program has no `else`. The succeeding branch partitions and calls itself on $[lo, p)$ and on $[p+1, hi)$.

```
Definition qsort_body : expr :=
  Let (Some "a") (StructLoad (Var "args") 0)
  (Let (Some "lo") (StructLoad (Var "args") 1)
  (Let (Some "hi") (StructLoad (Var "args") 2)
    (If (BinOp LeOp (plus1 (Var "lo")) (minus1 (Var "hi")))
      (Let (Some "p")
        (App (Rec None (Some "args") partition_body)
          (Struct [] [Var "a"; Var "lo"; Var "hi"]))
        (Let None
          (App (Var "qsort") (Struct [] [Var "a"; Var "lo"; Var "p"]))
          (App (Var "qsort")
            (Struct [] [Var "a"; plus1 (Var "p"); Var "hi"]))))
      unitv))).
```

The specification follows the fuel of `qsort_slice`. With fuel at least $hi - lo$, the call replaces the represented list by the sorted slice and leaves the ghost g unchanged. Outside the slice the functional specification already says the elements are untouched, so the representation of the whole list is the specification of the slice.

```
Lemma wp_qslice n g xs (a : val) (lo hi : nat) :
  rigid a ->
  (hi <= length xs)%nat ->
  (hi - lo <= n)%nat ->
  R g xs a -*
  WP call_qsort a lo hi {{ _, R g (qsort_slice n xs lo hi) a }}.
```

`sort` reads the length and calls `qsort` on $[0, n)$.

```
Definition sort_body : expr :=
  Let (Some "n") (App (Rec None (Some "a") len_body) (Var "a"))
```

```
    (Let (Some "pack") (Struct [] [Var "a"; zero; Var "n"])
      (App (Rec (Some "qsort") (Some "args") qsort_body) (Var "pack"))).
```

Theorem 25 (Sort)**.** *If a is rigid, then*

$$R(g, xs, a) \vdash \mathsf{wp}\,(\mathsf{sort}(a))\{\,_.\ R(g,\ \mathsf{qsort}(xs),\ a)\,\}.$$

```
Lemma sort_spec g xs (a : val) :
  rigid a ->
  R g xs a -*
  WP call_sort a {{ _, R g (qsort xs) a }}.
```

The proof is the slice specification at fuel $|xs|$, on the slice $[0, |xs|)$. The length read produces that upper bound. A permutation preserves the length, so an array list's size, which is the length of xs, is the same after sorting.

13.6 The two instances

The array instance takes G to be unit and $R(_, ys, b)$ to be $\mathsf{isArray}(ys, b)$, the predicate of Section 11. The three bodies are `length_body`, `get_body`, and `set_body`. Their closedness hypotheses are computed from those concrete terms. Their specifications are the library lemmas `length_spec`, `get_spec`, and `set_spec`, each applied in range. The resulting statement mentions only the array predicate and `qsort`.

```
Lemma array_qsort_spec xs (a : val) :
  is_array xs a -*
  WP call_sort length_body get_body set_body a
    {{ _, is_array (qsort xs) a }}.
```

The array-list instance takes G to be $\mathbb{N} \times \mathbb{Z}$ and $R((c, v), ys, b)$ to be $\mathsf{isArrayList}(ys, c, v, b)$. The three bodies are `size_body`, `al_get_body`, and `al_set_body`. The write specification of the library already returns the same capacity and the same version, and the interface requires exactly that. Size is $|xs|$ before the call and $|\mathsf{qsort}(xs)|$ after it, and those lengths are equal.

```
Lemma arraylist_qsort_spec xs cap ver (a : val) :
  is_arraylist xs cap ver a -*
  WP call_sort size_body al_get_body al_set_body a
    {{ _, is_arraylist (qsort xs) cap ver a }}.
```

13.7 The printed programs

`get` is a free function on `Array` and a method on `ArrayList`, and each generated file is compiled by itself, so the same three procedures are printed twice. The array file carries `make`, `length`, `get`, `set`, `swap`, `partition`, `qsort`, and `sort`. The array-list file carries `initArrayList`, `size`, `capacity`, `version`, `get`, `set`, `grow`, `add`, and the same three procedures. A client fills the list with `add`, and `add` calls `grow`. `sort` itself calls the length, the read, and the write. On the array those calls are `length(a)`, `get(a, i)`, and `set(a, i, x)`. On the array list they are `a.size()`, `a.get(i)`, and `a.set(i, x)`.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

class Array {
    public var len: Int64
    public var data: CPointer<Int64>
    public init(len: Int64, data: CPointer<Int64>) {
        this.len = len
        this.data = data
    }
}

func make(n: Int64): Array {
    let p = unsafe { CPointer<Int64>(malloc(UIntNative(8 * n))) }
    Array(n, p)
}

func length(a: Array): Int64 {
    a.len
}

func get(arr: Array, i: Int64): Int64 {
    let p = arr.data
    unsafe { (p + i).read() }
}

func set(arr: Array, i: Int64, x: Int64): Unit {
    let p = arr.data
    unsafe { (p + i).write(x) }
}

func swap(a: Array, i: Int64, j: Int64): Unit {
    let t = get(a, i)
    let u = get(a, j)
    set(a, i, u)
    set(a, j, t)
}

func partition(a: Array, lo: Int64, hi: Int64): Int64 {
    let pivot = get(a, hi - 1)
    var i = lo
    var j = lo
    while ((j + 1) <= (hi - 1)) {
        let v = get(a, j)
        if (v <= pivot) {
            swap(a, i, j)
            i = i + 1
        }
        j = j + 1
    }
    swap(a, i, hi - 1)
```

```
    i
}

func qsort(a: Array, lo: Int64, hi: Int64): Unit {
    if ((lo + 1) <= (hi - 1)) {
        let p = partition(a, lo, hi)
        qsort(a, lo, p)
        qsort(a, p + 1, hi)
    }
}

func sort(a: Array): Unit {
    let n = length(a)
    qsort(a, 0, n)
}

main(): Unit {
    let a = make(3)
    set(a, 0, 3)
    set(a, 1, 1)
    set(a, 2, 2)
    sort(a)
}
```

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

class Array {
    public var len: Int64
    public var data: CPointer<Int64>
    public init(len: Int64, data: CPointer<Int64>) {
        this.len = len
        this.data = data
    }
}

class ArrayList {
    public var myData: Array
    public var mySize: Int64
    public var myVersion: Int64
    public init(myData: Array, mySize: Int64, myVersion: Int64) {
        this.myData = myData
        this.mySize = mySize
        this.myVersion = myVersion
    }

    public func size(): Int64 {
        this.mySize
    }

    public func capacity(): Int64 {
```

```
        let buf = this.myData
        buf.len
    }

    public func version(): Int64 {
        this.myVersion
    }

    public func get(i: Int64): Int64 {
        let buf = this.myData
        let p = buf.data
        unsafe { (p + i).read() }
    }

    public func set(i: Int64, x: Int64): Unit {
        let buf = this.myData
        let p = buf.data
        unsafe { (p + i).write(x) }
    }

    public func grow(min: Int64): Unit {
        let buf = this.myData
        let pcap = buf.len
        let doubled = pcap + pcap
        let newCap = if (min <= doubled) { doubled } else { min }
        let src = buf.data
        let len = this.mySize
        let soff = 0
        let doff = 0
        let dst = unsafe { CPointer<Int64>(malloc(UIntNative(8 * newCap))) }
        let nb = Array(newCap, dst)
        var i = 0
        while ((i + 1) <= len) {
            unsafe { (dst + (i + doff)).write(unsafe { (src + (i + soff)).read() }) }
            i = i + 1
        }
        this.myData = nb
    }

    public func add(x: Int64): Unit {
        let n = this.mySize
        let buf = this.myData
        let pcap = buf.len
        let min = n + 1
        if (pcap <= n) {
            this.grow(min)
        }
        let buf2 = this.myData
        let p = buf2.data
        unsafe { (p + n).write(x) }
        let n1 = n + 1
        this.mySize = n1
        let ver = this.myVersion
        this.myVersion = ver + 1
```

```
    }
}

func initArrayList(): ArrayList {
    let p = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 16))) }
    let buf = Array(16, p)
    ArrayList(buf, 0, 0)
}

func swap(a: ArrayList, i: Int64, j: Int64): Unit {
    let t = a.get(i)
    let u = a.get(j)
    a.set(i, u)
    a.set(j, t)
}

func partition(a: ArrayList, lo: Int64, hi: Int64): Int64 {
    let pivot = a.get(hi - 1)
    var i = lo
    var j = lo
    while ((j + 1) <= (hi - 1)) {
        let v = a.get(j)
        if (v <= pivot) {
            swap(a, i, j)
            i = i + 1
        }
        j = j + 1
    }
    swap(a, i, hi - 1)
    i
}

func qsort(a: ArrayList, lo: Int64, hi: Int64): Unit {
    if ((lo + 1) <= (hi - 1)) {
        let p = partition(a, lo, hi)
        qsort(a, lo, p)
        qsort(a, p + 1, hi)
    }
}

func sort(a: ArrayList): Unit {
    let n = a.size()
    qsort(a, 0, n)
}

main(): Unit {
    let a = initArrayList()
    a.add(3)
    a.add(1)
    a.add(2)
    sort(a)
}
```

14 Hash maps

`theories/examples/hash_map.v` implements a monomorphic map from integer keys to integer values. The example uses open addressing: each key belongs in one of three parallel arrays, and collisions are resolved by scanning subsequent slots. The verified executable includes lookup, overwrite, insertion, removal, and rehashing when capacity doubles. Its model is a finite map $m : \mathbb{Z} \rightharpoonup \mathbb{Z}$, represented in Coq by `gmap Z Z`.

14.1 Storage and the representation invariant

A `HashMap` object has four fields: `mySize`, `tags`, `keys`, and `vals`. All three arrays have the same capacity c. At index i, tag 0 means empty, tag 1 means occupied, and tag 2 means deleted, or a *tombstone*. Only an occupied slot contributes its key and value to the mathematical map. Unoccupied keys and values are ignored; removal does not need to clear them.

The pure function `contents_of tags keys vals` walks the arrays and inserts occupied pairs into a finite map. By itself this function would conceal duplicate keys: a later map insertion can overwrite an earlier one. The invariant therefore proves separately that `NoDup (occ_keys tags keys)`. Under that condition, `size_contents` proves that the finite map's size is exactly `count_occ tags`, the number of live slots.

The record `wf_table c tags keys vals` contains eight obligations.

1. The tag array has length c.
2. The key array has length c.
3. The value array has length c.
4. $c = 2^{e+1}$ for some natural number e, so c is positive and even.
5. Every tag is one of 0, 1, and 2.
6. Occupied keys have no duplicates.
7. $2\,$`count_occ tags` $\leq c$; at most half the slots are occupied.
8. Every occupied key is *placed*: the probe path from its bucket to its slot contains no empty slot before it.

The last condition is essential. A search may stop at an empty slot only because no live key that belongs farther along that path can have crossed it. A tombstone is allowed on the path, so deleting an earlier key does not make a later colliding key unreachable.

The spatial predicate `is_hashmap m c a` adds ownership to these pure facts. It existentially names the map object, the three array objects, and their lists of contents. It owns all four map fields and all three array predicates, requires `contents_of = m`, and requires `mySize = Z.of_nat (count_occ tags)`. Thus a proof must maintain both the finite-map meaning and the physical size field. The separating conjunction between the three array predicates also provides the disjoint ownership needed to write one array while framing the others.

14.2 Remainders and wrapped indexes

The starting bucket is

$$b(k, c) = \mathrm{Z.\,to_nat}(\texttt{residue}\ \ k\ c).$$

The executable remainder calculation first adds c until a negative key becomes nonnegative, then subtracts c until the result is below c. `residue_spec` proves both $0 \le$ `residue` $k\ c < c$ and equality with the mathematical remainder $k \bmod c$. Each adjustment preserves congruence modulo c; the two loop lemmas prove that sufficiently many adjustments establish the range. The weakest-precondition proofs `wp_nonneg` and `wp_below` relate those pure recurrences to a mutable stack slot. `residue_run_wp` returns ownership of that slot as well as its result, allowing `residue_wp` to close the local variable's scope.

A probe at step s visits $(b + s) \bmod c$. The bounded scan maintains $b < c$ and $s < c$, hence $b + s < 2c$. At most one subtraction is required:

$$\texttt{probe_slot}(b, s, c) = \begin{cases} b + s, & b + s < c, \\ b + s - c, & c \le b + s. \end{cases}$$

`probe_slot_sub` proves this identity and `probe_slot_e_spec` verifies the executable branch. In the language's available operators, the strict test is written as $b + s + 1 \le c$. For example, capacity 16, bucket 15, and steps $0, 1, 2$ visit slots $15, 0, 1$. The comparison must choose the unmodified sum precisely in the first case; choosing it after crossing the capacity would produce an out-of-bounds read.

14.3 The bounded probe and its proof

The pure function `probe_from` carries a remaining fuel n, a starting bucket b, a current step s, and an optional first deleted slot `hole`. The executable `scan_body` has the same parameters. It uses -1 for no remembered hole, and a nonnegative index for an existing hole. `hole_or_e_spec` proves that the executable sentinel test agrees with the pure function `hole_or`.

An empty tag ends the search with an absent result. An occupied tag causes a key comparison: a match returns the current index and value; a different key continues with one less unit of fuel and one greater step. A tombstone also continues, remembering its index only if no earlier tombstone was remembered. This distinction prevents a subtle duplicate-key bug. After seeing a tombstone, insertion must keep looking for an existing equal key; it cannot immediately reuse the deleted slot. Otherwise an update could create a second occupied copy of a key that already lies farther along the path.

The prefix invariant `probe_prefix` records what the search has already established: the step is within the cycle, earlier visited slots contain neither an empty tag nor an occupied copy of the requested key, and the remembered hole agrees with the first tombstone in that prefix. `prefix_zero` establishes it before any slot is read. `probe_from_inv` advances it through each tag case and proves that a returned index is below capacity. For a found result it additionally identifies the occupied tag, matching key, and stored value. For an absent result it identifies an empty or tombstone slot and a path to that slot with no earlier empty tag.

Why can a complete cycle return a valid hole? The placement invariant alone does not establish this. The load bound implies `count_occ tags` $< c$, so a complete cycle cannot consist entirely of occupied slots. If the scan has not stopped at an empty slot, it must have visited a tombstone. `full_scan_hole` proves this implication. This also covers a table whose slots are all tombstones after successive insertions and removals: the scan consumes exactly c steps and returns the first remembered hole. It does not loop forever or attempt a step beyond the cycle.

`probe_shape` connects the slot result to `contents_of`. For a found result, `contents_at` and uniqueness show that the finite map returns the same value. For an absent result, the prefix and placement conditions exclude every possible occupied location for the requested key. `probe_found` proves the complementary property: any occupied slot for an existing key is reached by the probe.

The operational theorem `scan_spec` is an induction on remaining fuel. Its preconditions include $s + n \leq c$ and equal array lengths. It returns the exact value encoding of `probe_from`, together with ownership of all three unchanged arrays. The zero-fuel branch verifies the remembered-hole encoding. In a successor branch, `probe_slot_e_spec` yields a bounded index, and `arr_get_spec` reads the tag. The proof follows the same tag and key cases as the pure function. Recursive cases reduce fuel and increase step, preserving $s + n \leq c$; the induction hypothesis then applies. The found branch reads the value at the same bounded index. Every read returns its array ownership, so the caller can reconstruct the complete map predicate.

`buffer_probe_spec` first verifies the remainder call, rewrites its result to the bucket index, and instantiates `scan_spec` with $n = c$, $s = 0$, and no hole. `probe_spec` opens the map object, reads its array fields and capacity, applies that result, then closes the predicate. Its postcondition states not merely the shape of a returned struct but the semantic equation

$$m[k] = \begin{cases} \texttt{Some}\ v, & \texttt{found} = \texttt{true}, \\ \texttt{None}, & \texttt{found} = \texttt{false}. \end{cases}$$

An absent result additionally carries the canonical value zero.

14.4 Insertion and overwrite

`put_body` is the shared operation on three buffers. It probes first. When the key is found, it writes only the value at the returned index. When absent, it writes an occupied tag, the requested key, and the requested value. `put_spec` proves that its final arrays are exactly the triple computed by `write_fresh`; it deliberately separates this physical claim from the stronger invariant claim.

The array helper proofs expose the array's data pointer and use the bounded index to focus one cell with `big_sepL_lookup_acc` or `big_sepL_insert_acc`. A raw load or store acts on that cell, and the closing continuation restores ownership of the complete array. The store proof also establishes that an in-range list insertion preserves array length. These are the memory steps beneath both ordinary insertion and rehashing.

For an existing key, `write_fresh_found` proves that overwriting the value preserves all eight table conditions and leaves the occupied count unchanged. No new key is introduced and no tag or probe path changes. For a missing key, `write_fresh_new` assumes strict room, $2\,\texttt{count_occ tags} < c$. It proves that the chosen slot is available, that writing it introduces no duplicate, that its placement is valid, that old placements remain valid, and that the occupied count increases by one. Because c is even, strict room implies $2(\texttt{count_occ tags} + 1) \leq c$, preserving the half-full bound. `write_fresh_effect` packages these two cases as one map update and a conditional occupied-count equation.

`add_room_spec` opens the object, applies `put_spec`, and uses `write_fresh_effect` to close the invariant. It increments `mySize` exactly when the initial probe said the key was absent. The public `add` checks presence before considering growth. An existing key is updated directly, including at the load limit. For a missing key, it reads size and capacity and grows when $c \leq 2\,\texttt{size}$. Otherwise strict room already holds. After doubling, the old load bound $2\,\texttt{size} \leq c$ and $c > 0$ imply strict room in the new table.

The public contract therefore permits the resulting capacity to change:

```
Lemma add_spec m cap (a : val_cjr) (k v : Z) :
  is_hashmap m cap a -∗
  WP call_add a k v
    {{ u, ⌜ u = LitV LitUnit ⌝ ∗
          ∃ cap', is_hashmap (<[k:=v]> m) cap' a }}.
```

A fixed-capacity contract would be incorrect for an insertion that crosses the load threshold. In this implementation each call either keeps capacity or doubles it once.

14.5 Rehashing: why allocation alone is insufficient

Doubling changes bucket locations. Existing entries must be probed and inserted again under the new capacity; copying cells to the same indexes does not in general preserve placement. `grow_table` describes the pure operation: allocate three zero lists of length $2c$, then fold over the old arrays with `rehash_acc`. Empty and tombstone slots are skipped; each occupied pair is inserted into the destination with `write_fresh`.

The induction in `rehash_acc_lookup` maintains a well-formed destination, valid and unique remaining source keys, and two additional facts. First, every remaining occupied source key is absent from the destination. Second,

$$2(\texttt{count_occ destination} + \texttt{count_occ remaining}) \leq 2c.$$

When the source head is occupied, disjointness proves the insertion is new and the combined load bound proves strict room. `write_fresh_new` produces a well-formed destination whose count is one greater. Removing the source head decreases the remaining count by one, so the combined bound is preserved. Source uniqueness ensures the newly inserted key is absent from the remaining source, preserving disjointness. A skipped source head changes neither the destination nor the occupied count. At the empty source, the destination already satisfies the desired invariant.

The lookup conclusion states that the final map is the source map combined with the initial accumulator. For fresh zero arrays the accumulator is empty. `grow_table_spec` obtains equality with the original map by pointwise finite-map equality. It also uses $2c = 2^{(e+1)+1}$ to preserve the capacity condition. This proof explains both why no old entry disappears and why deleted entries are not resurrected.

The executable `rehash_body` is a recursive scan over the old arrays. Its arguments include a source index and remaining fuel, the three source arrays, and the new capacity and destination arrays. It calls `put_body` only for an occupied source tag. `rehash_spec` relates the index to suffixes of the source lists using `drop`. The small lemma `head_drop_hm` identifies the cell at the current index and the suffix at the next index. The proof follows the pure fold's induction, preserving all old array ownership and returning exactly its destination triple.

Finally, `grow_spec` allocates the new arrays, establishes `zeros_wf`, invokes `rehash_spec`, and only then replaces the object's three array fields. The old size field is retained. `size_contents` applied to both well-formed tables and their equal maps proves that this unchanged size is the correct occupied count of the new table. The capacity becomes $2c$ and the contents remain m. The example's `grow(minCap)` interface performs one doubling; `minCap` is a retained interface argument, not a promise to grow repeatedly to an arbitrary requested capacity.

14.6 Removal, queries, and client proofs

`write_remove_spec` proves that marking a found slot as a tombstone produces `delete k m`. Uniqueness identifies the one occurrence being removed. `placed_bury` preserves the paths of other keys:

a tombstone, unlike an empty tag, does not terminate a probe. `count_bury` proves that the old occupied count is the successor of the new one. The executable `remove_spec` uses the precise probe result to write that tag, subtract one from `mySize`, return the old value, and reconstruct ownership. For a missing key it returns zero, performs no writes, and proves that deleting an absent key leaves the mathematical map unchanged.

`get_spec` returns `Int64Option(true,v)` when $m[k] =$ `Some` v and `Int64Option(false,0)` when the key is absent. `contains_spec` returns the boolean indicating whether that lookup is `Some`. Both proofs frame the unchanged map through `probe_spec`, then verify the appropriate struct projections. The flag distinguishes a stored zero from absence. The older `remove` interface returns an integer alone, so removing a stored zero and removing a missing key both return zero; the map and its size still change differently.

`init_spec` verifies three zero-array allocations and the four-field object construction. Zero contents imply an empty map, valid tags, no duplicate keys, zero load, and vacuous placement. `size_spec`, `capacity_spec`, and `is_empty_spec` read the owned fields and use the representation equations to obtain their abstract meanings. The three `init_then_*` lemmas compose initialization with those queries.

The client theorems `add_then_get_spec` and `add_then_contains_spec` are short because the representation has already done the hard work. The first call establishes `is_hashmap (<[k:=v]> m) cap' a`. The second uses the query contract and the finite-map law `lookup_insert`: the updated map returns `Some v` at k. The resulting capacity is existentially retained, so the composition works both with and without growth. All program proofs are checked Coq proofs; the assumption audit includes the public operations and these client properties.

14.7 Executable validation and scope

The generated file below comes from these same core expressions. Its `ProbeResult` and `Int64Option` declarations are value structs; the mutable buffers and map remain classes. Runtime tests cover fresh maps, updates, removal, collision chains that wrap around capacity, negative keys, stored zero, a tombstone before an existing key, update at the load threshold, one and several doublings, explicit growth, and a complete cycle of tombstones. These tests exercise printing and compilation in addition to the formal core.

The core's integer semantics uses unbounded `Z`; extracted Cangjie uses `Int64`. The Coq results do not establish freedom from finite-machine integer overflow for arbitrarily large keys, counts, or capacities. Remainder normalization uses repeated addition and subtraction, and a collision chain can scan the entire capacity; this is a small verified example rather than a performance specification for the full Cangjie standard library. Rehashing allocates new buffers without reclaiming the old ones, consistently with the example language's object-allocation model.

14.8 The checked proof scripts

These listings include the complete proofs of the public operations and client properties, together with the central probe, insertion, and rehash arguments. They are read directly from the checked source, rather than maintained as separate proof sketches. In the scripts, `hm_admin` performs pure administrative reductions while leaving function calls for their specifications; `hm_pure` also unfolds applications. `hm_read` and `hm_write` focus owned object fields, and `hm_take` focuses the right-hand side of a let. Their evaluation-context definitions are in `theories/examples/hm_tactics.v`. None supplies an assumption or changes the postcondition: each application constructs a proof from the language rules.

The mutable remainder computation. The intermediate theorem retains the local stack slot. This ownership is required to discharge the scope cleanup in the following wrapper.

```
Lemma residue_run_wp (lslot : Z) (key cap : Z) :
  (0 < cap)%Z →
  StackId lslot ↦s LitV (LitInt key) -∗
  WP (Seq (nonneg_while lslot cap) (Seq (below_while lslot cap) (sld lslot)))
    {{ v, ⌜ v = LitV (LitInt (residue key cap)) ⌝ ∗
         StackId lslot ↦s LitV (LitInt (residue key cap)) }}.
Proof.
  intros Hcap. set (r := residue key cap).
  iIntros "Hh".
  iApply (wp_bind [SeqCtx (Seq (below_while lslot cap) (sld lslot))]).
  iApply (wp_wand with "[Hh]").
  { iApply (wp_nonneg with "Hh"). exact Hcap. }
  iIntros (w) "(%h1 & %Hh1 & Hh)".
  assert (0 ≤ h1)%Z as Hnn1.
  { rewrite Hh1. apply (proj1 (bring_nonneg_spec key cap (Z.to_nat (Z.max 0 (- key)))
   Hcap ltac:(lia))). }
  iApply wp_seq.
  iApply (wp_bind [SeqCtx (sld lslot)]).
  iApply (wp_wand with "[Hh]").
  { iApply (wp_below lslot h1 cap Hcap Hnn1 with "Hh"). }
  iIntros (w') "(%h2 & %Hh2 & Hh)".
  iApply wp_seq.
  iApply (wp_stack_load (StackId lslot) with "Hh").
  iIntros "!> _".
  assert (h2 = r)%Z as Heqr.
  { rewrite /r. unfold residue. rewrite Hh1 in Hh2. exact Hh2. }
  iIntros "Hstack".
  rewrite Heqr. iFrame. done.
Qed.
```

The remainder wrapper. The wrapper allocates the mutable binding, uses the two verified loops through the intermediate theorem, and returns both the result equation and the stack ownership needed by the binding rule.

```
Lemma residue_wp (k cap : Z) :
  (0 < cap)%Z →
  ⊢ WP call_residue k cap {{ v, ⌜ v = LitV (LitInt (residue k cap)) ⌝ }}.
Proof.
  intros Hcap. unfold call_residue, residue_body. hm_pure.
  iApply wp_var_bind. iIntros (lr) "Hr".
  rewrite cap_residue_stack.
  iApply (wp_wand with "[Hr]").
  { iApply (residue_run_wp lr k cap Hcap with "Hr"). }
  iIntros (r) "[%Hr Hslot]".
  iExists (LitV (LitInt (residue k cap))). iFrame. done.
Qed.
```

Initialization. Each array allocation is verified before the object is constructed. The final pure obligations are the zero-table invariant and the equality between zero occupied count and the stored size.

```
  Lemma init_spec :
    ⊢ WP call_init {{ m, is_hashmap ∅ default_capacity m }}.
  Proof.
    unfold call_init, init_body. hm_pure.
    hm_take. iApply (wp_wand with "[]"). { iApply make_zero_spec. }
    iIntros (ta) "Ht".
    iDestruct (is_array_obj_hm with "Ht") as (ot) "(%Ht & Ht)". subst ta.
    hm_pure.
    hm_take. iApply (wp_wand with "[]"). { iApply make_zero_spec. }
    iIntros (ka) "Hk".
    iDestruct (is_array_obj_hm with "Hk") as (ok) "(%Hk & Hk)". subst ka.
    hm_pure.
    hm_take. iApply (wp_wand with "[]"). { iApply make_zero_spec. }
    iIntros (va) "Hv".
    iDestruct (is_array_obj_hm with "Hv") as (ov) "(%Hv & Hv)". subst va.
    hm_pure.
    iApply wp_new. iIntros (o) "Hfields".
    iEval (simpl) in "Hfields".
    iDestruct "Hfields" as "(Hsz & Hta & Hka & Hva & _)".
    iExists o, 0%Z, (LitV (LitObj ot)), (LitV (LitObj ok)), (LitV (LitObj ov)),
      (replicate default_capacity 0%Z), (replicate default_capacity 0%Z),
      (replicate default_capacity 0%Z).
    rewrite contents_zeros count_zeros. iFrame.
    iSplit; [done|]. iSplit; [done|]. iSplit; [done|].
    unfold is_hm_slots. iFrame. iPureIntro.
    split; [apply length_replicate|].
    split; [apply length_replicate|].
    split; [apply length_replicate|].
    apply zeros_wf. apply default_cap_pow.
  Qed.
```

The pure search result. This proof combines the prefix invariant with placement and uniqueness. The found branch identifies the finite-map entry; the absent branch rules out every occupied location for the key.

```
Lemma probe_shape cap tags keys vals k :
  (0 < cap)%nat →
  length tags = cap → length keys = cap → length vals = cap →
  tags_ok tags → NoDup (occ_keys tags keys) → placed cap tags keys →
  (count_occ tags < cap)%nat →
  match probe cap (bucket k cap) tags keys vals k with
  | PFound idx v => contents_of tags keys vals !! k = Some v
  | PAbsent idx =>
      (idx < cap)%nat ∧
      (tags !! idx = Some tag_empty ∨ tags !! idx = Some tag_tomb) ∧
      no_empty_before cap (bucket k cap) idx tags ∧
      contents_of tags keys vals !! k = None
  end.
```

```
Proof.
  intros Hcap Hltag Hlkey Hlval Hok Hdup Hplace Hcount.
  set (b := bucket k cap).
  assert (b < cap)%nat as Hb by (unfold b; apply bucket_lt; done).
  assert (probe_prefix cap b 0 tags keys k None) as Hpre.
  { apply prefix_zero. lia. }
  pose proof (probe_from_inv cap b 0 cap tags keys vals k None Hcap Hb
                ltac:(lia) Hltag Hlkey Hlval Hok Hcount Hpre) as Hinv.
  unfold probe.
  destruct (probe_from cap b 0 cap tags keys vals k None)
    as [idx v|idx] eqn:Hr.
  - destruct Hinv as (Hidx & Ht & Hk & Hv & _).
    apply (contents_at tags keys vals idx k v).
    { rewrite Hlkey. rewrite Hltag. reflexivity. }
    { rewrite Hlval. rewrite Hltag. reflexivity. }
    { exact Hdup. }
    { exact Ht. }
    { exact Hk. }
    { exact Hv. }
  - destruct Hinv as (Hidx & Htag & Hpath).
    split; [exact Hidx|]. split; [exact Htag|]. split; [exact Hpath|].
    assert (length keys = length tags) as Hlk.
    { rewrite Hlkey. rewrite Hltag. reflexivity. }
    assert (length vals = length tags) as Hlv.
    { rewrite Hlval. rewrite Hltag. reflexivity. }
    apply (contents_none_iff tags keys vals k Hlk Hlv).
    intros Hin.
    destruct (occ_index tags keys k Hin) as (i & Ht & Hk).
    assert (i < cap)%nat as Hi.
    { apply lookup_lt_Some in Ht. lia. }
    destruct (lookup_lt_is_Some_2 vals i) as [v Hv].
    { rewrite Hlval. exact Hi. }
    assert (probe_from cap b 0 cap tags keys vals k None = PFound i v)
      as Hr'.
    { unfold b. change (probe cap (bucket k cap) tags keys vals k = PFound i v).
      apply probe_found; try done. }
    rewrite Hr in Hr'. discriminate.
Qed.
```

The executable bounded scan. The induction follows the fuel parameter. Array reads preserve ownership, and each recursive call consumes one step while preserving the bound on the complete scan.

```
  Lemma scan_spec fuel :
    ∀ start step cap ta ka va tags keys vals k hole,
    (0 < cap)%nat → (start < cap)%nat → (step + fuel ≤ cap)%nat →
    length tags = cap → length keys = cap → length vals = cap →
    is_array tags ta -∗ is_array keys ka -∗ is_array vals va -∗
    WP call_scan fuel start step cap ta ka va k hole
      {{ r, ⌜ r = probe_val (probe_from fuel start step cap tags keys vals k hole) ⌝ ∗
            is_array tags ta ∗ is_array keys ka ∗ is_array vals va }}.
  Proof.
```

```
induction fuel as [|fuel IH]; intros start step cap ta ka va tags keys vals k hole
  Hcap Hstart Hbound Lt Lk Lv; iIntros "Ht Hk Hv";
  iDestruct (is_array_obj_hm with "Ht") as (ot) "(%Ht & Ht)";
  iDestruct (is_array_obj_hm with "Hk") as (ok) "(%Hk & Hk)";
  iDestruct (is_array_obj_hm with "Hv") as (ov) "(%Hv & Hv)";
  subst ta ka va;
  unfold call_scan, scan_v, scan_body, scan_loop, scan_call_e, probe_done;
  hm_pure.
- iApply wp_if_true. hm_pure. destruct hole as [h|]; simpl.
  + rewrite bool_decide_eq_false_2; [|lia]. iApply wp_if_false. hm_pure. iFrame.
done.
  + iApply wp_if_true. hm_pure. iFrame. done.
- iApply wp_if_false.
  hm_take. iApply (wp_wand with "[]").
  { iApply probe_slot_e_spec; lia. }
  iIntros (r) "%Hr". subst r. hm_admin.
  set (idx := probe_slot start step cap).
  assert (idx < cap)%nat as Hidx by (unfold idx; apply probe_slot_lt; done).
  destruct (lookup_lt_is_Some_2 tags idx) as [tag Htag]; [lia|].
  hm_take. iApply (wp_wand with "[Ht]").
  { iApply (arr_get_spec tags _ idx tag Htag with "Ht"). }
  iIntros (r) "[%Hr Ht]". subst r. hm_admin.
  change (tags !! probe_slot start step cap = Some tag) in Htag.
  cbn [probe_from]. rewrite Htag.
  destruct (decide (tag = tag_empty)) as [He|He].
  + rewrite bool_decide_eq_true_2; [|exact He]. iApply wp_if_true.
    iApply array.wp_struct_step. simpl.
    iApply (wp_bind [StructCtx [LitV (LitBool false)] [zlit 0]]).
    iApply (wp_wand with "[]"). { iApply hole_or_e_spec. }
    iIntros (r) "%Hr". subst r. hm_admin. iFrame. done.
  + rewrite bool_decide_eq_false_2; [|exact He]. iApply wp_if_false. hm_admin.
    destruct (decide (tag = tag_occ)) as [Ho|Ho].
    * rewrite bool_decide_eq_true_2; [|exact Ho]. iApply wp_if_true. hm_admin.
      destruct (lookup_lt_is_Some_2 keys idx) as [key Hkey]; [lia|].
      destruct (lookup_lt_is_Some_2 vals idx) as [value Hval]; [lia|].
      hm_take. iApply (wp_wand with "[Hk]").
      { iApply (arr_get_spec keys _ idx key Hkey with "Hk"). }
      iIntros (r) "[%Hr Hk]". subst r. hm_admin.
      change (keys !! probe_slot start step cap = Some key) in Hkey.
      change (vals !! probe_slot start step cap = Some value) in Hval.
      rewrite Hkey Hval.
      destruct (decide (key = k)) as [Hhit|Hmiss].
      -- rewrite bool_decide_eq_true_2; [|exact Hhit]. iApply wp_if_true. hm_admin.
         iApply (wp_bind [StructCtx
           [LitV (LitBool true); LitV (LitInt (Z.of_nat idx))] []]).
         iApply (wp_wand with "[Hv]").
         { iApply (arr_get_spec vals _ idx value Hval with "Hv"). }
         iIntros (r) "[%Hr Hv]". subst r. hm_admin. iFrame. done.
      -- rewrite bool_decide_eq_false_2; [|exact Hmiss]. iApply wp_if_false.
hm_admin.
         replace (Z.of_nat (S fuel) - 1)%Z with (Z.of_nat fuel) by lia.
         replace (Z.of_nat step + 1)%Z with (Z.of_nat (S step)) by lia.
         iApply (IH start (S step) cap _ _ _ tags keys vals k hole with "Ht Hk Hv");
 try done; lia.
```

```
      * rewrite bool_decide_eq_false_2; [|exact Ho]. iApply wp_if_false. hm_admin.
        destruct hole as [h|]; simpl.
        -- rewrite bool_decide_eq_false_2; [|lia]. iApply wp_if_false. hm_admin.
           replace (Z.of_nat (S fuel) - 1)%Z with (Z.of_nat fuel) by lia.
           replace (Z.of_nat step + 1)%Z with (Z.of_nat (S step)) by lia.
           iApply (IH start (S step) cap _ _ _ tags keys vals k (Some h) with "Ht Hk
  Hv"); try done; lia.
        -- iApply wp_if_true. hm_admin.
           replace (Z.of_nat (S fuel) - 1)%Z with (Z.of_nat fuel) by lia.
           replace (Z.of_nat step + 1)%Z with (Z.of_nat (S step)) by lia.
           iApply (IH start (S step) cap _ _ _ tags keys vals k (Some idx) with "Ht
  Hk Hv"); try done; lia.
Qed.
```

From a remainder to a buffer search. The range theorem permits converting the remainder to a natural bucket index. The scan is then instantiated with the full capacity as fuel.

```
Lemma buffer_probe_spec cap ta ka va tags keys vals k :
  (0 < cap)%nat → length tags = cap → length keys = cap → length vals = cap →
  is_array tags ta -∗ is_array keys ka -∗ is_array vals va -∗
  WP call_buffer_probe ta ka va cap k
    {{ r, ⌜ r = probe_val (probe cap (bucket k cap) tags keys vals k) ⌝ ∗
      is_array tags ta ∗ is_array keys ka ∗ is_array vals va }}.
Proof.
  intros Hcap Lt Lk Lv. iIntros "Ht Hk Hv".
  iDestruct (is_array_obj_hm with "Ht") as (ot) "(%Ht & Ht)".
  iDestruct (is_array_obj_hm with "Hk") as (ok) "(%Hk & Hk)".
  iDestruct (is_array_obj_hm with "Hv") as (ov) "(%Hv & Hv)". subst ta ka va.
  unfold call_buffer_probe, buffer_probe_body. iApply wp_app. simpl. hm_admin.
  try try hm_take. hm_admin. iApply (wp_wand with "[]").
  { iApply residue_wp. lia. }
  iIntros (r) "%Hr". subst r. hm_admin.
  replace (residue k (Z.of_nat cap)) with (Z.of_nat (bucket k cap)) by
    (unfold bucket; rewrite Z2Nat.id; pose proof (residue_spec k (Z.of_nat cap) ltac
  :(lia)); lia).
  iApply (scan_spec cap (bucket k cap) 0 cap _ _ _ tags keys vals k None with "Ht Hk
  Hv");
    try done; try (apply bucket_lt; done); lia.
Qed.
```

Physical insertion and overwrite. The proof first obtains the exact probe result. It writes one value cell for a hit and three corresponding cells for an absent key; its postcondition is the exact pure update triple.

```
Lemma put_spec cap ta ka va tags keys vals k v :
  wf_table cap tags keys vals →
  is_array tags ta -∗ is_array keys ka -∗ is_array vals va -∗
  WP call_put ta ka va cap k v
    {{ r, let '(t2,k2,v2) := write_fresh cap tags keys vals k v in
      ⌜ r = LitV LitUnit ⌝ ∗ is_array t2 ta ∗ is_array k2 ka ∗ is_array v2 va }}.
Proof.
```

```
    intros Hwf. iIntros "Ht Hk Hv".
    iDestruct (is_array_obj_hm with "Ht") as (ot) "(%Ht & Ht)".
    iDestruct (is_array_obj_hm with "Hk") as (ok) "(%Hk & Hk)".
    iDestruct (is_array_obj_hm with "Hv") as (ov) "(%Hv & Hv)". subst ta ka va.
    unfold call_put, put_body. iApply wp_app. simpl. hm_admin.
    try hm_take. hm_admin. iApply (wp_wand with "[Ht Hk Hv]").
    { iApply (buffer_probe_spec with "Ht Hk Hv");
        [eapply cap_pos; exact Hwf|apply (wf_tags_len _ _ _ _ Hwf)|
         apply (wf_keys_len _ _ _ _ Hwf)|apply (wf_vals_len _ _ _ _ Hwf)]. }
    iIntros (r) "(%Hr & Ht & Hk & Hv)". subst r.
    pose proof (probe_slots cap tags keys vals k Hwf) as Hslots.
    unfold write_fresh. destruct (probe cap (bucket k cap) tags keys vals k) as [idx
    old|idx]; simpl in *.
    - destruct Hslots as (Hi & Htag & Hkey & Hval). hm_admin. iApply wp_if_true.
    hm_admin.
      iApply (wp_wand with "[Hv]"). { iApply (arr_set_spec vals _ idx old v Hval with "
    Hv"). }
      iIntros (r) "[%Hr Hv]". iFrame. done.
    - destruct Hslots as [Hi Htag].
      destruct (lookup_lt_is_Some_2 tags idx) as [tag Htidx]; [rewrite (wf_tags_len _ _
     _ _ Hwf); done|].
      destruct (lookup_lt_is_Some_2 keys idx) as [key Hkidx]; [rewrite (wf_keys_len _ _
     _ _ Hwf); done|].
      destruct (lookup_lt_is_Some_2 vals idx) as [old Hvidx]; [rewrite (wf_vals_len _ _
     _ _ Hwf); done|].
      hm_admin. iApply wp_if_false. hm_admin.
      iApply (wp_bind [SeqCtx _]).
      iApply (wp_wand with "[Ht]"). { iApply (arr_set_spec tags _ idx tag tag_occ Htidx
     with "Ht"). }
      iIntros (r) "[%Hr Ht]". subst r. hm_admin.
      iApply (wp_bind [SeqCtx _]).
      iApply (wp_wand with "[Hk]"). { iApply (arr_set_spec keys _ idx key k Hkidx with "
    Hk"). }
      iIntros (r) "[%Hr Hk]". subst r. hm_admin.
      iApply (wp_wand with "[Hv]"). { iApply (arr_set_spec vals _ idx old v Hvidx with "
    Hv"). }
      iIntros (r) "[%Hr Hv]". unfold place_slot. iFrame. done.
  Qed.
```

Preserving the invariant for a new key. The pure proof checks the tag, key, value, uniqueness, load, and placement obligations. Strict room is used to preserve the half-full bound after increasing the live count.

```
Lemma write_fresh_new cap tags keys vals k v :
  wf_table cap tags keys vals →
  contents_of tags keys vals !! k = None →
  (2 * count_occ tags < cap)%nat →
  ∃ idx,
    write_fresh cap tags keys vals k v =
      (<[idx:=tag_occ]> tags, <[idx:=k]> keys, <[idx:=v]> vals) ∧
    (idx < cap)%nat ∧
    wf_table cap (<[idx:=tag_occ]> tags) (<[idx:=k]> keys) (<[idx:=v]> vals) ∧
```

```
    contents_of (<[idx:=tag_occ]> tags) (<[idx:=k]> keys) (<[idx:=v]> vals) =
      <[k:=v]> (contents_of tags keys vals) ∧
    count_occ (<[idx:=tag_occ]> tags) = S (count_occ tags).
Proof.
  intros Hwf Hnone Hroom.
  destruct Hwf as [Hltag Hlkey Hlval Hpow Hok Hdup Hload Hplace].
  assert (0 < cap)%nat as Hcap by (destruct Hpow as [e ->]; apply pow_positive).
  assert (count_occ tags < cap)%nat as Hcount by lia.
  pose proof (probe_shape cap tags keys vals k Hcap Hltag Hlkey Hlval Hok Hdup
                Hplace Hcount) as Hsh.
  unfold write_fresh.
  destruct (probe cap (bucket k cap) tags keys vals k) as [idx found|idx] eqn:Hr.
  - rewrite Hsh in Hnone. discriminate.
  - destruct Hsh as (Hidx & Htag & Hpath & _).
    assert (tags !! idx ≠ Some tag_occ) as Hnot.
    { destruct Htag as [-> | ->].
      - intros Heq. discriminate Heq.
      - intros Heq. discriminate Heq. }
    assert (length keys = length tags) as Hlk.
    { rewrite Hlkey. rewrite Hltag. reflexivity. }
    assert (length vals = length tags) as Hlv.
    { rewrite Hlval. rewrite Hltag. reflexivity. }
    assert (k ∉ occ_keys tags keys) as Hnin.
    { apply (contents_none_iff tags keys vals k Hlk Hlv). exact Hnone. }
    exists idx. split; [reflexivity|]. split; [exact Hidx|]. split; [|split].
    + apply (Wf cap (<[idx:=tag_occ]> tags) (<[idx:=k]> keys) (<[idx:=v]> vals)).
      * rewrite insert_length_eq; [done|lia].
      * rewrite insert_length_eq; [done|lia].
      * rewrite insert_length_eq; [done|lia].
      * exact Hpow.
      * apply tags_ok_insert; [done| |lia]. unfold tag_ok. right. by left.
      * apply nodup_place; [exact Hlk|lia|exact Hdup|exact Hnin|exact Hnot].
      * rewrite count_insert_fresh; [|exact Htag]. apply room_for_one; done.
      * apply placed_place; try done.
    + apply contents_place; [exact Hlk|exact Hlv|lia|exact Hnot|exact Hnin].
    + rewrite count_insert_fresh; [done|exact Htag].
Qed.
```

Preserving the invariant for overwrite. A present key determines an occupied slot. Updating its value changes neither the occupied key list nor any placement path.

```
Lemma write_fresh_found cap tags keys vals k v w :
  wf_table cap tags keys vals →
  contents_of tags keys vals !! k = Some w →
  ∃ idx,
    write_fresh cap tags keys vals k v =
      (tags, keys, <[idx:=v]> vals) ∧
    (idx < cap)%nat ∧
    wf_table cap tags keys (<[idx:=v]> vals) ∧
    contents_of tags keys (<[idx:=v]> vals) = <[k:=v]> (contents_of tags keys vals).
Proof.
  intros Hwf Hsome.
  destruct Hwf as [Hltag Hlkey Hlval Hpow Hok Hdup Hload Hplace].
```

```
  assert (0 < cap)%nat as Hcap by (destruct Hpow as [e ->]; apply pow_positive).
  assert (count_occ tags < cap)%nat as Hcount by lia.
  assert (length keys = length tags) as Hlk by (rewrite Hlkey; rewrite Hltag;
    reflexivity).
  assert (length vals = length tags) as Hlv by (rewrite Hlval; rewrite Hltag;
    reflexivity).
  assert (k ∈ occ_keys tags keys) as Hin.
  { destruct (contents_none_iff tags keys vals k Hlk Hlv) as [_ Hincl].
    destruct (decide (k ∈ occ_keys tags keys)); [done|].
    exfalso. rewrite (Hincl n) in Hsome. discriminate. }
  destruct (occ_index tags keys k Hin) as (i & Ht & Hk).
  assert (i < cap)%nat as Hi by (apply lookup_lt_Some in Ht; lia).
  destruct (lookup_lt_is_Some_2 vals i) as [old Hv].
  { rewrite Hlval. exact Hi. }
  assert (probe cap (bucket k cap) tags keys vals k = PFound i old) as Hprobe.
  { apply probe_found; try done. }
  unfold write_fresh. rewrite Hprobe. simpl.
  exists i. split; [reflexivity|]. split; [exact Hi|]. split.
  + apply (Wf cap tags keys (<[i:=v]> vals)).
    * done.
    * done.
    * rewrite insert_length_eq; [|rewrite Hlval; exact Hi]. exact Hlval.
    * exact Hpow.
    * exact Hok.
    * exact Hdup.
    * exact Hload.
    * exact Hplace.
  + apply contents_overwrite; [exact Hlk|exact Hlv|exact Hdup|exact Ht|exact Hk|].
    { rewrite Hlval; exact Hi. }
Qed.
```

The pure rehash induction. Disjointness and the combined occupied-count bound are strengthened for the recursive call. The lookup equation then establishes preservation of every source key and accumulator key.

```
Lemma rehash_acc_lookup cap tags keys vals accT accK accV :
  wf_table cap accT accK accV →
  length keys = length tags → length vals = length tags →
  tags_ok tags → NoDup (occ_keys tags keys) →
  (∀ x, x ∈ occ_keys tags keys → contents_of accT accK accV !! x = None) →
  (2 * (count_occ accT + count_occ tags) ≤ cap)%nat →
  match rehash_acc cap tags keys vals accT accK accV with
  | (t2, k2, v2) =>
      wf_table cap t2 k2 v2 ∧
      ∀ x, contents_of t2 k2 v2 !! x =
        match contents_of tags keys vals !! x with
        | Some w => Some w
        | None => contents_of accT accK accV !! x
        end
  end.
Proof.
  revert keys vals accT accK accV.
```

```
  induction tags as [|t tags IH]; intros keys vals accT accK accV
      Hwf Hlk Hlv Hok Hdup Hdis Hload.
  - destruct keys, vals; simpl in *; try discriminate.
    split; [exact Hwf|]. intros x. simpl. done.
  - destruct keys as [|k keys]; [simpl in Hlk; discriminate|].
    destruct vals as [|v vals]; [simpl in Hlv; discriminate|].
    simpl in Hlk, Hlv. injection Hlk as Hlk. injection Hlv as Hlv.
    apply Forall_cons in Hok as [Htok Hok].
    simpl rehash_acc. simpl contents_of. simpl occ_keys in Hdup, Hdis.
    simpl count_occ in Hload.
    destruct (decide (t = tag_occ)) as [->|Hnot].
    + simpl in Hdup, Hdis. apply NoDup_cons in Hdup as [Hnin Hdup].
      assert (contents_of accT accK accV !! k = None) as Hmiss.
      { apply Hdis. apply elem_of_cons. by left. }
      assert (2 * count_occ accT < cap)%nat as Hroom by lia.
      destruct (write_fresh_new cap accT accK accV k v Hwf Hmiss Hroom)
        as (idx & Heq & _ & Hwf2 & Hins & Hcount).
      rewrite Heq. simpl.
      assert (∀ x, x ∈ occ_keys tags keys →
                contents_of (<[idx:=tag_occ]> accT) (<[idx:=k]> accK)
                  (<[idx:=v]> accV) !! x = None) as Hdis'.
      { intros x Hin. rewrite Hins. rewrite lookup_insert_ne.
        - apply Hdis. apply elem_of_cons. by right.
        - intros ->. apply Hnin. exact Hin. }
      assert (2 * (count_occ (<[idx:=tag_occ]> accT) + count_occ tags) ≤ cap)%nat
        as Hload'.
      { rewrite Hcount. lia. }
      specialize (IH keys vals _ _ _ Hwf2 Hlk Hlv Hok Hdup Hdis' Hload').
      destruct (rehash_acc cap tags keys vals
                  (<[idx:=tag_occ]> accT) (<[idx:=k]> accK) (<[idx:=v]> accV))
        as [[t2 k2] v2].
      destruct IH as [Hwf' Hlook]. split; [exact Hwf'|].
      intros x. rewrite Hlook. rewrite Hins.
      destruct (decide (x = k)) as [->|Hx].
      * assert (contents_of tags keys vals !! k = None) as Htnone.
        { apply (contents_none_iff tags keys vals k Hlk Hlv). exact Hnin. }
        rewrite Htnone. rewrite lookup_insert.
        simpl. destruct (decide (tag_occ = tag_occ)) as [_|Hbad];
          [|exfalso; apply Hbad; reflexivity].
        rewrite lookup_insert. reflexivity.
      * rewrite lookup_insert_ne; [|symmetry; exact Hx].
        simpl.
        destruct (decide (tag_occ = tag_occ)) as [_|Hbad];
          [|exfalso; apply Hbad; reflexivity].
        rewrite lookup_insert_ne; [|symmetry; exact Hx].
        reflexivity.
    + assert (∀ x, x ∈ occ_keys tags keys → contents_of accT accK accV !! x = None)
        as Hdis' by (intros x Hin; apply Hdis; exact Hin).
      assert (2 * (count_occ accT + count_occ tags) ≤ cap)%nat as Hload' by lia.
      specialize (IH keys vals accT accK accV Hwf Hlk Hlv Hok Hdup Hdis' Hload').
      destruct (rehash_acc cap tags keys vals accT accK accV) as [[t2 k2] v2].
      destruct IH as [Hwf' Hlook]. split; [exact Hwf'|].
      intros x. rewrite Hlook. simpl. done.
Qed.
```

The executable rehash induction. The source index is related to remaining suffixes with drop. Source arrays are framed through insertion into the destination, and the postcondition tracks the exact pure fold.

```
Lemma rehash_spec tags :
  ∀ keys vals i st sk sv sta ska sva cap dt dk dv accT accK accV,
  drop i st = tags → drop i sk = keys → drop i sv = vals →
  length keys = length tags → length vals = length tags →
  tags_ok tags → NoDup (occ_keys tags keys) →
  wf_table cap accT accK accV →
  (∀ x, x ∈ occ_keys tags keys → contents_of accT accK accV !! x = None) →
  (2 * (count_occ accT + count_occ tags) ≤ cap)%nat →
  is_array st sta -∗ is_array sk ska -∗ is_array sv sva -∗
  is_array accT dt -∗ is_array accK dk -∗ is_array accV dv -∗
  WP call_rehash (length tags) i sta ska sva cap dt dk dv
    {{ r, let '(t2,k2,v2) := rehash_acc cap tags keys vals accT accK accV in
      ⌜r = LitV LitUnit⌝ ∗ is_array st sta ∗ is_array sk ska ∗ is_array sv sva ∗
      is_array t2 dt ∗ is_array k2 dk ∗ is_array v2 dv }}.
Proof.
  induction tags as [|tag tags IH]; intros keys vals i st sk sv sta ska sva cap dt dk
   dv accT accK accV
    Hdt Hdk Hdv Hlk Hlv Hok Hdup Hwf Hdis Hload;
    iIntros "Hst Hsk Hsv Ht Hk Hv";
    iDestruct (is_array_obj_hm with "Hst") as (ost) "(%Hsta & Hst)";
    iDestruct (is_array_obj_hm with "Hsk") as (osk) "(%Hska & Hsk)";
    iDestruct (is_array_obj_hm with "Hsv") as (osv) "(%Hsva & Hsv)";
    iDestruct (is_array_obj_hm with "Ht") as (ot) "(%Hdta & Ht)";
    iDestruct (is_array_obj_hm with "Hk") as (ok) "(%Hdka & Hk)";
    iDestruct (is_array_obj_hm with "Hv") as (ov) "(%Hdva & Hv)";
    subst sta ska sva dt dk dv;
    unfold call_rehash, rehash_v, rehash_body; iApply wp_app; simpl; hm_admin.
  - destruct keys, vals; simpl in *; try discriminate.
    iApply wp_if_true. hm_admin. iFrame. done.
  - destruct keys as [|key keys]; [simpl in Hlk; discriminate|].
    destruct vals as [|value vals]; [simpl in Hlv; discriminate|].
    simpl in Hlk, Hlv. injection Hlk as Hlk. injection Hlv as Hlv.
    apply Forall_cons in Hok as [Htok Hok].
    destruct (head_drop_hm _ _ _ _ Hdt) as [Htag Hdt'];
    destruct (head_drop_hm _ _ _ _ Hdk) as [Hkey Hdk'];
    destruct (head_drop_hm _ _ _ _ Hdv) as [Hvalue Hdv'].
    iApply wp_if_false. hm_admin. hm_take.
    iApply (wp_wand with "[Hst]"). { iApply (arr_get_spec st _ i tag Htag with "Hst").
   }
    iIntros (r) "[%Hr Hst]". subst r. hm_admin.
    simpl rehash_acc. simpl occ_keys in Hdup, Hdis. simpl count_occ in Hload.
    destruct (decide (tag = tag_occ)) as [->|Hnot].
    + rewrite bool_decide_eq_true_2; [|done]. iApply wp_if_true. hm_admin.
      hm_take. iApply (wp_wand with "[Hsk]").
      { iApply (arr_get_spec sk _ i key Hkey with "Hsk"). }
      iIntros (r) "[%Hr Hsk]". subst r. hm_admin.
      hm_take. iApply (wp_wand with "[Hsv]").
```

```
      { iApply (arr_get_spec sv _ i value Hvalue with "Hsv"). }
      iIntros (r) "[%Hr Hsv]". subst r. hm_admin.
      apply NoDup_cons in Hdup as [Hnin Hdup].
      assert (contents_of accT accK accV !! key = None) as Hmiss.
      { apply Hdis. apply elem_of_cons. by left. }
      destruct (write_fresh_new cap accT accK accV key value Hwf Hmiss ltac:(lia))
        as (idx & Hwrite & Hidx & Hwf2 & Hins & Hcount).
      iApply (wp_wand with "[Ht Hk Hv]"). { iApply (put_spec with "Ht Hk Hv"). exact
  Hwf. }
      iIntros (r) "Hslots". iEval (rewrite Hwrite) in "Hslots".
      iDestruct "Hslots" as "(%Hr & Ht & Hk & Hv)". subst r. hm_admin.
      replace (Z.of_nat (S (length tags)) - 1)%Z with (Z.of_nat (length tags)) by lia.

      replace (Z.of_nat i + 1)%Z with (Z.of_nat (S i)) by lia.
      rewrite Hwrite. simpl.
      iApply (IH keys vals (S i) st sk sv _ _ _ cap _ _ _ _ _ _
        Hdt' Hdk' Hdv' Hlk Hlv Hok Hdup Hwf2 with "Hst Hsk Hsv Ht Hk Hv").
      * intros x Hin. rewrite Hins. rewrite lookup_insert_ne.
        -- apply Hdis. apply elem_of_cons. by right.
        -- intros ->. apply Hnin. exact Hin.
      * rewrite Hcount. lia.
    + rewrite bool_decide_eq_false_2; [|done]. iApply wp_if_false. hm_admin.
      replace (Z.of_nat (S (length tags)) - 1)%Z with (Z.of_nat (length tags)) by lia.

      replace (Z.of_nat i + 1)%Z with (Z.of_nat (S i)) by lia.
      iApply (IH keys vals (S i) st sk sv _ _ _ cap _ _ _ accT accK accV
        Hdt' Hdk' Hdv' Hlk Hlv Hok Hdup Hwf Hdis with "Hst Hsk Hsv Ht Hk Hv"). lia.
Qed.
```

Updating the object and its stored size. The buffer update is combined with the pure occupied-count equation. The size field is incremented in the absent-key case and retained in the overwrite case.

```
Lemma add_room_spec m cap a k v found :
  found = bool_decide (is_Some (m !! k)) →
  (m !! k = None → (2 * size m < cap)%nat) →
  is_hashmap m cap a -∗
  WP call_add_room a k v found
    {{ r, ⌜ r = LitV LitUnit ⌝ ∗ is_hashmap (<[k:=v]> m) cap a }}.
Proof.
  intros Hfound Hroom. iIntros "Hm".
  iDestruct "Hm" as (o sz ta ka va tags keys vals)
    "(%Ha & %Hm & %Hsz & Hsz & Hta & Hka & Hva & Hslots)".
  iDestruct "Hslots" as "(Ht & Hk & Hv & %Lt & %Lk & %Lv & %Hwf)".
  iDestruct (is_array_obj_hm with "Ht") as (ot) "(%Ht & Ht)".
  iDestruct (is_array_obj_hm with "Hk") as (ok) "(%Hk & Hk)".
  iDestruct (is_array_obj_hm with "Hv") as (ov) "(%Hv & Hv)".
  subst a ta ka va sz m.
  assert (size (contents_of tags keys vals) = count_occ tags) as Hsize.
  { apply size_contents; [lia|lia|exact (wf_dup _ _ _ _ Hwf)]. }
  rewrite Hsize in Hroom.
  pose proof (write_fresh_effect cap tags keys vals k v Hwf Hroom) as Heffect.
```

```
  destruct (write_fresh cap tags keys vals k v) as [[t2 k2] v2] eqn:Hwrite.
  destruct Heffect as (Hwf2 & Hcontents & Hcount).
  rewrite <- Hfound in Hcount.
  unfold call_add_room, add_room_body. iApply wp_app. simpl. hm_admin.
  hm_read. hm_admin. hm_read. hm_admin. hm_read. hm_admin.
  hm_take. iApply (wp_wand with "[Ht]"). { iApply (array.length_spec with "Ht"). }
  iIntros (r) "[%Hr Ht]". subst r. rewrite Lt. hm_admin.
  iApply (wp_wand with "[Ht Hk Hv]"). { iApply (put_spec with "Ht Hk Hv"). exact Hwf.
   }
  iIntros (r) "Hslots". iEval (rewrite Hwrite) in "Hslots".
  iDestruct "Hslots" as "(%Hr & Ht & Hk & Hv)". subst r. hm_admin.
  destruct found.
  - iApply wp_if_true. hm_admin. iSplit; [done|].
    iExists o, (Z.of_nat (count_occ tags)), (LitV (LitObj ot)), (LitV (LitObj ok)),
      (LitV (LitObj ov)), t2, k2, v2. unfold is_hm_slots.
    iFrame. repeat iSplit; try done; iPureIntro; first [lia|apply Hwf2].
  - iApply wp_if_false. hm_admin. hm_read. hm_admin.
    hm_write.
    replace (Z.of_nat (count_occ tags) + 1)%Z with (Z.of_nat (count_occ t2)) by (
  rewrite Hcount; lia).
    iSplit; [done|]. iExists o, (Z.of_nat (count_occ t2)), (LitV (LitObj ot)),
      (LitV (LitObj ok)), (LitV (LitObj ov)), t2, k2, v2.
    unfold is_hm_slots. iFrame. repeat iSplit; try done; iPureIntro; first [lia|apply
   Hwf2].
Qed.
```

The public search contract. The object fields are opened, the buffer search is verified, and the complete representation is reconstructed. The semantic lookup equation and canonical absent value are exposed to clients.

```
Lemma probe_spec m cap (a : val_cjr) (k : Z) :
  is_hashmap m cap a -∗
  WP call_probe a k
    {{ res, ∃ found idx v,
        ⌜ res = StructV [LitV (LitBool found); LitV (LitInt (Z.of_nat idx)); LitV (
  LitInt v)] ⌝ ∗
        ⌜ m !! k = if found then Some v else None ⌝ ∗
        ⌜ found = false → v = 0%Z ⌝ ∗
        is_hashmap m cap a }}.
Proof.
  iIntros "Hm". iDestruct "Hm" as
    (o sz ta ka va tags keys vals) "(%Ha & %Hm & %Hsz & Hsz & Hta & Hka & Hva &
  Hslots)".
  iDestruct "Hslots" as "(Ht & Hk & Hv & %Lt & %Lk & %Lv & %Hwf)".
  iDestruct (is_array_obj_hm with "Ht") as (ot) "(%Ht & Ht)".
  iDestruct (is_array_obj_hm with "Hk") as (ok) "(%Hk & Hk)".
  iDestruct (is_array_obj_hm with "Hv") as (ov) "(%Hv & Hv)".
  subst a ta ka va sz m.
  assert (0 < cap)%nat as Hcap.
  { destruct (wf_pow _ _ _ _ Hwf) as [e ->]. apply pow_positive. }
  unfold call_probe, probe_for_body, probe_open. hm_pure.
  hm_read. hm_admin. hm_read. hm_admin. hm_read. hm_admin.
```

```
  hm_take. iApply (wp_wand with "[Ht]").
  { iApply (array.length_spec with "Ht"). }
  iIntros (r) "[%Hr Ht]". subst r. rewrite Lt. hm_admin.
  try hm_take. iApply (wp_wand with "[]").
  { iApply residue_wp. lia. }
  iIntros (r) "%Hr". subst r. hm_args.
  assert (0 ≤ residue k (Z.of_nat cap))%Z as Hres by
    (pose proof (residue_spec k (Z.of_nat cap) ltac:(lia)); lia).
  replace (residue k (Z.of_nat cap)) with (Z.of_nat (bucket k cap)) by
    (unfold bucket; rewrite Z2Nat.id; lia).
  iApply (wp_wand with "[Ht Hk Hv]").
  { iApply (scan_spec cap (bucket k cap) 0 cap _ _ _ tags keys vals k None with "Ht
  Hk Hv");
      try done; try (apply bucket_lt; done); lia. }
  iIntros (r) "(%Hr & Ht & Hk & Hv)".
  assert (count_occ tags < cap)%nat as Hcount by
    (pose proof (wf_load _ _ _ _ Hwf); lia).
  pose proof (probe_shape cap tags keys vals k Hcap Lt Lk Lv
    (wf_ok _ _ _ _ Hwf) (wf_dup _ _ _ _ Hwf) (wf_place _ _ _ _ Hwf) Hcount) as Hshape.

  change (r = probe_val (probe cap (bucket k cap) tags keys vals k)) in Hr.
  destruct (probe cap (bucket k cap) tags keys vals k) as [idx v|idx]; simpl in *;
  subst r.
  - iExists true, idx, v. iSplit; [done|]. iSplit; [done|]. iSplit; [by iPureIntro;
  discriminate|].
    iExists o, (Z.of_nat (count_occ tags)), (LitV (LitObj ot)), (LitV (LitObj ok)),
      (LitV (LitObj ov)), tags, keys, vals. unfold is_hm_slots. iFrame. repeat iSplit;
   done.
  - destruct Hshape as (_ & _ & _ & Hnone).
    iExists false, idx, 0%Z. iSplit; [done|]. iSplit; [done|]. iSplit; [done|].
    iExists o, (Z.of_nat (count_occ tags)), (LitV (LitObj ot)), (LitV (LitObj ok)),
      (LitV (LitObj ov)), tags, keys, vals. unfold is_hm_slots. iFrame. repeat iSplit;
   done.
Qed.
```

Presence queries. The probe contract supplies the abstract lookup meaning. Projecting its first component yields exactly the presence boolean.

```
Lemma contains_spec m cap (a : val_cjr) (k : Z) :
  is_hashmap m cap a -∗
  WP call_contains a k
    {{ v, ⌜ v = LitV (LitBool (bool_decide (is_Some (m !! k)))) ⌝ ∗ is_hashmap m cap a
   }}.
Proof.
  iIntros "Hm". iDestruct (is_hashmap_obj with "Hm") as (o) "(%Ha & Hm)". subst a.
  unfold call_contains, contains_body.
  iApply (wp_bind [AppLCtx (Val (StructV [LitV (LitObj o); LitV (LitInt k)]))]).
  iApply wp_rec. iIntros "!> _". iApply wp_app. simpl. hm_admin.
  try hm_take. hm_admin. iApply (wp_wand with "[Hm]").
  { iApply (probe_spec with "Hm"). }
  iIntros (r) "(%found & %idx & %v & %Hr & %Hlookup & %Hzero & Hm)". subst r.
  hm_pure. rewrite Hlookup. destruct found; simpl; iFrame; done.
Qed.
```

Value queries. The two projections form the option result. The canonical absent-value fact is needed to prove that a missing result carries zero.

```
Lemma get_spec m cap (a : val_cjr) (k : Z) :
  is_hashmap m cap a -∗
  WP call_get a k
    {{ res, ⌜ res = match m !! k with
                    | Some v => StructV [LitV (LitBool true); LitV (LitInt v)]
                    | None => StructV [LitV (LitBool false); LitV (LitInt 0)]
                    end ⌝ ∗ is_hashmap m cap a }}.
Proof.
  iIntros "Hm". iDestruct (is_hashmap_obj with "Hm") as (o) "(%Ha & Hm)". subst a.
  unfold call_get, get_body.
  iApply (wp_bind [AppLCtx (Val (StructV [LitV (LitObj o); LitV (LitInt k)]))]).
  iApply wp_rec. iIntros "!> _". iApply wp_app. simpl. hm_admin.
  try hm_take. hm_admin. iApply (wp_wand with "[Hm]").
  { iApply (probe_spec with "Hm"). }
  iIntros (r) "(%found & %idx & %v & %Hr & %Hlookup & %Hzero & Hm)". subst r.
  hm_pure. rewrite Hlookup. destruct found; simpl; iFrame.
  - done.
  - rewrite (Hzero eq_refl). done.
Qed.
```

Replacing the buffers after rehashing. New array ownership is established first, then the rehash theorem is applied, and only afterward are the three object fields replaced. Equality of map contents proves equality of occupied counts.

```
Lemma grow_spec m cap (a : val_cjr) (minCap : nat) :
  is_hashmap m cap a -∗
  WP call_grow a (Z.of_nat minCap)
    {{ u, ⌜ u = LitV LitUnit ⌝ ∗ is_hashmap m (doubled_cap cap) a }}.
Proof.
  iIntros "Hm". iDestruct "Hm" as
    (o sz ta ka va tags keys vals) "(%Ha & %Hm & %Hsz & Hsz & Hta & Hka & Hva &
  Hslots)".
  iDestruct "Hslots" as "(Ht & Hk & Hv & %Lt & %Lk & %Lv & %Hwf)".
  iDestruct (is_array_obj_hm with "Ht") as (ot) "(%Ht & Ht)".
  iDestruct (is_array_obj_hm with "Hk") as (ok) "(%Hk & Hk)".
  iDestruct (is_array_obj_hm with "Hv") as (ov) "(%Hv & Hv)". subst a ta ka va sz m.
  set (nc := doubled_cap cap).
  destruct (wf_pow _ _ _ _ Hwf) as [e Hpow].
  assert (nc = (2 ^ S (S e))%nat) as Hnc by (apply doubled_cap_pow; done).
  assert (0 < nc)%nat as Hpos by (rewrite Hnc; apply pow_positive).
  assert (wf_table nc (replicate nc 0%Z) (replicate nc 0%Z) (replicate nc 0%Z)) as
  Hempty
    by (apply zeros_wf; eauto).
  unfold call_grow, grow_body. iApply wp_app. simpl. hm_admin.
  hm_read. hm_admin. hm_read. hm_admin. hm_read. hm_admin.
  hm_take. iApply (wp_wand with "[Ht]"). { iApply (array.length_spec with "Ht"). }
```

```
  iIntros (r) "[%Hr Ht]". subst r. rewrite Lt. hm_admin.
  replace (Z.of_nat cap + Z.of_nat cap)%Z with (Z.of_nat nc) by (unfold nc,
  doubled_cap; lia).
  hm_take. iApply (wp_wand with "[]"). { iApply array.make_spec. exact Hpos. }
  iIntros (nta) "Hnt". iDestruct (is_array_obj_hm with "Hnt") as (ont) "(%Hnta & Hnt)
  ". subst nta. hm_admin.
  hm_take. iApply (wp_wand with "[]"). { iApply array.make_spec. exact Hpos. }
  iIntros (nka) "Hnk". iDestruct (is_array_obj_hm with "Hnk") as (onk) "(%Hnka & Hnk)
  ". subst nka. hm_admin.
  hm_take. iApply (wp_wand with "[]"). { iApply array.make_spec. exact Hpos. }
  iIntros (nva) "Hnv". iDestruct (is_array_obj_hm with "Hnv") as (onv) "(%Hnva & Hnv)
  ". subst nva. hm_admin.
  rewrite <- Lt.
  iApply (wp_wand with "[Ht Hk Hv Hnt Hnk Hnv]").
  { iApply (rehash_spec tags keys vals 0 tags keys vals _ _ _ nc _ _ _
      (replicate nc 0%Z) (replicate nc 0%Z) (replicate nc 0%Z)
      with "Ht Hk Hv Hnt Hnk Hnv"); try done.
    - lia.
    - lia.
    - exact (wf_ok _ _ _ _ Hwf).
    - exact (wf_dup _ _ _ _ Hwf).
    - intros x Hin. rewrite contents_zeros. done.
    - rewrite count_zeros. pose proof (wf_load _ _ _ _ Hwf). unfold nc, doubled_cap.
  lia. }
  iIntros (r) "Harrays".
  destruct (grow_table cap tags keys vals) as [[t2 k2] v2] eqn:Hg.
  assert (rehash_acc nc tags keys vals (replicate nc 0%Z) (replicate nc 0%Z)
    (replicate nc 0%Z) = (t2,k2,v2)) as Hrehash by exact Hg.
  iEval (rewrite Hrehash) in "Harrays".
  iDestruct "Harrays" as "(%Hr & Ht & Hk & Hv & Hnt & Hnk & Hnv)". subst r. hm_admin.
  hm_write. hm_admin. hm_write. hm_admin. hm_write.
  pose proof (grow_table_spec cap tags keys vals e Hpow Hwf) as Hpure.
  rewrite Hg in Hpure. destruct Hpure as [Hwf2 Hcontents].
  assert (count_occ t2 = count_occ tags) as Hcount.
  { pose proof (size_contents tags keys vals ltac:(lia) ltac:(lia) (wf_dup _ _ _ _
  Hwf)) as Hold.
    pose proof (size_contents t2 k2 v2
      ltac:(pose proof (wf_tags_len _ _ _ _ Hwf2); pose proof (wf_keys_len _ _ _ _
  Hwf2); lia)
      ltac:(pose proof (wf_tags_len _ _ _ _ Hwf2); pose proof (wf_vals_len _ _ _ _
  Hwf2); lia)
      (wf_dup _ _ _ _ Hwf2)) as Hnew.
    rewrite Hcontents in Hnew. lia. }
  iSplit; [done|]. iExists o, (Z.of_nat (count_occ tags)), (LitV (LitObj ont)),
    (LitV (LitObj onk)), (LitV (LitObj onv)), t2, k2, v2.
  unfold is_hm_slots. iFrame. repeat iSplit; try done; iPureIntro; first [lia|apply
  Hwf2].
Qed.
```

The public insertion proof. The initial presence check separates overwrite from insertion. Only the missing-key branch checks the load threshold. Either strict room already holds or one verified doubling establishes it.

```
Lemma add_spec m cap (a : val_cjr) (k v : Z) :
  is_hashmap m cap a -∗
  WP call_add a k v
    {{ u, ⌜ u = LitV LitUnit ⌝ ∗ ∃ cap', is_hashmap (<[k:=v]> m) cap' a }}.
Proof.
  iIntros "Hm". iDestruct (is_hashmap_obj with "Hm") as (o) "(%Ha & Hm)". subst a.
  iDestruct (hashmap_bounds with "Hm") as "[%Hbounds Hm]".
  destruct Hbounds as [Hcap Hload].
  unfold call_add, add_body. iApply wp_app. simpl. hm_admin.
  iApply (wp_wand with "[Hm]"). { iApply (probe_spec with "Hm"). }
  iIntros (r) "(%found & %idx & %old & %Hr & %Hlookup & %Hzero & Hm)". subst r.
  hm_admin.
  destruct found.
  - iApply wp_if_true. hm_admin. iApply (wp_wand with "[Hm]").
    { iApply (add_room_spec with "Hm"); [rewrite Hlookup; done|intros Hnone;
  congruence]. }
    iIntros (r) "[%Hr Hm]". iSplit; [done|]. iExists cap. iFrame.
  - iApply wp_if_false. hm_admin. hm_take.
    iApply (wp_wand with "[Hm]"). { iApply (size_spec with "Hm"). }
    iIntros (r) "[%Hr Hm]". subst r. hm_admin. hm_take.
    iApply (wp_wand with "[Hm]"). { iApply (capacity_spec with "Hm"). }
    iIntros (r) "[%Hr Hm]". subst r. hm_admin.
    destruct (decide (cap ≤ 2 * size m)%nat) as [Hgrow|Hroom].
    + rewrite bool_decide_eq_true_2; [|lia]. iApply wp_if_true. hm_admin.
      replace (Z.of_nat cap + 1)%Z with (Z.of_nat (S cap)) by lia.
      iApply (wp_wand with "[Hm]"). { iApply (grow_spec with "Hm"). }
      iIntros (r) "[%Hr Hm]". subst r. hm_admin.
      iApply (wp_wand with "[Hm]").
      { iApply (add_room_spec with "Hm"); [rewrite Hlookup; done|intros _; unfold
  doubled_cap; lia]. }
      iIntros (r) "[%Hr Hm]". iSplit; [done|]. iExists (doubled_cap cap). iFrame.
    + rewrite bool_decide_eq_false_2; [|lia]. iApply wp_if_false. hm_admin.
      iApply (wp_wand with "[Hm]").
      { iApply (add_room_spec with "Hm"); [rewrite Hlookup; done|intros _; lia]. }
      iIntros (r) "[%Hr Hm]". iSplit; [done|]. iExists cap. iFrame.
Qed.
```

The public removal proof. The found branch marks the slot as a tombstone and uses the occupied-count decrement equation to update the size. The missing branch proves deletion of an absent map key.

```
Lemma remove_spec m cap (a : val_cjr) (k : Z) :
  is_hashmap m cap a -∗
  WP call_remove a k
    {{ res, ⌜ res = match m !! k with
                    | Some v => LitV (LitInt v)
                    | None => LitV (LitInt 0)
                    end ⌝ ∗ is_hashmap (delete k m) cap a }}.
Proof.
  iIntros "Hm". iDestruct "Hm" as
    (o sz ta ka va tags keys vals) "(%Ha & %Hm & %Hsz & Hsz & Hta & Hka & Hva &
  Hslots)".
```

```
  iDestruct "Hslots" as "(Ht & Hk & Hv & %Lt & %Lk & %Lv & %Hwf)".
  iDestruct (is_array_obj_hm with "Ht") as (ot) "(%Ht & Ht)".
  iDestruct (is_array_obj_hm with "Hk") as (ok) "(%Hk & Hk)".
  iDestruct (is_array_obj_hm with "Hv") as (ov) "(%Hv & Hv)". subst a ta ka va sz m.
  unfold call_remove, remove_body. iApply wp_app. simpl. hm_admin.
  hm_read. hm_admin. hm_read. hm_admin. hm_read. hm_admin.
  hm_take. iApply (wp_wand with "[Ht]"). { iApply (array.length_spec with "Ht"). }
  iIntros (r) "[%Hr Ht]". subst r. rewrite Lt. hm_admin.
  iApply (wp_wand with "[Ht Hk Hv]").
  { iApply (buffer_probe_spec with "Ht Hk Hv"); try done; eapply cap_pos; exact Hwf. }

  iIntros (r) "(%Hr & Ht & Hk & Hv)". subst r.
  pose proof (probe_slots cap tags keys vals k Hwf) as Hslots.
  pose proof (probe_shape cap tags keys vals k (cap_pos _ _ _ _ Hwf) Lt Lk Lv
    (wf_ok _ _ _ _ Hwf) (wf_dup _ _ _ _ Hwf) (wf_place _ _ _ _ Hwf)
    ltac:(pose proof (wf_load _ _ _ _ Hwf); pose proof (cap_pos _ _ _ _ Hwf); lia))
  as Hlookup.
  pose proof (write_remove_spec cap tags keys vals k Hwf) as Hremove.
  unfold write_remove in Hremove.
  destruct (probe cap (bucket k cap) tags keys vals k) as [idx old|idx]; simpl in *.
  - destruct Hslots as (Hi & Htag & Hkey & Hvalue).
    destruct Hremove as [Hwf2 Hcontents].
    hm_admin. iApply wp_if_true. hm_admin. iApply (wp_bind [SeqCtx _]).
    iApply (wp_wand with "[Ht]"). { iApply (arr_set_spec tags _ idx tag_occ tag_tomb
  Htag with "Ht"). }
    iIntros (r) "[%Hr Ht]". subst r. hm_admin. hm_read. hm_admin.
    hm_write. hm_admin. rewrite Hlookup.
    iSplit; [done|]. iExists o, (Z.of_nat (count_occ tags) - 1)%Z,
      (LitV (LitObj ot)), (LitV (LitObj ok)), (LitV (LitObj ov)), (<[idx:=tag_tomb]>
  tags), keys, vals.
    unfold is_hm_slots. iFrame. repeat iSplit; try done; iPureIntro;
      first [pose proof (count_bury tags idx Htag); lia|apply Hwf2].
  - destruct Hremove as [Hwf2 Hcontents]. destruct Hlookup as (_ & _ & _ & Hlookup).
    hm_admin. iApply wp_if_false. hm_admin. rewrite Hlookup.
    iSplit; [done|]. iExists o, (Z.of_nat (count_occ tags)), (LitV (LitObj ot)),
      (LitV (LitObj ok)), (LitV (LitObj ov)), tags, keys, vals.
    unfold is_hm_slots. iFrame. repeat iSplit; done.
Qed.
```

The stored size query. The representation equation and size-contents lemma identify the field value with the finite map size.

```
Lemma size_spec m cap (a : val_cjr) :
  is_hashmap m cap a -∗
  WP call_size a {{ v, ⌜ v = LitV (LitInt (Z.of_nat (size m))) ⌝ ∗ is_hashmap m cap a
  }}.
Proof.
  iIntros "Hm".
  iDestruct "Hm" as (o sz ta ka va tags keys vals)
    "(%Ha & %Hmap & %Hsz & Hsize & Hta & Hka & Hva & Hslots)".
  subst a sz. iDestruct "Hslots" as "(Ht & Hk & Hv & %Lt & %Lk & %Lv & %Hwf)".
  assert (size m = count_occ tags) as Hcount.
  { rewrite <- Hmap. apply size_contents; try lia. exact (wf_dup _ _ _ _ Hwf). }
```

```
  unfold call_size, size_body. hm_pure. hm_read.
  iSplit; [by rewrite Hcount|].
  iExists o, (Z.of_nat (count_occ tags)), ta, ka, va, tags, keys, vals.
  iFrame. repeat iSplit; try done.
Qed.
```

The capacity query. The owned tag array length equals the abstract capacity, so reading its length field returns that capacity.

```
Lemma capacity_spec m cap (a : val_cjr) :
  is_hashmap m cap a -∗
  WP call_capacity a {{ v, ⌜ v = LitV (LitInt (Z.of_nat cap)) ⌝ ∗ is_hashmap m cap a
  }}.
Proof.
  iIntros "Hm".
  iDestruct "Hm" as (o sz ta ka va tags keys vals)
    "(%Ha & %Hmap & %Hsz & Hsize & Hta & Hka & Hva & Hslots)".
  subst a sz. iDestruct "Hslots" as "(Ht & Hk & Hv & %Lt & %Lk & %Lv & %Hwf)".
  iDestruct "Ht" as (ot base) "(%Hta & Hlen & Hptr & Hblk & Hcells)". subst ta.
  unfold call_capacity, capacity_body. hm_pure. hm_read. hm_pure. hm_read.
  iSplit; [by rewrite Lt|].
  iExists o, (Z.of_nat (count_occ tags)), (LitV (LitObj ot)), ka, va, tags, keys,
  vals.
  iFrame. repeat iSplit; try done.
Qed.
```

The emptiness query. The field equality and occupied-count theorem connect the zero comparison to emptiness of the mathematical map.

```
Lemma is_empty_spec m cap (a : val_cjr) :
  is_hashmap m cap a -∗
  WP call_is_empty a
    {{ v, ⌜ v = LitV (LitBool (bool_decide (size m = 0%nat))) ⌝ ∗ is_hashmap m cap a
  }}.
Proof.
  iIntros "Hm".
  iDestruct "Hm" as (o sz ta ka va tags keys vals)
    "(%Ha & %Hmap & %Hsz & Hsize & Hta & Hka & Hva & Hslots)".
  subst a sz. iDestruct "Hslots" as "(Ht & Hk & Hv & %Lt & %Lk & %Lv & %Hwf)".
  assert (size m = count_occ tags) as Hcount.
  { rewrite <- Hmap. apply size_contents; try lia. exact (wf_dup _ _ _ _ Hwf). }
  unfold call_is_empty, is_empty_body. hm_pure. hm_read. hm_pure.
  iSplit.
  { iPureIntro. f_equal. f_equal. apply bool_decide_ext. rewrite Hcount. lia. }
  iExists o, (Z.of_nat (count_occ tags)), ta, ka, va, tags, keys, vals.
  iFrame. repeat iSplit; try done.
Qed.
```

Initialization followed by size. The allocation result is bound to the next call, whose contract returns zero for the empty map.

```
Lemma init_then_size_spec :
  ⊢ WP init_then_size
    {{ v, ∃ m, ⌜ v = LitV (LitInt 0) ⌝ ∗ is_hashmap ∅ default_capacity m }}.
Proof.
  iApply (wp_bind [LetCtx (Some "m") (size_at (Var "m"))]).
  iApply wp_wand. { iApply init_spec. }
  iIntros (m) "Hm". iApply wp_let. simpl.
  iApply (wp_wand with "[Hm]"). { iApply (size_spec with "Hm"). }
  iIntros (v) "[%Hv Hm]". iExists m. iSplit; [done|]. iFrame.
Qed.
```

Initialization followed by capacity. The initialized representation supplies the default capacity to the query contract.

```
Lemma init_then_capacity_spec :
  ⊢ WP init_then_capacity
    {{ v, ∃ m, ⌜ v = LitV (LitInt (Z.of_nat default_capacity)) ⌝ ∗
                is_hashmap ∅ default_capacity m }}.
Proof.
  iApply (wp_bind [LetCtx (Some "m") (capacity_at (Var "m"))]).
  iApply wp_wand. { iApply init_spec. }
  iIntros (m) "Hm". iApply wp_let. simpl.
  iApply (wp_wand with "[Hm]"). { iApply (capacity_spec with "Hm"). }
  iIntros (v) "[%Hv Hm]". iExists m. iSplit; [done|]. iFrame.
Qed.
```

Initialization followed by emptiness. The empty finite map supplies the true result to the emptiness query.

```
Lemma init_then_empty_spec :
  ⊢ WP init_then_empty
    {{ v, ∃ m, ⌜ v = LitV (LitBool true) ⌝ ∗ is_hashmap ∅ default_capacity m }}.
Proof.
  iApply (wp_bind [LetCtx (Some "m") (is_empty_at (Var "m"))]).
  iApply wp_wand. { iApply init_spec. }
  iIntros (m) "Hm". iApply wp_let. simpl.
  iApply (wp_wand with "[Hm]"). { iApply (is_empty_spec with "Hm"). }
  iIntros (v) "[%Hv Hm]". simpl in Hv. iExists m. iSplit; [done|]. iFrame.
Qed.
```

Insertion followed by lookup. The new capacity is unpacked from the insertion result and passed to the query contract. The finite-map insertion law determines the returned value.

```
Lemma add_then_get_spec m cap (a : val_cjr) (k v : Z) :
  is_hashmap m cap a -∗
  WP add_then_get a k v
    {{ res, ⌜ res = StructV [LitV (LitBool true); LitV (LitInt v)] ⌝ ∗
            (∃ cap', is_hashmap (<[k:=v]> m) cap' a) }}.
Proof.
```

```
    iIntros "Hm". unfold add_then_get.
    iApply (wp_bind [LetCtx None (call_get a k)]).
    iApply (wp_wand with "[Hm]"). { iApply (add_spec with "Hm"). }
    iIntros (u) "[%Hu Hm]". iDestruct "Hm" as (cap') "Hm". subst u.
    iApply wp_let. simpl. iApply (wp_wand with "[Hm]"). { iApply (get_spec with "Hm"). }

    iIntros (r) "[%Hr Hm]". rewrite lookup_insert in Hr. iSplit; [done|]. iExists cap'.
     iFrame.
  Qed.
```

Insertion followed by presence. The same composition uses the insertion lookup law to establish a true presence result.

```
  Lemma add_then_contains_spec m cap (a : val_cjr) (k v : Z) :
    is_hashmap m cap a -∗
    WP add_then_contains a k v
      {{ b, ⌜ b = LitV (LitBool true) ⌝ ∗ (∃ cap', is_hashmap (<[k:=v]> m) cap' a) }}.
  Proof.
    iIntros "Hm". unfold add_then_contains.
    iApply (wp_bind [LetCtx None (call_contains a k)]).
    iApply (wp_wand with "[Hm]"). { iApply (add_spec with "Hm"). }
    iIntros (u) "[%Hu Hm]". iDestruct "Hm" as (cap') "Hm". subst u.
    iApply wp_let. simpl. iApply (wp_wand with "[Hm]"). { iApply (contains_spec with "
    Hm"). }
    iIntros (r) "[%Hr Hm]". rewrite lookup_insert in Hr. simpl in Hr.
    iSplit; [done|]. iExists cap'. iFrame.
  Qed.
```

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

struct Int64Option {
    public var ok: Bool
    public var value: Int64
    public init(ok: Bool, value: Int64) {
        this.ok = ok
        this.value = value
    }
}

struct ProbeResult {
    public var found: Bool
    public var index: Int64
    public var value: Int64
    public init(found: Bool, index: Int64, value: Int64) {
        this.found = found
        this.index = index
        this.value = value
    }
}
```

```
class Array {
    public var len: Int64
    public var data: CPointer<Int64>
    public init(len: Int64, data: CPointer<Int64>) {
        this.len = len
        this.data = data
    }
}

class HashMap {
    public var mySize: Int64
    public var tags: Array
    public var keys: Array
    public var vals: Array
    public init(mySize: Int64, tags: Array, keys: Array, vals: Array) {
        this.mySize = mySize
        this.tags = tags
        this.keys = keys
        this.vals = vals
    }

    public func probe(k: Int64): ProbeResult {
        let tags = this.tags
        let keys = this.keys
        let vals = this.vals
        let cap = length(tags)
        let start = residue(k, cap)
        scan(cap, start, 0, cap, tags, keys, vals, k, -1)
    }

    public func addRoom(k: Int64, v: Int64, found: Bool): Unit {
        let tags = this.tags
        let keys = this.keys
        let vals = this.vals
        let cap = length(tags)
        put(tags, keys, vals, cap, k, v)
        if (found) {
            ()
        } else {
            let sz = this.mySize
            this.mySize = sz + 1
        }
    }

    public func size(): Int64 {
        this.mySize
    }

    public func capacity(): Int64 {
        let tags = this.tags
        tags.len
    }
```

```
    public func isEmpty(): Bool {
        this.mySize == 0
    }

    public func contains(k: Int64): Bool {
        let res = this.probe(k)
        res.found
    }

    public func get(k: Int64): Int64Option {
        let res = this.probe(k)
        Int64Option(res.found, res.value)
    }

    public func add(k: Int64, v: Int64): Unit {
        let res = this.probe(k)
        let found = res.found
        if (found) {
            this.addRoom(k, v, found)
        } else {
            let sz = this.size()
            let cap = this.capacity()
            if (cap <= (sz + sz)) {
                this.grow(cap + 1)
            }
            this.addRoom(k, v, found)
        }
    }

    public func remove(k: Int64): Int64 {
        let tags = this.tags
        let keys = this.keys
        let vals = this.vals
        let cap = length(tags)
        let res = bufferProbe(tags, keys, vals, cap, k)
        if (res.found) {
            let idx = res.index
            let old = res.value
            arraySet(tags, idx, 2)
            let sz = this.mySize
            this.mySize = sz - 1
            old
        } else {
            0
        }
    }

    public func grow(min: Int64): Unit {
        let tags = this.tags
        let keys = this.keys
        let vals = this.vals
        let cap = length(tags)
        let newCap = cap + cap
        let ntags = make(newCap)
```

```
        let nkeys = make(newCap)
        let nvals = make(newCap)
        rehash(cap, 0, tags, keys, vals, newCap, ntags, nkeys, nvals)
        this.tags = ntags
        this.keys = nkeys
        this.vals = nvals
    }
}

func make(n: Int64): Array {
    let p = unsafe { CPointer<Int64>(malloc(UIntNative(8 * n))) }
    Array(n, p)
}

func length(a: Array): Int64 {
    a.len
}

func arrayGet(arr: Array, i: Int64): Int64 {
    let p = arr.data
    unsafe { (p + i).read() }
}

func arraySet(arr: Array, i: Int64, x: Int64): Unit {
    let p = arr.data
    unsafe { (p + i).write(x) }
}

func residue(k: Int64, cap: Int64): Int64 {
    var rem = k
    while (rem <= -1) {
        rem = rem + cap
    }
    while (cap <= rem) {
        rem = rem - cap
    }
    rem
}

func scan(fuel: Int64, start: Int64, step: Int64, cap: Int64, tags: Array, keys: Array,
     vals: Array, k: Int64, hole: Int64): ProbeResult {
    if (fuel <= 0) {
        ProbeResult(false, if (hole == -1) { start } else { hole }, 0)
    } else {
        let slot = if (((start + step) + 1) <= cap) {
            start + step
        } else {
            (start + step) - cap
        }
        let tag = arrayGet(tags, slot)
        if (tag == 0) {
            ProbeResult(false, if (hole == -1) { slot } else { hole }, 0)
        } else {
            if (tag == 1) {
```

```
                let key = arrayGet(keys, slot)
                if (key == k) {
                    ProbeResult(true, slot, arrayGet(vals, slot))
                } else {
                    scan(fuel - 1, start, step + 1, cap, tags, keys, vals, k, hole)
                }
            } else {
                scan(fuel - 1, start, step + 1, cap, tags, keys, vals, k, if (hole ==
    -1) { slot } else { hole })
            }
        }
    }
}

func bufferProbe(tags: Array, keys: Array, vals: Array, cap: Int64, k: Int64):
    ProbeResult {
    let start = residue(k, cap)
    scan(cap, start, 0, cap, tags, keys, vals, k, -1)
}

func put(tags: Array, keys: Array, vals: Array, cap: Int64, k: Int64, v: Int64): Unit {
    let res = bufferProbe(tags, keys, vals, cap, k)
    let idx = res.index
    if (res.found) {
        arraySet(vals, idx, v)
    } else {
        arraySet(tags, idx, 1)
        arraySet(keys, idx, k)
        arraySet(vals, idx, v)
    }
}

func rehash(fuel: Int64, i: Int64, st: Array, sk: Array, sv: Array, cap: Int64, dt:
    Array, dk: Array, dv: Array): Unit {
    if (fuel <= 0) {
        ()
    } else {
        let tag = arrayGet(st, i)
        if (tag == 1) {
            let key = arrayGet(sk, i)
            let value = arrayGet(sv, i)
            put(dt, dk, dv, cap, key, value)
        }
        rehash(fuel - 1, i + 1, st, sk, sv, cap, dt, dk, dv)
    }
}

func initHashMap(): HashMap {
    let tags = { =>
        let p = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 16))) }
        Array(16, p)
    }()
    let keys = { =>
        let p = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 16))) }
```

```
        Array(16, p)
    }()
    let vals = { =>
        let p = unsafe { CPointer<Int64>(malloc(UIntNative(8 * 16))) }
        Array(16, p)
    }()
    HashMap(0, tags, keys, vals)
}

main(): Unit {
    let m = initHashMap()
    m.add(1, 2)
    let r = m.get(1)
    m.size()
}
```

15 Runtime comparison with the Cangjie standard library

The generated implementations were benchmarked against the standard linked list, array, array list, and hash map, specialized at `Int64`, on 9 October 2026, after removing array-list bounds checks. The compiler and the standard library were those of Cangjie 1.0.5 (cjnative), with optimization `-O2`, targeting `aarch64-apple-darwin`. The machine was an Apple M4 Mac mini, with ten cores (four performance and six efficiency), 16 GB RAM, and macOS 26.6.2. These measurements compare the actual printed programs with the SDK's standard types; they do not establish the performance-parity claim for all CJR programs.

15.1 Method and comparable workloads

Each of the four generated source files was compiled with a Cangjie benchmark test containing both implementations. The executable ran in three independent processes, sequentially, with seven samples of each implementation at each size in each process. The 36 workload/size points therefore produced 1,512 timed samples. The measured array-list accesses have no runtime bounds checks; all benchmark inputs use valid indexes. Both implementations were warmed up, and their batch sizes were calibrated independently toward 20 milliseconds. Allocating workloads were capped at $262144/n$ repetitions; other workloads at 65,536. The shortest observed batch was 828 microseconds. The order of the two implementations alternated between trials. There was no core pinning, forced garbage collection, or subtraction of an empty-loop timing.

The timer was `std.time.MonoTime.now()`. Compilation, test-framework setup, steady-state data preparation, printing, and checksum checks were excluded from the timed intervals. Loop and callback overheads were included. Each measured batch returned an observable checksum which was checked after timing; all twelve process-level benchmark suites passed. Map setup checked every key and value, and array-list rotation checked that the final sequence and size were restored.

Lists. A traversal sums the contents via CJR's `head`/`tail`, or the standard list's `firstNode`/`next`. Prepend-and-scan builds the same sequence and traverses it, including allocation. Sizes are 256, 4,096, and 16,384; the unit is time per visited node, or per inserted-and-visited item. The generated list is singly linked, whereas the standard list is doubly linked and maintains collection membership. This is a comparison of common operations, not of identical representations or guarantees.

Arrays. The indexed read visits every initialized cell in the order $(17i) \bmod n$. A write-and-read batch performs n writes and then n reads, dividing its time by $2n$. The sizes are 256, 4,096, and 65,536. Every raw CJR cell is explicitly initialized before its first read; array constructors are excluded. CJR accesses raw pointers without bounds checks, whereas the standard array is indexed through its public API.

Array lists. Indexed read uses the same permutation. Append-and-scan starts at the default constructor, appends n values with natural capacity growth, then reads them all; its unit is one inserted-and-visited item. Both use $n = 256, 4096, 16384$. Front rotation removes index zero and appends the removed value, n times, restoring the sequence; its unit is one remove/append pair and its sizes are 256, 1,024, and 4,096. Setup is excluded for reads and rotations.

Hash maps. Insert-and-scan includes default construction, growth, insertion of keys $0, \ldots, n-1$ with values $3i+1$, and reading every key. Hit and miss batches query $0, \ldots, n-1$ and $n, \ldots, 2n-1$ respectively. A large-key hit batch uses $1000000, \ldots, 1000000+n-1$. These workloads use sizes 256, 4,096, and 16,384. The collision workload uses keys $i(2n)$, at sizes 256, 1,024, and 4,096. They all start in bucket zero at the CJR map's final capacity $2n$; the standard map receives exactly the same keys, using its own bucket policy. Successful reads check presence on both sides. Lookup setup is excluded. Insert-and-scan time is per inserted-and-read item; other map time is per query.

15.2 Measured results

Figure 2 shows the median paired time ratio at the largest tested size of each workload. A ratio below one means the generated code ran faster. There are 21 paired samples per point; the whiskers show their interquartile range, not a confidence interval. The following table also gives the individual median times. Different rows have the workload units defined above, so their absolute values should not be compared as if each row timed the same operation.

Workload	n	CJR (ns)	Std (ns)	Ratio
Linked lists: Traverse	16,384	1.75	2.09	0.84
Linked lists: Prepend + scan	16,384	4.25	14.40	0.30
Arrays: Indexed read	65,536	0.95	0.77	1.23
Arrays: Write + read passes	65,536	0.96	0.55	1.75
Array lists: Indexed read	16,384	0.93	0.82	1.13
Array lists: Append + scan	16,384	3.43	4.36	0.81
Array lists: Front removal + append	4,096	4861.69	305.63	15.79
Hash maps: Insert + scan	16,384	26.43	18.62	1.42
Hash maps: Lookup hit	16,384	5.77	2.40	2.41
Hash maps: Lookup miss	16,384	3.90	1.60	2.38
Hash maps: Large-key hit	16,384	17.39	2.39	7.24
Hash maps: Colliding hit	4,096	18816.41	970.49	19.40

Lists and arrays. Prepend-and-scan on the smaller CJR list representation takes about 0.30 times the standard list time, and traversal about 0.84 times. Those results reflect the different list representations, as well as optimization and runtime behavior. The generated arrays are not faster in these tests: indexed read takes 1.23 times the standard-array time, and the write/read workload 1.75 times. The lack of raw-pointer bounds checks therefore does not imply a measured speed advantage for these loops.

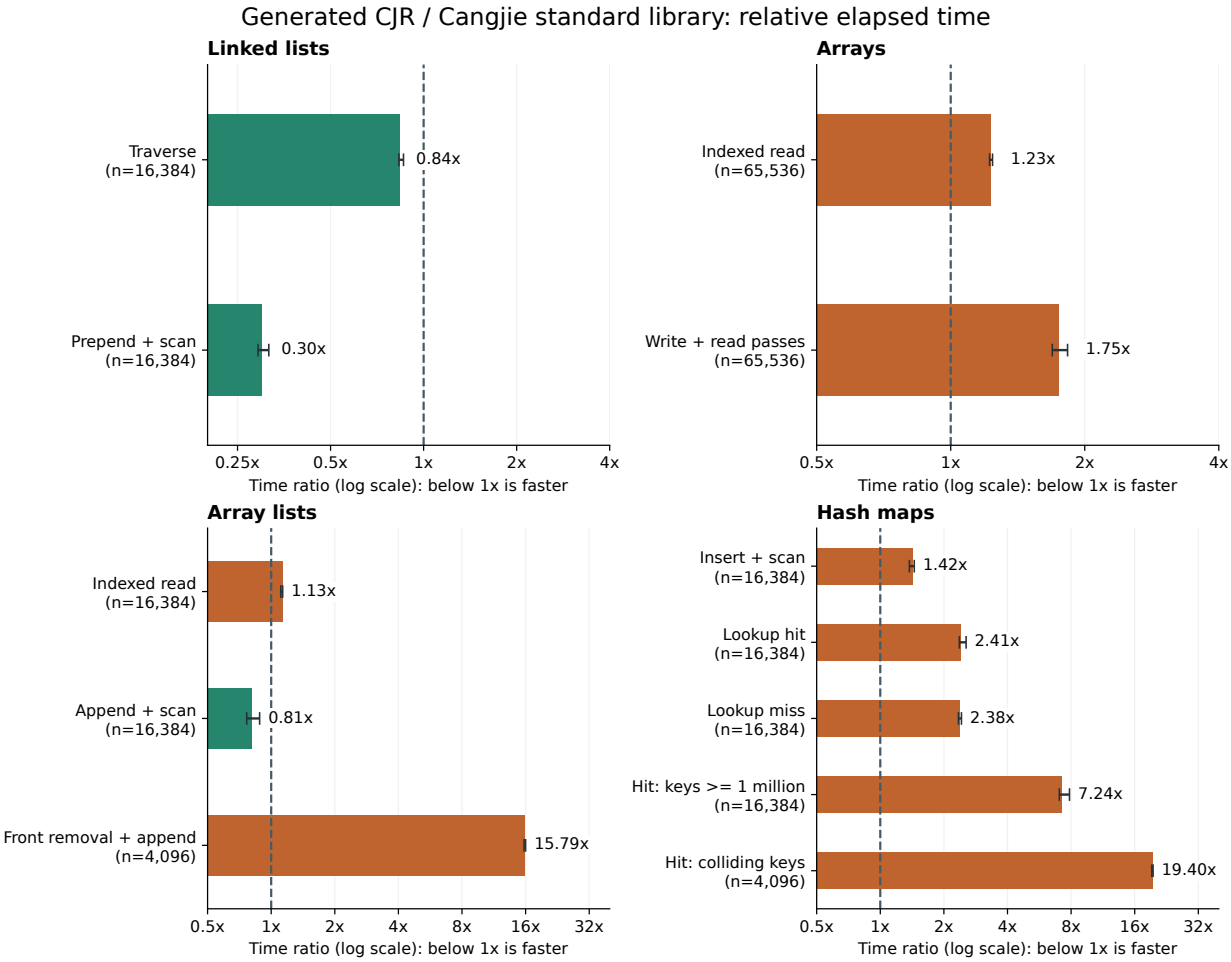


Figure 2: Relative elapsed time of generated CJR and standard-library workloads at their largest tested sizes. The dashed line is equal time; bars and error whiskers are the median and interquartile range of paired CJR/standard ratios. List build/scan, array-list build/scan, and map build/scan include construction and growth; the other setups are excluded.

Array lists. Append-and-scan takes about 0.81 times the standard time, while indexed read takes 1.13 times. Front rotation is much more expensive: about 15.79 times the standard time at $n = 4096$. The generated `remove` explicitly reads and writes every shifted word. These measurements establish the observed cost difference, without an assembly analysis identifying all compiler and library contributions to it.

Hash maps. Ordinary successful lookup takes about 2.41 times the standard time, misses 2.38 times, and insert-and-scan 1.42 times at $n = 16384$. The key pattern can cause a much greater slowdown. The large-key workload at $n = 256$ has median times of approximately 672.4 ns and 2.4 ns per query (roughly 283 times slower for CJR). At $n = 16384$, the paired ratio is still 7.24. Colliding keys at $n = 4096$ take about 19.40 times the standard time. Both maps slow down on this collision workload; the generated map does not have uniform lookup performance.

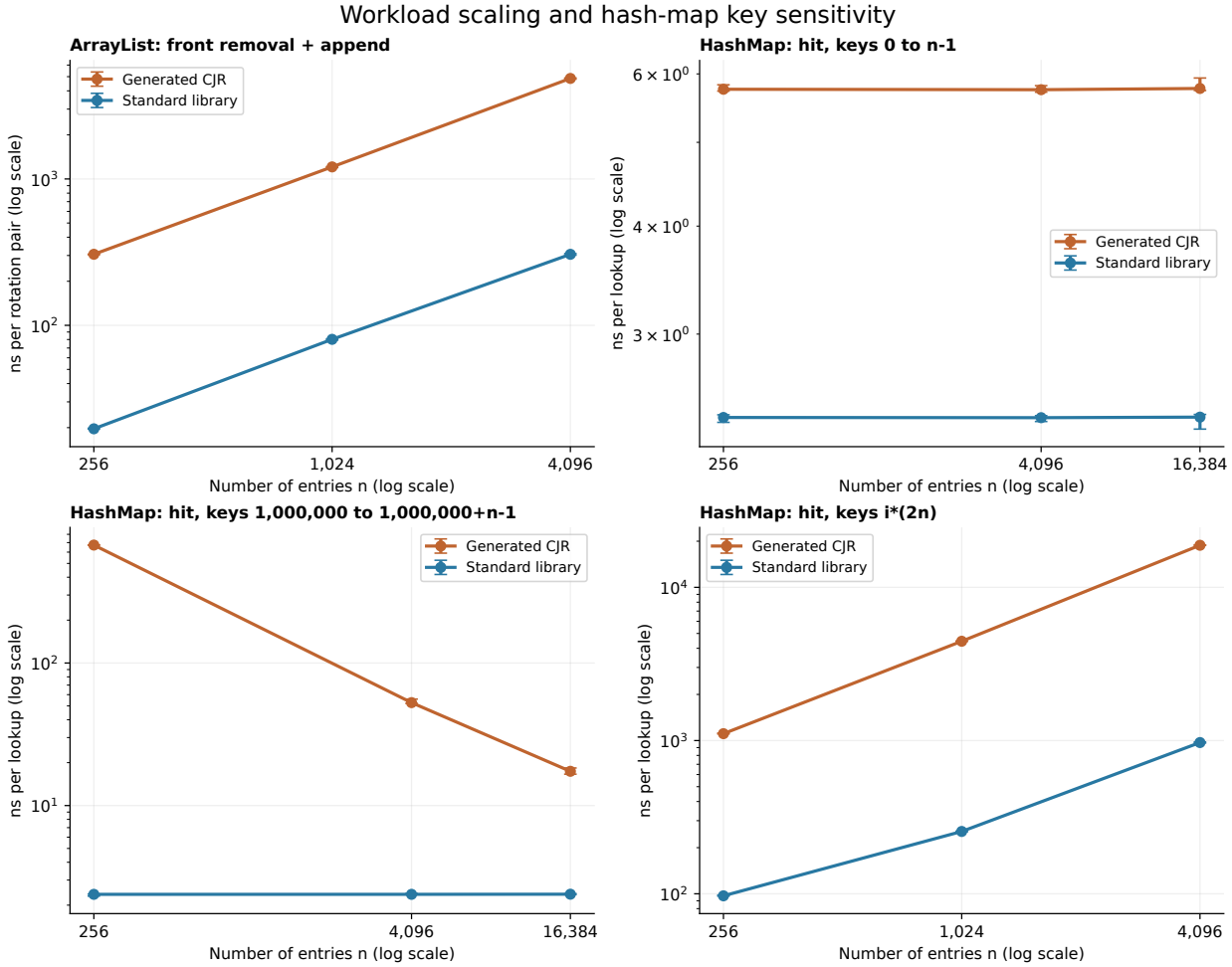


Figure 3: Scaling of front rotation and hash-map queries. Both axes use logarithmic scales. Lines show individual median times, with interquartile error whiskers. In the large-key case, the CJR time falls as capacity grows, because remainder normalization needs fewer subtractions. Colliding keys show increasing query costs for both implementations.

15.3 Interpretation, limitations, and reproduction

The large-key behavior has a concrete source-level explanation: CJR's `residue` routine repeatedly subtracts the capacity. A small map needs many such steps for keys near one million; a larger map needs fewer. The collision workload also combines a long probe with a costly remainder calculation. These are valid keys and are materially different from the sequential-key test. The measurements do not justify treating all hash-map keys as equally cheap or extrapolating to the extremes of `Int64`.

The generated resizable buffers never free their old `malloc` blocks. The harness does not repair that behavior or free buffers; batch limits and process exits bound the experiment's allocation pressure. Standard arrays and collections use the managed runtime instead. This comparison includes their natural allocation costs, but it is not a measurement of sustained memory consumption or the cost of reclamation that a production raw-memory implementation would need.

Similarly, raw array construction is not equivalent to the standard array's initialized construction. Explicitly writing cells before reads avoids using uninitialized contents in the array timing. The generated map still relies on zero-valued fresh tags: its checks passed on this runtime, but the benchmark does not establish or fix the allocation assumption discussed in Appendix A. The tests use valid indexes, bounded values, a warm working set, and single-threaded code on an unpinned desktop process. They support workload-specific comparisons on this machine, not a general correctness, complexity, or performance theorem.

These results replace the previous measurements. Differences between runs do not isolate the effect of removing bounds checks: desktop scheduling, calibration, and runtime variation can also affect the timings.

The reproducible harness is in `benchmark/`. The unchanged source hashes, compiler configuration, every timed sample, and summary statistics are recorded in `benchmark/results/metadata.json`, `raw.csv`, and `summary.json`. The figures and table are generated from that data by `benchmark/plot.py`. Re-run `python3 benchmark/run.py` with the matching SDK and linker SDK paths, followed by `python3 benchmark/plot.py` (Matplotlib is required). `benchmark/README.md` explains the exact build command, units, warmups, repetitions, and the existing macOS linker-stub workaround.

16 Most frequent integer: paired sort and scan

This example takes an initialized array of integers and returns its most frequent value. An empty array returns `None`. If several values have the same maximum frequency, the result is the value encountered first in the original array. For example, $[8, 3, 3, 8]$ returns 8, even though 3 is numerically smaller. The input is preserved.

Verification status. Rocq checks the abstract specification, the complete functional sort-and-scan correctness theorem, and an Iris weakest-precondition proof of the actual CJR operation for both empty and nonempty inputs. The proof includes allocation, input copying, paired Lomuto quicksort, run scanning, the first-occurrence tie rule, both temporary-buffer frees, and preservation of the input. Checked expression equalities connect the loop lemmas to the terms supplied to the printer. The emitted Cangjie program is compiled, tested, and benchmarked; native compilation and the printer are outside the CJR semantic proof, as described below.

16.1 Specification in elementary mathematics

Let $A = (a_0, \ldots, a_{n-1})$ and define

$$f_A(x) = \big|\{i : 0 \le i < n \ \wedge \ a_i = x\}\big|, \qquad t_A(x) = \min\{i : 0 \le i < n \ \wedge \ a_i = x\}.$$

For absent values set $t_A(x) = n$. The required answer r satisfies

$$\begin{aligned} r = \text{None} \quad &\Longleftrightarrow \quad n = 0, \\ r = \text{Some}(x) \quad &\Longrightarrow \quad f_A(x) > 0 \ \wedge \ (\forall y \in \mathbb{Z},\ f_A(y) \le f_A(x)) \\ &\qquad\qquad \wedge \ (\forall y \in \mathbb{Z},\ f_A(y) = f_A(x) \Rightarrow t_A(x) \le t_A(y)). \end{aligned}$$

Among maximum-frequency values the first-index condition determines a unique value: different values cannot occupy the same original index.

English readout. Return no value exactly when the input is empty. Otherwise return a value that occurs in the input, occurs at least as often as every other integer, and whose first occurrence is no later than the first occurrence of any other equally frequent value. Comparing only values, or returning the first group in numeric sort order, would violate this contract.

16.2 Gallina specification

The executable specification uses `count_occ` for frequency and a recursive search for the first index; `mf_first_index_or_len` returns the array length for an absent value. Indices and counts are natural numbers. Values are unbounded Rocq integers.

```
Definition most_frequent_spec (xs : list Z) (answer : option Z) : Prop :=
  match xs, answer with
  | [], None => True
  | [], Some _ => False
  | _ :: _, None => False
  | _ :: _, Some x =>
      (mf_frequency x xs > 0)%nat /\
      (forall y, mf_frequency y xs <= mf_frequency x xs)%nat /\
      (forall y, mf_frequency y xs = mf_frequency x xs ->
         mf_first_index_or_len x xs <= mf_first_index_or_len y xs)%nat
  end.
```

The independently proved `mf_choose xs xs` computes this answer by counting each candidate against the whole input. It is a quadratic reference oracle, not the efficient implementation. Its theorem `mf_choose_correct` proves the specification by induction over candidate order, retaining the earlier candidate on a frequency tie.

The verified functional implementation composes paired Lomuto sort, adjacent-value grouping, and summary selection. Written with `option_map` in place of the equivalent source infix notation:

```
Definition mf_sort_scan xs : option Z :=
  option_map fst (mf_best_summary (mf_run_summaries (mf_sorted_pairs xs))).

Theorem mf_sort_scan_correct xs :
  most_frequent_spec xs (mf_sort_scan xs).
```

This theorem has no summary-validity hypothesis: the grouping and sorting proofs discharge it. It also includes the empty case and the specified tie rule. The CJR refinement theorem below connects the concrete operation to this functional result.

16.3 Algorithm walkthrough and pseudocode

Copy the input into records (a_i, i), represented by two parallel arrays. Sorting these records lexicographically places equal values together and orders their original indices within each group. Every swap moves both fields. For $[8, 3, 3, 8]$, the sorted records are $[(3, 1), (3, 2), (8, 0), (8, 3)]$. Their summaries are $(3, 2, 1)$ and $(8, 2, 0)$, where each triple is (value, count, first index). Both counts are two; the second summary wins because index zero precedes index one.

```
mostFrequent(A):
  n := length(A)
  if n = 0: return None
  values := copy(A); indices := [0, ..., n-1]
  quicksortPairs(values, indices, 0, n)  // half-open interval
  bestCount := 0; bestFirst := n; start := 0
  while start < n:
    x := values[start]; end := start + 1
    first := indices[start]
    while end < n:
      if values[end] != x: break
      first := min(first, indices[end]); end := end + 1
    count := end - start
    if count > bestCount or
       (count = bestCount and first < bestFirst):
      bestValue := x; bestCount := count; bestFirst := first
    start := end
  free(values.data); free(indices.data)
  return Some(bestValue)
```

Lomuto partition uses the last record as pivot. It maintains a boundary i and visits records j from `lo` through `hi-2`; records at most the pivot are swapped into the left prefix. The pivot is then swapped into position i. Recursion sorts $[\texttt{lo}, i)$ and $[i + 1, \texttt{hi})$, leaving the pivot out of both recursive calls.

Under the usual random-order assumption for the distinct paired keys, quicksort takes expected $O(n \log n)$ comparisons; its worst case is $O(n^2)$. This implementation does not randomize pivots. Sorted, reverse-sorted, and all-equal *value* inputs can be particularly poor: in the all-equal case the increasing original indices already put the paired keys in sorted order. Copying and scanning take $O(n)$. The two buffers require $O(n)$ space; recursive stack depth is $O(n)$ in the worst case. Both raw buffers are freed before returning.

16.4 Formal CJR implementation and emitted Cangjie

The implementation lives in `theories/examples/most_frequent.v` as core-language expressions `mf_swap_body`, `mf_partition_body`, `mf_qsort_body`, and `mf_body`. A pair comparison is expressed using the available `LeOp` and `If`; it never subtracts two input values or adds one to an input value, so comparisons also work at the `Int64` boundaries. The group condition uses an explicit `If` because CJR's `AndOp` evaluates both operands. An eager conjunction containing a raw read at `end == n` would access beyond the array.

```
If (BinOp LeOp (Var "v") (Var "pv"))
  (If (BinOp LeOp (Var "pv") (Var "v"))
    (BinOp LeOp (Var "ix") (Var "pi"))
    (Val (LitV (LitBool true))))
  (Val (LitV (LitBool false)))
```

The following is the complete core-language definition of the pair swap, partition, recursive sort, copy, and group scan. Calls package their arguments in a structural value, `VarBind` introduces mutable stack variables, and `Assign` updates those variables. The loop guards, array reads, and final block frees are therefore explicit syntax subject to the CJR semantics.

```
Definition mf_swap_body : expr :=
 (Let (Some "values") (StructLoad (Var "args") 0)
 (Let (Some "indices") (StructLoad (Var "args") 1)
 (Let (Some "i") (StructLoad (Var "args") 2)
 (Let (Some "j") (StructLoad (Var "args") 3)
 (Let (Some "x") (App (Rec None (Some "args") array.get_body) (Struct [] [(Var "values"); (Var "i
    ")]))
 (Let (Some "xi") (App (Rec None (Some "args") array.get_body) (Struct [] [(Var "indices"); (Var "
    i")]))
 (Let (Some "y") (App (Rec None (Some "args") array.get_body) (Struct [] [(Var "values"); (Var "j
    ")]))
 (Let (Some "yi") (App (Rec None (Some "args") array.get_body) (Struct [] [(Var "indices"); (Var "
    j")]))
 (Let None (App (Rec None (Some "args") array.set_body) (Struct [] [(Var "values"); (Var "i"); (
    Var "y")]))
 (Let None (App (Rec None (Some "args") array.set_body) (Struct [] [(Var "indices"); (Var "i"); (
    Var "yi")]))
 (Let None (App (Rec None (Some "args") array.set_body) (Struct [] [(Var "values"); (Var "j"); (
    Var "x")]))
 (App (Rec None (Some "args") array.set_body) (Struct [] [(Var "indices"); (Var "j"); (Var "xi")])
    ))))))))))).

Definition mf_partition_body : expr :=
 (Let (Some "values") (StructLoad (Var "args") 0)
 (Let (Some "indices") (StructLoad (Var "args") 1)
 (Let (Some "lo") (StructLoad (Var "args") 2)
 (Let (Some "hi") (StructLoad (Var "args") 3)
 (Let (Some "pv") (App (Rec None (Some "args") array.get_body) (Struct [] [(Var "values"); (BinOp
     MinusOp (Var "hi") (Val (LitV (LitInt 1))))]))
 (Let (Some "pi") (App (Rec None (Some "args") array.get_body) (Struct [] [(Var "indices"); (
    BinOp MinusOp (Var "hi") (Val (LitV (LitInt 1))))]))
 (VarBind "i" (Var "lo")
 (VarBind "j" (Var "lo")
 (Let None (While (BinOp LeOp (BinOp PlusOp (Var "j") (Val (LitV (LitInt 1)))) (BinOp MinusOp (
    Var "hi") (Val (LitV (LitInt 1)))))
 (Let (Some "v") (App (Rec None (Some "args") array.get_body) (Struct [] [(Var "values"); (Var "j
    ")]))
 (Let (Some "ix") (App (Rec None (Some "args") array.get_body) (Struct [] [(Var "indices"); (Var "
    j")]))
 (Let None (If (If (BinOp LeOp (Var "v") (Var "pv"))
 (If (BinOp LeOp (Var "pv") (Var "v"))
 (BinOp LeOp (Var "ix") (Var "pi"))
 (Val (LitV (LitBool true))))
 (Val (LitV (LitBool false))))
 (Let None (App (Rec None (Some "args") mf_swap_body) (Struct [] [(Var "values"); (Var "indices");
     (Var "i"); (Var "j")]))
```

```
 (Assign "i" (BinOp PlusOp (Var "i") (Val (LitV (LitInt 1))))))
 (Val (LitV LitUnit)))
 (Assign "j" (BinOp PlusOp (Var "j") (Val (LitV (LitInt 1)))))))))
 (Let None (App (Rec None (Some "args") mf_swap_body) (Struct [] [(Var "values"); (Var "indices");
     (Var "i"); (BinOp MinusOp (Var "hi") (Val (LitV (LitInt 1))))]))
 (Var "i")))))))))))).

Definition mf_qsort_body : expr :=
 (Let (Some "values") (StructLoad (Var "args") 0)
 (Let (Some "indices") (StructLoad (Var "args") 1)
 (Let (Some "lo") (StructLoad (Var "args") 2)
 (Let (Some "hi") (StructLoad (Var "args") 3)
 (If (BinOp LeOp (BinOp PlusOp (Var "lo") (Val (LitV (LitInt 1)))) (BinOp MinusOp (Var "hi") (Val
     (LitV (LitInt 1)))))
 (Let (Some "p") (App (Rec None (Some "args") mf_partition_body) (Struct [] [(Var "values"); (Var
     "indices"); (Var "lo"); (Var "hi")]))
 (Let None (App (Var "pairQsort") (Struct [] [(Var "values"); (Var "indices"); (Var "lo"); (Var "
    p")]))
 (App (Var "pairQsort") (Struct [] [(Var "values"); (Var "indices"); (BinOp PlusOp (Var "p") (Val
     (LitV (LitInt 1)))); (Var "hi")])))))
 (Val (LitV LitUnit)))))))).

Definition mf_body : expr :=
 (Let (Some "n") (App (Rec None (Some "a") array.length_body) (Var "input"))
 (If (BinOp EqOp (Var "n") (Val (LitV (LitInt 0))))
 (Struct [] [(Val (LitV (LitBool false))); (Val (LitV (LitInt 0)))])
 (Let (Some "values") (App (Rec None (Some "n") array.make_body) (Var "n"))
 (Let (Some "indices") (App (Rec None (Some "n") array.make_body) (Var "n"))
 (Let None (VarBind "copyAt" (Val (LitV (LitInt 0)))
 (While (BinOp LeOp (BinOp PlusOp (Var "copyAt") (Val (LitV (LitInt 1)))) (Var "n"))
 (Let None (App (Rec None (Some "args") array.set_body) (Struct [] [(Var "values"); (Var "copyAt")
    ; (App (Rec None (Some "args") array.get_body) (Struct [] [(Var "input"); (Var "copyAt")]))])
    )
 (Let None (App (Rec None (Some "args") array.set_body) (Struct [] [(Var "indices"); (Var "copyAt
    "); (Var "copyAt")]))
 (Assign "copyAt" (BinOp PlusOp (Var "copyAt") (Val (LitV (LitInt 1)))))))))
 (Let None (App (Rec (Some "pairQsort") (Some "args") mf_qsort_body) (Struct [] [(Var "values");
    (Var "indices"); (Val (LitV (LitInt 0))); (Var "n")]))
 (VarBind "bestValue" (App (Rec None (Some "args") array.get_body) (Struct [] [(Var "values"); (
    Val (LitV (LitInt 0)))]))
 (VarBind "bestCount" (Val (LitV (LitInt 0)))
 (VarBind "bestFirst" (Var "n")
 (VarBind "start" (Val (LitV (LitInt 0)))
 (Let None (While (BinOp LeOp (BinOp PlusOp (Var "start") (Val (LitV (LitInt 1)))) (Var "n"))
 (Let (Some "value") (App (Rec None (Some "args") array.get_body) (Struct [] [(Var "values"); (
    Var "start")]))
 (VarBind "end" (BinOp PlusOp (Var "start") (Val (LitV (LitInt 1))))
 (VarBind "first" (App (Rec None (Some "args") array.get_body) (Struct [] [(Var "indices"); (Var "
    start")]))
 (Let None (While (If (BinOp LeOp (BinOp PlusOp (Var "end") (Val (LitV (LitInt 1)))) (Var "n"))
 (BinOp EqOp (App (Rec None (Some "args") array.get_body) (Struct [] [(Var "values"); (Var "end")
    ])) (Var "value"))
 (Val (LitV (LitBool false))))
 (Let (Some "ix") (App (Rec None (Some "args") array.get_body) (Struct [] [(Var "indices"); (Var "
    end")]))
 (Let None (If (BinOp LeOp (BinOp PlusOp (Var "ix") (Val (LitV (LitInt 1)))) (Var "first"))
 (Assign "first" (Var "ix"))
 (Val (LitV LitUnit)))
 (Assign "end" (BinOp PlusOp (Var "end") (Val (LitV (LitInt 1)))))))))
```

```
 (Let (Some "count") (BinOp MinusOp (Var "end") (Var "start"))
 (Let None (If (If (BinOp LeOp (BinOp PlusOp (Var "bestCount") (Val (LitV (LitInt 1)))) (Var "
    count"))
 (Val (LitV (LitBool true)))
 (If (BinOp EqOp (Var "bestCount") (Var "count"))
 (BinOp LeOp (BinOp PlusOp (Var "first") (Val (LitV (LitInt 1)))) (Var "bestFirst"))
 (Val (LitV (LitBool false)))))
 (Let None (Assign "bestValue" (Var "value"))
 (Let None (Assign "bestCount" (Var "count"))
 (Assign "bestFirst" (Var "first"))))
 (Val (LitV LitUnit)))
 (Assign "start" (Var "end")))))))))
 (Let None (Free (FieldLoad (Var "values") 1))
 (Let None (Free (FieldLoad (Var "indices") 1))
 (Struct [] [(Val (LitV (LitBool true))); (Var "bestValue")])))))))))))))).
```

The printer uses these expressions directly through `p_most_frequent`; `make cj` regenerates the following standalone program. `Int64Option(false,0)` represents `None`; its numeric field has no meaning in that case.

```
// Generated by theories/examples/emit_cj.v from the CJR core terms.

foreign func malloc(size: UIntNative): CPointer<Unit>
foreign func free(ptr: CPointer<Unit>): Unit

struct Int64Option {
    public var ok: Bool
    public var value: Int64
    public init(ok: Bool, value: Int64) {
        this.ok = ok
        this.value = value
    }
}

class Array {
    public var len: Int64
    public var data: CPointer<Int64>
    public init(len: Int64, data: CPointer<Int64>) {
        this.len = len
        this.data = data
    }
}

func make(n: Int64): Array {
    let p = unsafe { CPointer<Int64>(malloc(UIntNative(8 * n))) }
    Array(n, p)
}

func length(a: Array): Int64 {
    a.len
}

func get(arr: Array, i: Int64): Int64 {
    let p = arr.data
    unsafe { (p + i).read() }
```

```
}

func set(arr: Array, i: Int64, x: Int64): Unit {
    let p = arr.data
    unsafe { (p + i).write(x) }
}

func pairSwap(values: Array, indices: Array, i: Int64, j: Int64): Unit {
    let x = get(values, i)
    let xi = get(indices, i)
    let y = get(values, j)
    let yi = get(indices, j)
    set(values, i, y)
    set(indices, i, yi)
    set(values, j, x)
    set(indices, j, xi)
}

func pairPartition(values: Array, indices: Array, lo: Int64, hi: Int64): Int64 {
    let pv = get(values, hi - 1)
    let pi = get(indices, hi - 1)
    var i = lo
    var j = lo
    while ((j + 1) <= (hi - 1)) {
        let v = get(values, j)
        let ix = get(indices, j)
        if (if (v <= pv) {
            if (pv <= v) { ix <= pi } else { true }
        } else {
            false
        }) {
            pairSwap(values, indices, i, j)
            i = i + 1
        }
        j = j + 1
    }
    pairSwap(values, indices, i, hi - 1)
    i
}

func pairQsort(values: Array, indices: Array, lo: Int64, hi: Int64): Unit {
    if ((lo + 1) <= (hi - 1)) {
        let p = pairPartition(values, indices, lo, hi)
        pairQsort(values, indices, lo, p)
        pairQsort(values, indices, p + 1, hi)
    }
}

func mostFrequent(input: Array): Int64Option {
    let n = length(input)
    if (n == 0) {
        Int64Option(false, 0)
    } else {
        let values = make(n)
```

```
        let indices = make(n)
        var copyAt = 0
        while ((copyAt + 1) <= n) {
            set(values, copyAt, get(input, copyAt))
            set(indices, copyAt, copyAt)
            copyAt = copyAt + 1
        }
        pairQsort(values, indices, 0, n)
        var bestValue = get(values, 0)
        var bestCount = 0
        var bestFirst = n
        var start = 0
        while ((start + 1) <= n) {
            let value = get(values, start)
            var end = start + 1
            var first = get(indices, start)
            while (if ((end + 1) <= n) {
                get(values, end) == value
            } else {
                false
            }) {
                let ix = get(indices, end)
                if ((ix + 1) <= first) {
                    first = ix
                }
                end = end + 1
            }
            let count = end - start
            if (if ((bestCount + 1) <= count) {
                true
            } else {
                if (bestCount == count) {
                    (first + 1) <= bestFirst
                } else {
                    false
                }
            }) {
                bestValue = value
                bestCount = count
                bestFirst = first
            }
            start = end
        }
        unsafe { free(CPointer<Unit>(values.data)) }
        unsafe { free(CPointer<Unit>(indices.data)) }
        Int64Option(true, bestValue)
    }
}

main(): Unit {
    let input = make(1)
    set(input, 0, 42)
    mostFrequent(input)
}
```

The input must have a correct length and valid initialized storage at every in-range index. The empty case reads only the length field and allocates no temporary buffers. The tests construct an empty view over a positive allocation instead of depending on `malloc(0)`. The full native storage contract also requires allocation byte sizes and indices to fit their machine types. As elsewhere in this tutorial, a Rocq proof over unbounded integers does not by itself establish the compiler's `Int64` arithmetic and allocation behavior.

16.5 Correctness argument and checked components

Record preservation. `mf_pair_swap_perm` proves that swapping two records preserves the record multiset, including self-swaps and out-of-range no-op cases in the functional model. Induction on partition fuel lifts this to `mf_lomuto_perm`. The final pivot swap preserves the same multiset, proving `mf_partition_perm`. Induction on recursive fuel composes the two recursive permutations and the partition permutation, proving `mf_qsort_perm`. It follows that no original (value,index) record is lost or duplicated by the functional sort.

Ordering. For a positive width w larger than every record index, define the *proof-only* rank $\rho_w(x, i) = xw + i$. If $x < y$, then $\rho_w(x, i) < \rho_w(y, j)$ because $0 \le i, j < w$; if $x = y$, comparing ranks is exactly comparing indices. Conversely, a rank comparison cannot put a larger value before a smaller value, since an index difference has magnitude less than w. Lemma `mf_rank_order` checks this argument with integer arithmetic. Lemmas `mf_swap_map`, `mf_lomuto_map`, `mf_partition_map`, and `mf_qsort_slice_map` show that mapping records to ranks commutes with the actual functional Lomuto operations. The existing scalar quicksort ordering theorem therefore yields `mf_qsort_sorted`. For input records, `mf_pairs_bounded` supplies $w = n$ when $n > 0$. No rank is computed in the emitted program.

Frequency and provenance. `mf_pairs_values` proves that projecting original records gives exactly the input. Mapping the sort permutation through that projection preserves every frequency, which is `mf_sorted_pairs_frequency`. The lookup lemma and permutation also prove `mf_sorted_pairs_origin`: every sorted record's index points to its value in the original input. This is stronger than preserving a multiset of values alone.

Grouping obligation. In a lexicographically sorted record sequence, all records with value x form one contiguous run. A different value between two x records would have to be both at least x and at most x, contradicting its difference. Scanning from the run's first record until the first different value therefore visits exactly all x records. Its length is $f_A(x)$ and the minimum original index is $t_A(x)$. To mechanize the imperative scan, the inner invariant must state that $[\texttt{start}, \texttt{end})$ is exactly the portion of this run already visited, and that `first` is the minimum index there. It must also retain ownership of both buffers and establish each read is in bounds. The functional grouping is now checked: `mf_run_summaries_sound` proves that each summary has its global frequency, a witnessed minimum original index, and positive count, that summary keys do not repeat, and that all record values are represented. The functional scan builds the same adjacent-value groups by structural recursion from the tail, extending the first summary when its value agrees with the current record. Its induction uses sortedness to exclude an earlier value from every later, different-value group. The forward-loop invariant and its refinement of this grouping are proved by the concrete scan lemmas described below.

Selection. A summary s has value, count, and first-index fields. Its score is better than another summary when its count is larger, or its count is equal and its first index is no larger. The checked lemmas establish a total, transitive score order. `mf_best_summary_sound` proves by induction that the selected summary belongs to the list and dominates every other summary. The theorem `mf_summary_selection_correct` then establishes `most_frequent_spec`, provided all summaries have their true positive frequencies and first indices, and every input value has a summary. Absent values have frequency zero, so the domination condition extends to all integers. The checked theorem `mf_run_summaries_complete` now establishes that precondition for the actual functional groups of the paired sort. Permutation transfers the minimum-index witnesses back to the original input; `mf_first_index_attained` and `mf_first_index_minimum` show that this minimum is exactly the specification's first-occurrence index. Composing these results gives `mf_sort_scan_correct`, a correctness theorem for the complete functional algorithm rather than only the independent quadratic oracle.

Concrete paired-array Iris proof. The predicate `mf_pair_arrays ps oa ob` owns two initialized arrays: one contains the value projection of the record list, and the other contains the original-index projection, embedded from naturals into integers. `mf_swap_arrays_wp` checks all four reads and all four writes of the actual swap body. On a self-swap the two initial lookup facts imply the read values agree, which justifies the second write after the first update. Mapping the functional record swap through both projections gives `mf_swap_pairs_wp`.

`mf_lex_compare_wp` proves the concrete nested comparisons return exactly `mf_pair_leb`. Each partition iteration reads the current record, applies this comparison, optionally swaps it into the stored prefix and increments i, then increments j. `mf_partition_step_wp` accounts for both arrays and both mutable stack cells throughout that step. Induction on the remaining scan length, $(\texttt{hi} - 1) - j$, yields `mf_partition_scan_wp`: its result is exactly the functional `mf_lomuto` result and its final j is `hi-1`. The induction maintains $i \leq j \leq$ `hi-1` and `hi-1` below the array length, so both record reads and swaps are in bounds. Equality lemmas extract the loop directly from `mf_partition_body` and check that it is the expression proved by these step and scan theorems.

`mf_partition_wp` opens the two local stack variables, reads the last record as pivot, applies the scan theorem, performs the final pivot swap, and returns the functional partition index. It returns both array representations and supplies the stack cells for their lexical scope cleanup. The bound $\texttt{lo} \leq p < \texttt{hi}$ makes both recursive subintervals strictly shorter. Induction on a fuel at least $\texttt{hi} - \texttt{lo}$ then proves `mf_qsort_slice_wp` for the actual recursive CJR function. Its postcondition is the exact functional `mf_qsort_slice` array contents, not merely an assertion that some sorted list exists. At full length, `mf_qsort_wp` therefore connects the emitted paired quicksort to the previously proved sorted-permutation model. No abstract comparator or swap correctness is assumed here.

Complete-buffer cleanup. The new `mf_wp_free_block` rule owns a block token for the exact buffer length and all of its raw cells. Its proof deletes the cells in reverse index order from the Iris ghost map, deletes the block token, and uses `mf_wf_free_block` to preserve the state invariant. Disjoint live blocks retain their cells: the deleted interval cannot intersect any of their intervals. `mf_release_array_wp` applies this rule to the actual `Free(FieldLoad(array,1))` expression. `mf_cleanup_wp` composes the two final frees and result construction while framing ownership of the unchanged input array. The cleanup preserves the already selected scalar value; it reads no buffer cell after freeing a buffer.

Copying and initialization. For input A of length n, after j copy iterations in the CJR allocation model the value buffer is

$$\text{take}(j, A) +\!\!+ \text{replicate}(n - j, 0),$$

and the index buffer is $[0, \ldots, j-1]$ followed by $n - j$ zero cells. The zero suffix describes the modeled allocation. The emitted native routine writes every temporary cell before reading it, so this operation does not rely on `malloc` returning zeroed memory. The initialized prefix grows by one cell on each iteration. The loop condition supplies $j < n$, justifying the source read and both destination writes. Ownership of the input is retained throughout. At $j = n$, the buffers are exactly the two projections of `mf_pairs A 0`. `mf_copy_step_wp` proves the actual nested read/write step; `mf_copy_loop_wp` proves the loop by induction on $n - j$. `mf_copy_loop_is_code` is an expression equality obtained from `mf_body`, so a separate copy program cannot satisfy this proof in place of the emitted one.

Concrete run and winner scan. The pure helper `mf_scan_run value first suffix` returns the number of additional equal-value records and the minimum original index including the already visited records. It stops on an empty suffix or a different value. `mf_group_inner_wp` proves the actual guarded inner loop computes exactly this pair: if the loop starts at position j with minimum t, it finishes at $j + \text{extra}$ with the computed minimum. Its induction follows the unread suffix. An exhausted suffix implies $j = n$; the explicit conditional returns false without loading a cell. A nonempty suffix implies $j < n$, so the equality test is safe. On a matching record, the index read is safe, the conditional minimum update preserves its stack cell, and advancing the cursor decreases the unread suffix. A different value leaves the cursor and minimum unchanged. The theorem retains both arrays and both mutable stack cells in every case.

`mf_group_summary_decomposition` connects this forward scan to `mf_run_summaries`: for a first record p and suffix P, scanning the additional equal records produces the first summary $(p.\text{value}, 1 + \text{extra}, \text{minimum})$, and the remaining summaries are exactly those of the suffix after dropping those records. The proof combines adjacent summaries; count addition and minimum are associative. Thus the forward implementation and the tail-recursive functional grouping agree, without assuming that agreement as a precondition.

`mf_strict_better` uses a larger count, or an equal count and a strictly smaller first index, exactly as the native operation does. `mf_fold_best_sound` proves that the accumulated winner is one of the seed and processed summaries and dominates all of them in the score order. The seed has count zero. Every real summary has positive count, so a nonempty completed scan cannot select the seed. `mf_forward_scan_correct` combines this fact, completeness of all run summaries, and the score order to prove the specification.

The concrete update lemmas prove the nested count/tie comparison and all three assignments to the best-value, best-count, and best-first stack cells. `mf_group_step_wp` composes the first-record reads, creation of local end/minimum cells, inner loop, summary selection, and cursor update. It also returns the local stack ownership required for scope cleanup. `mf_group_outer_wp` proves the full outer loop by induction on a fuel at least the unread suffix length. Each iteration consumes at least its first record; its next cursor is in bounds and its remaining suffix is shorter. Its final best cells contain the exact `mf_forward_scan` result. Checked expression equalities link both loops to `mf_body`.

Complete Iris contract. The input predicate handles the empty header separately because the CJR raw-allocation model requires positive block lengths:

```
Definition mf_input `{!cjrGS Σ} (xs : list Z) oi : iProp Σ :=
  match xs with
  | [] => ObjId oi ↦o[0] LitV (LitInt 0)
  | _ :: _ => is_array xs (LitV (LitObj oi))
  end.
```

Here $\mapsto_o [0]$ denotes ownership of object field zero. The empty case requires only the zero length field because the operation reads no element or pointer and allocates nothing. The positive case owns the initialized input array, including its exact length, data pointer, raw-block token, and element cells. `mf_option_value` encodes `None` as the false/zero result structure and `Some x` as the true/x structure. The checked complete theorem is

$$\text{mfInput}(A, a) \;\vdash\; \text{WP mostFrequent}(a)\ \{v.\ \lceil v = \text{encode}(\text{mfSortScan}(A))\rceil * \text{mfInput}(A, a)\}.$$

```
Theorem mf_wp `{!cjrGS Σ} `{!invGS_gen HasLc Σ} xs oi :
  mf_input xs oi -∗
  WP App (Rec None (Some "input") mf_body) (Val (LitV (LitObj oi)))
    {{ v, ⌜v = mf_option_value (mf_sort_scan xs)⌝ ∗ mf_input xs oi }}.
```

The connective $-\!*$ is separating implication: it consumes the input ownership to establish the weakest-precondition assertion. The postcondition combines a pure result equality with ownership of the same input using separating conjunction. `mf_nonempty_wp` proves allocation of two positive-length buffers, applies the copy and quicksort theorems, initializes the winner and cursor cells, then applies the outer scan theorem. The actual final `Free` expressions consume both temporary raw buffers. The result is read from the best-value stack cell after cleanup. The input ownership is framed throughout and returned unchanged. The proof supplies all local stack cells for lexical scope cleanup. `most_frequent_spec_unique` shows that two answers satisfying the specification are equal: their maximum frequencies agree, their first indices agree, and the input cell at that index determines their value. It identifies the forward scan's answer with `mf_sort_scan`. `mf_wp` combines this nonempty proof with `mf_empty_wp`; `mf_spec_wp` adds the specification to its postcondition. No copying, grouping, comparator, or sorting hypothesis is assumed for this contract. The representative project assumption audit checks 73 theorems, including the complete operation and its kernels.

16.6 Testing and comparative benchmarks

Eleven Cangjie unit-test methods pass against the emitted source: empty, singleton, all-equal, all-distinct, interleaved two-way and three-way ties, negative and boundary integers, sorted and reversed inputs, exhaustive small arrays, and paired-sort preservation. The exhaustive test checks all 3,280 arrays of lengths zero through seven over $\{-1, 0, 1\}$ against an independent quadratic reference. Every result test also checks that the input is unchanged. The pair test checks lexicographic order, distinct original indices, and each value's correspondence to its original index. Tests check native behavior in addition to the CJR refinement proof; they do not prove the printer or native compiler correct.

The baselines are explicitly written equivalent operations using the SDK's `std.sort.sort` and `std.collection.HashMap`. They are not built-in most-frequent functions. The sort baseline sorts an array of original indices with a value/index comparator and scans equal-value runs. The HashMap baseline counts and maintains the maximum count in one pass, then visits the original input until it finds a value with that count, preserving first-occurrence semantics. All return the same option encoding, preserve the input, and check the empty case.

The measurements were rerun on 9 October 2026. The compiler is Cangjie 1.0.5 cjnative at `-O2`, targeting `aarch64-apple-darwin`, on an Apple M4 Mac mini with 16 GB RAM and macOS 26.6.2. Three independent sequential processes each record seven samples per implementation at each workload/size point: 693 measurements in total. Each operation receives identical preconstructed values. Timing includes its allocations, copying or map construction, sorting/counting, scan, result construction, and the CJR buffer frees. Input setup, warmups, checksums, and printing are outside the timer. Each implementation receives two warmup calls. Batch repetitions are $262144/n$ for all implementations; trial order rotates through the three implementations. There is no core pinning, forced garbage collection, or subtraction of loop overhead.

The deterministic distributions are $a_i = ((37i) \bmod 101) - 50$ (dense), $a_i = ((48271i) \bmod n) - n/2$ (distinct permuted values), and $a_i = ((17i) \bmod 7) - 3$ (seven values), at $n = 256, 4096, 16384$. Sorted and reverse-sorted distinct inputs are measured only at $n = 256$ to bound the cost of the deliberately reused quadratic worst case. These workloads are not random samples from all input distributions. Raw results, source hashes, compiler metadata, and reproducible runners are in `benchmark/results/most_frequent/` and `benchmark/most_frequent_run.py`.

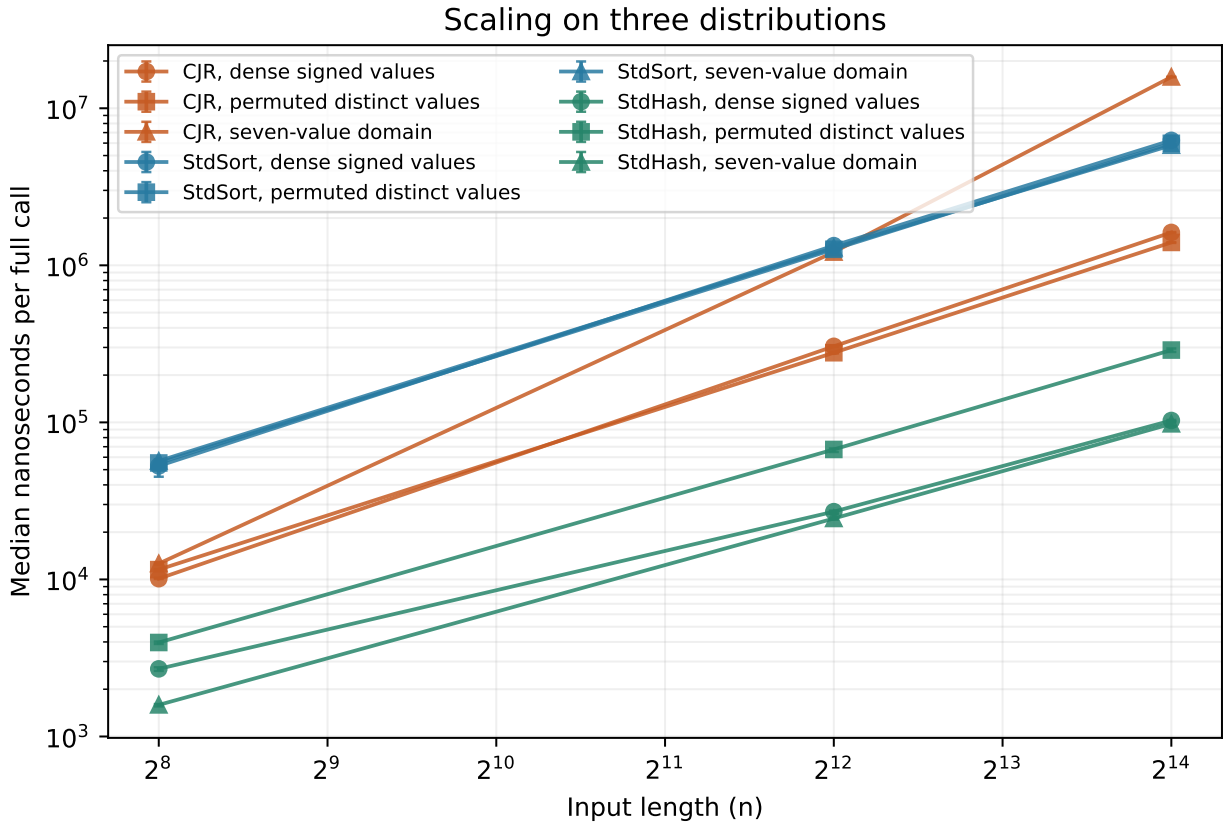


Figure 4: Complete-operation timings: median elapsed time per call; error bars show the interquartile range across 21 samples.

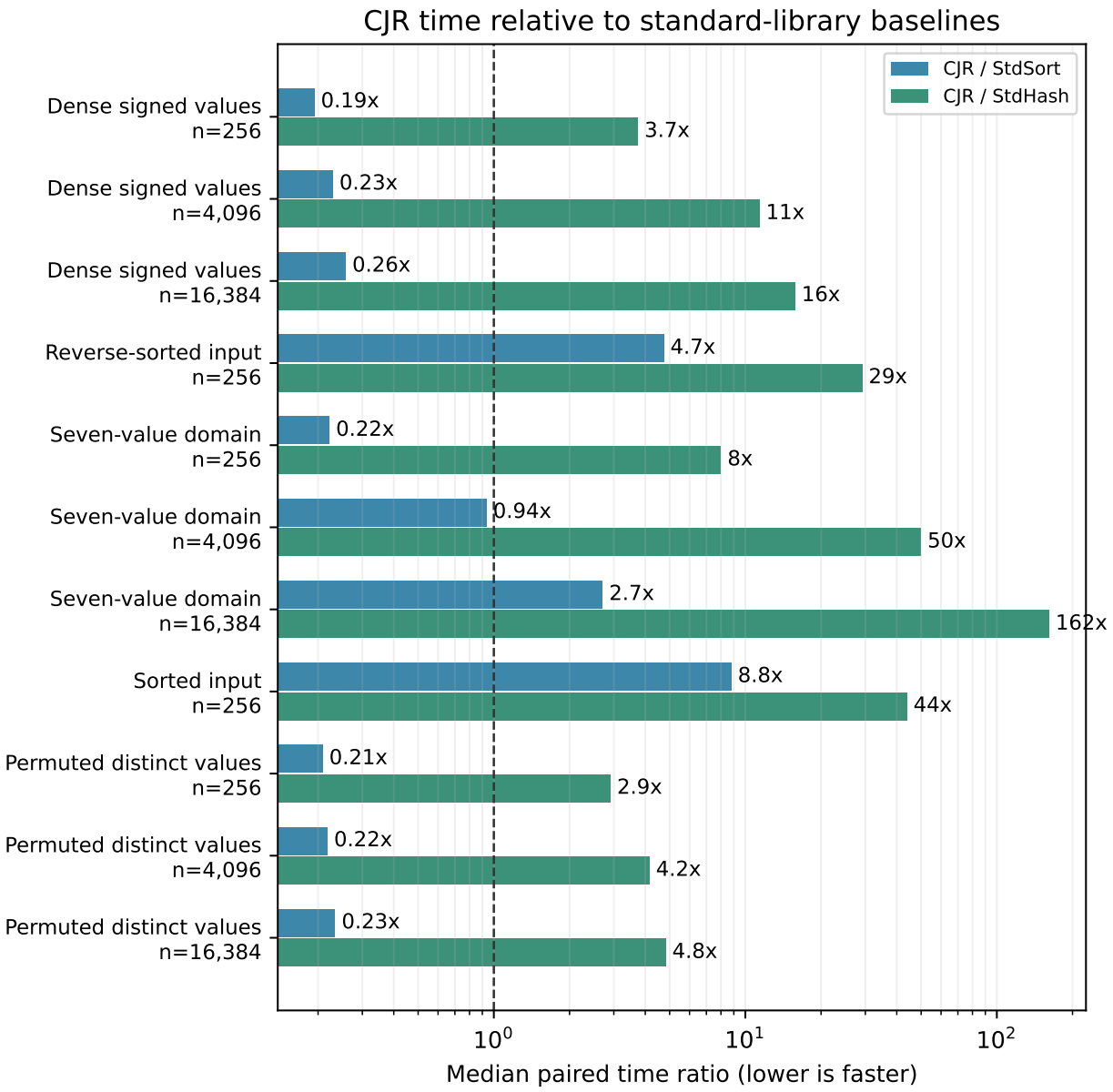


Figure 5: Median paired CJR/baseline ratios. Values above one mean CJR is slower.

Distribution	n	CJR (μs)	std.sort (μs)	HashMap (μs)
Dense signed values	256	10.11	52.78	2.70
Dense signed values	4,096	304.97	1,335.02	27.02
Dense signed values	16,384	1,620.19	6,247.12	102.81
Reverse-sorted input	256	108.45	22.64	3.74
Seven-value domain	256	12.61	56.72	1.59
Seven-value domain	4,096	1,213.67	1,291.91	24.44
Seven-value domain	16,384	15,819.06	5,813.62	97.88
Sorted input	256	174.20	19.58	3.96
Permuted distinct values	256	11.55	55.14	3.97
Permuted distinct values	4,096	277.62	1,264.59	67.23
Permuted distinct values	16,384	1,398.56	5,981.06	288.69

Table 1: Median time per complete operation; 21 samples per entry.

These measurements compare complete implementations with different representations and runtime costs. CJR uses unchecked raw accesses; the sort baseline uses standard arrays and a comparator that reads the input through their API. CJR allocates two buffers whereas that baseline allocates one index array. Consequently a speed difference does not isolate the sorting algorithm. The seven-value and already ordered cases expose the pivot strategy's weakness. The HashMap alternative avoids sorting and can be substantially faster, but relies on hashing behavior and a different verification effort. No timing establishes correctness, a general performance ranking, or worst-case bounds for the SDK's map.

17 What is deliberately missing

We deliberately omit concurrency from the semantics, so there is no invariant-opening rule and no fork. Float literals exist, yet no float operator does. Allocation of a positive word count is the rule used by the array library. Freeing in the named logic rules remains the one-word rule. Objects are not collected. Pointer arithmetic is word-addressed. Null, a freed address, and an address with no points-to fact are stuck rather than given a defined error value.

The type grammar is not enforced. The adequacy theorem is the safety theorem for those programs whose specifications have been proved.

18 Conclusion

This experiment made substantially more progress than expected, largely because of the power and usability of Iris. In less than one week of work, conducted at varying levels of intensity with several models, we developed a formal language fragment, its operational semantics and program logic, and verified examples ranging from elementary operations to a HashMap. Iris made it practical to reason compositionally about mutable state and to connect algorithmic correctness arguments to executable implementations.

Does this mean that we can generate code and use it without inspecting it? Perhaps, within carefully established boundaries. Formal verification shifts much of the inspection effort towards the specification: does the formal statement actually express the intended informal requirement? An agent can help make that correspondence accessible by presenting the Coq specification alongside an equivalent mathematical formulation and an English readout. These presentations give reviewers several ways to examine what has been proved, although their agreement with the formal statement also requires scrutiny.

Testing and benchmarking of the generated Cangjie provide additional sanity checks. We also run the Coq development through compilation, independent kernel checking with `coqchk`, scans for `Admitted`, `Axiom` and cheating mechanisms, and an audit of theorem assumptions. These checks strengthen confidence that the stated results have genuine proofs. They cannot establish that the statements capture every relevant requirement, or that the generated program faithfully implements the formal model.

Those boundaries still require human judgement. As discussed in Appendix A, a specification can omit an expected behaviour, impose an impractical precondition, or establish functional correctness without a useful performance guarantee. A further obligation is the correspondence between the formal CJR core and real-world Cangjie execution. The relevant Cangjie behaviour is described informally in its specification documents, and the current printer does not come with a proved preservation theorem. Consequently, the guarantees established for CJR transfer to generated Cangjie only insofar as that correspondence holds.

Array-list access operations rely on valid-index preconditions rather than runtime checks. The proofs establish safe memory access for calls covered by those preconditions. Calls that violate them and discrepancies in target behavior remain outside that guarantee.

The performance gap has related causes. The benchmark discussion and Appendix A explain how algorithm choices and the restricted language fragment affect the results. Better algorithms can be specified explicitly and verified, but some limitations are inherent in the available operations and representations. Modest language extensions could improve the situation; each extension, however, requires corresponding operational semantics, Iris rules and supporting proofs. Richer exceptional control, resumable effects, and concurrency are plausible next steps, with established Iris developments providing foundations rather than ready-made proofs for CJR.

Proving that invalid inputs raise a specified exception, and reasoning about recovery, would require defined exceptional outcomes, propagation through calls, and specifications for both normal and exceptional results. Cleanup and the ownership available to a handler would have to be included.

For general effect handlers, Hazel [13] provides an Iris-based logic for mutable state and shallow and deep handlers with one-shot continuations. Its effect protocols describe the operations a computation may perform and the replies its handler must provide. TesLogic [14] extends this line of work to dynamically allocated effect labels. These are mechanized precedents for a CJR logic with resumable effects, but adopting their ideas would require formalizing CJR's own handler and continuation semantics, and supplying a supported implementation or encoding in the Cangjie target.

Concurrency is already a central purpose of Iris: the framework supplies invariants and ownership reasoning for shared state, and HeapLang includes fork and atomic operations [9]. CJR would need thread creation and completion, synchronization primitives, and rules for transferring resources between threads. A first development could run operations on disjoint arrays, then protect the existing HashMap with one mutex, reusing its sequential contracts under exclusive ownership. ReLoC [15] provides mechanized relational reasoning for fine-grained concurrency and logical atomicity; Cosmo [16] addresses Multicore OCaml's weak memory model; and Actris [17] provides protocols for message-passing concurrency. These developments offer useful methods, but their language and memory-model assumptions must not be silently attributed to Cangjie. Safety proofs alone would not establish termination, deadlock freedom, or lock-freedom.

Effect handlers and concurrency are compatible in a single Iris-based development. Blaze [18] combines handlers, mutable state, primitive concurrency, and one-shot and multi-shot continuations, and relates handler-based asynchronous libraries to implementations using actual concurrency. This does not mean that two separate extensions can simply be joined unchanged. A combined CJR semantics must account for ownership captured by continuations, ownership transfer when

a continuation is resumed on another thread, restrictions preventing multiple resumptions from duplicating exclusive resources, and cleanup when a continuation is discarded or an exception escapes. Suspension while holding a lock also requires explicit contracts. A conservative development sequence would introduce ordinary exceptions, thread creation and joining, and mutexes first, followed by one-shot resumable effects; lock-free structures and multi-shot effects would be more demanding follow-up work.

The experiment therefore suggests that this fragment can support the construction of complex software module by module. The verified HashMap is significant evidence: it is a demanding mutable data structure whose correctness depends on interacting representation invariants and algorithms. Achieving competitive performance and idiomatic target code across a broader range of applications will require further development of the language core and its verification infrastructure. Control flow and concurrency remain substantial extensions of CJR, but the Iris literature shows that both, including their combination, have established formal foundations. The remaining work is to adapt those foundations to this language and justify their correspondence with Cangjie execution.

19 Reading list

The entries below are the sources this note relies on. The formal bibliography follows the same order.

The language. The Cangjie 1.0 documentation [1] is the surface syntax from which this fragment is cut: `func`, `let` and `var`, `if` and `while`, `struct`, non-`open` `class`, and `CPointer`. The current cut keeps unboxed structs, word-addressed raw blocks, and final classes. Generics and inheritance require lowering into that cut; concurrency is currently excluded and would require the semantic and logical extensions discussed in the conclusion.

Separation logic. Reynolds [2] is the logic of disjoint ownership of mutable structure, which is the reading of $\mapsto_s$, $\mapsto_r$, $\mapsto_b$, and $\mapsto_o$ in this development. O'Hearn, Reynolds, and Yang [3] is the local-reasoning paper: a specification mentions only the cells the command uses, and the frame rule supplies the rest. That is the justification of the framing steps in the examples and in `cons_spec`.

Iris. The POPL 2015 paper [4] introduces the resource algebra that the four ghost maps instantiate. The ESOP 2017 paper [5] is the one to read for the program logic used here: weakest preconditions, evaluation contexts, the bind rule, and the lifting of a deterministic head step. *Iris from the ground up* [6] is the self-contained account of the same logic, including adequacy. The Iris Proof Mode paper [7] explains the `iApply`, `iDestruct`, and `iFrame` style of the Coq proofs. The lecture notes [8] are the shortest path through all of that with exercises. HeapLang, in the Iris 4.4.0 Coq development [9], is the language whose `ectxi_language` instance this one copies; `ghost_map` is the library behind the points-to facts.

The proof assistant. The mechanization is checked with Coq 8.20.1 [10] and std++ 1.12.0 [11]. std++ supplies the finite maps, the list lemmas, and the `decide` tactic used for the pure arithmetic steps.

Possible extensions. The Cangjie 1.0.5 exception guide [12] documents exception operations that a future extension could target. Hazel [13] and TesLogic [14] provide foundations for effect-handler reasoning. ReLoC [15], Cosmo [16], and Actris [17] address concurrent refinement, weak memory,

and message passing respectively. Blaze [18] demonstrates a mechanized combination of effect handlers and primitive concurrency. These are references for future extensions, not components already integrated into CJR.

A Limitations and target-code risks

The proofs in this tutorial establish properties of the CJR programs under the operational semantics defined in Coq. They do not establish that every behavior of the separately printed Cangjie program agrees with that semantics. In particular, the printer is a string-producing function, and the generated Cangjie is not extracted from Coq or verified against the CJR semantics. The following cases identify concrete places where that gap matters. They should be read alongside the earlier specifications: some are documented limitations of the examples, while others are mismatches between the formal allocation model and the generated runtime code.

A.1 Fresh memory is zero in the proofs, but not in generated code

The CJR allocation rule used in the examples initializes every allocated word to zero. For example, `array.make` is specified to return an array containing `n` zeros, and its proof obtains those zero-valued points-to facts directly from allocation. The HashMap initialization proofs similarly depend on freshly allocated tag and value buffers containing zeros. The generated printer, however, translates `Alloc` to a call to `malloc` and does not emit a loop that clears the words. The C and POSIX `malloc` interfaces do not promise zero-filled storage; the initial values are unspecified. Consequently, a generated array read immediately after construction, a HashMap operation that inspects an uninitialized tag, or the generated `bumpKeep` example's read of its newly allocated word can observe a nonzero value. That would violate the corresponding CJR postcondition (and in the last example, the proved result zero).

The current runtime tests pass on the tested Cangjie 1.0.5 environment, but that does not establish that its allocator always returns zeroed memory. An attempted allocator-reuse test did not reproduce stale contents in that environment; it is a useful negative result, not a guarantee supplied by `malloc`. The generated programs also do not check for allocation failure before dereferencing the returned pointer. A target-level repair needs to zero-initialize allocated words (or otherwise explicitly establish every initial value required by the representation) and handle allocation failure consistently. The CJR allocation semantics and their proofs would then need to be related to that emitted initialization behavior.

A.2 HashMap growth does not honor its minimum-capacity argument

The method named `grow(min)` doubles the current capacity once and does not inspect `min`. This is also what the formal body and its specification prove: the resulting capacity is exactly twice the old capacity. A runtime call `grow(64)` on a fresh map of capacity 16 therefore produces capacity 32, not 64; a standalone test of that expected minimum-capacity behavior fails. The checked-in growth test asks for 32, which is satisfied by the single doubling.

This behavior is documented by the example and its tutorial discussion, so it is not a hidden discrepancy between the Coq proof and the generated method. It is nevertheless a contract limitation relative to the usual meaning of a `grow(min)` operation and to the companion ArrayList example, which chooses at least the requested minimum. If HashMap is required to honor that conventional contract, both the formal body and its specification need to be strengthened, and a test where `min` is greater than twice the old capacity should be added.

A.3 HashMap residue computation can take impractical time

The generated residue routine normalizes a key by repeatedly adding or subtracting the capacity until the value is in range. This is mathematically finite for the supported positive capacities, and the proof establishes the result and range properties. It does not establish a useful execution time bound. At capacity 16, an extreme `Int64` key can require on the order of 2^{59} loop iterations. Thus a valid lookup or insertion with such a key may appear to hang, even though ordinary negative keys and the tested collision cases complete quickly. The current tests exercise moderate negative values, not the extremes. A practical implementation should use a bounded-time remainder operation or another logarithmic/bounded normalization, with proofs that preserve the nonnegative bucket range and the map's key-to-slot invariant.

These limitations do not invalidate the Coq proofs of the stated CJR specifications. They delimit what those proofs and the current runtime tests establish about the generated Cangjie: target allocation behavior is not yet shown equivalent to CJR allocation, the HashMap growth contract is intentionally narrower than a minimum-capacity contract, and the residue proof is a functional result rather than a performance guarantee.

References

[1] Cangjie documentation, version 1.0.0. `https://docs.cangjie-lang.cn/en/docs/1.0.0/`.

[2] John C. Reynolds. Separation logic: A logic for shared mutable data structures. In *Proceedings of LICS*, pages 55–74. IEEE, 2002.

[3] Peter W. O'Hearn, John C. Reynolds, and Hongseok Yang. Local reasoning about programs that alter data structures. In *Proceedings of CSL*, volume 2142 of *LNCS*, pages 1–19. Springer, 2001.

[4] Ralf Jung, David Swasey, Filip Sieczkowski, Kasper Svendsen, Aaron Turon, Lars Birkedal, and Derek Dreyer. Iris: Monoids and invariants as an orthogonal basis for concurrent reasoning. In *Proceedings of POPL*, pages 637–650. ACM, 2015.

[5] Robbert Krebbers, Ralf Jung, Aleš Bizjak, Jacques-Henri Jourdan, Derek Dreyer, and Lars Birkedal. The essence of higher-order concurrent separation logic. In *Proceedings of ESOP*, volume 10201 of *LNCS*, pages 696–723. Springer, 2017.

[6] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018.

[7] Robbert Krebbers, Amin Timany, and Lars Birkedal. Interactive proofs in higher-order concurrent separation logic. In *Proceedings of POPL*, pages 205–217. ACM, 2017.

[8] Lars Birkedal and Aleš Bizjak. Lecture notes on Iris: Higher-order concurrent separation logic. `https://iris-project.org/pdfs/iris-lecture-notes.pdf`.

[9] The Iris Project. The Iris Coq development, version 4.4.0, including HeapLang. `https://gitlab.mpi-sws.org/iris/iris`.

[10] The Coq Development Team. The Coq proof assistant, version 8.20.1. `https://coq.inria.fr/`.

[11] The Iris Project. coq-std++, version 1.12.0. `https://gitlab.mpi-sws.org/iris/stdpp`.

[12] Cangjie development guide, version 1.0.5. Throwing and handling exceptions (Chinese-language edition). https://docs.cangjie-lang.cn/docs/1.0.5/dev-guide/source_zh_cn/error_handle/handle.html.

[13] Paulo Emílio de Vilhena and François Pottier. A separation logic for effect handlers. *Proceedings of the ACM on Programming Languages*, 5(POPL), article 33, pages 1–28, 2021. https://doi.org/10.1145/3434314.

[14] Paulo Emílio de Vilhena and François Pottier. A type system for effect handlers and dynamic labels. In *Proceedings of ESOP*, volume 13990 of *LNCS*, pages 225–252. Springer, 2023. https://doi.org/10.1007/978-3-031-30044-8_9.

[15] Dan Frumin, Robbert Krebbers, and Lars Birkedal. ReLoC Reloaded: A mechanized relational logic for fine-grained concurrency and logical atomicity. *Logical Methods in Computer Science*, 17(3), article 9, pages 1–59, 2021. https://doi.org/10.46298/LMCS-17(3:9)2021.

[16] Glen Mével, Jacques-Henri Jourdan, and François Pottier. Cosmo: A concurrent separation logic for Multicore OCaml. *Proceedings of the ACM on Programming Languages*, 4(ICFP), article 96, pages 1–29, 2020. https://doi.org/10.1145/3408978.

[17] Jonas Kastberg Hinrichsen, Jesper Bengtson, and Robbert Krebbers. Actris 2.0: Asynchronous session-type based reasoning in separation logic. *Logical Methods in Computer Science*, 18(2), article 16, pages 1–64, 2022. https://doi.org/10.46298/LMCS-18(2:16)2022.

[18] Paulo Emílio de Vilhena, Simcha van Collem, Ines Wright, and Robbert Krebbers. A relational separation logic for effect handlers. *Proceedings of the ACM on Programming Languages*, 10(POPL), article 34, pages 981–1009, 2026. https://doi.org/10.1145/3776676. Rocq formalization: https://github.com/DeVilhena-Paulo/blaze.